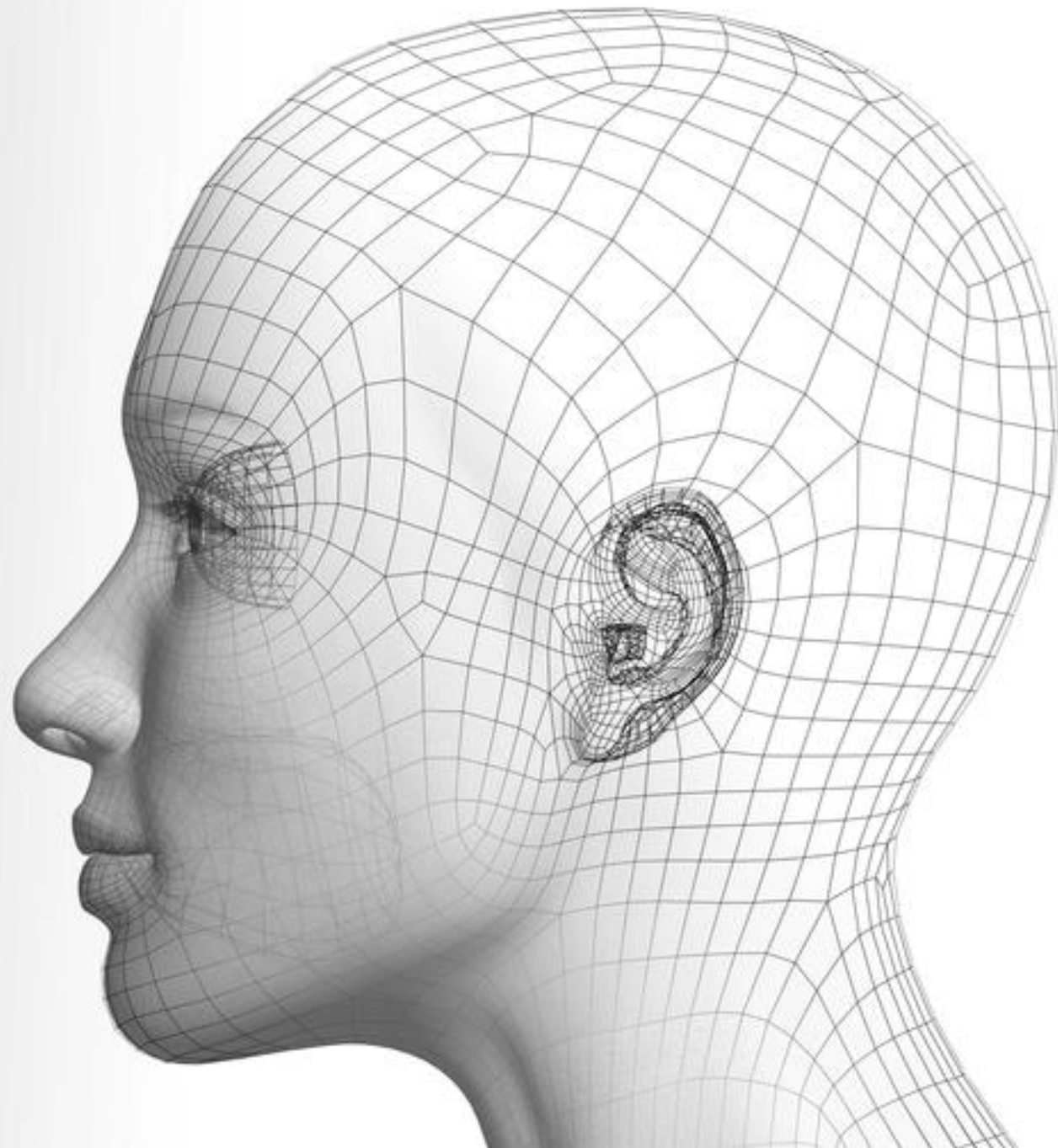


Image Segmentation

Ziwei Liu

刘子纬

<https://liuziwei7.github.io/>



Slide Credits

- Justin Johnson, EECS 498/598
- David Fouhey, EECS 442
- Paper Authors

Outline

Semantic Segmentation:

- Fully Convolutional Network
- Skip Connections
- Spatial Contexts

Instance Segmentation:

- Object Detection
- Mask R-CNN
- Joint Mask+X Prediction

Open Problems:

Deep Learning



What society thinks I do



What my friends think I do



What other computer scientists think I do



What mathematicians think I do



What I think I do

```
from theano import *
```

What I actually do

Many Structured Prediction Tasks

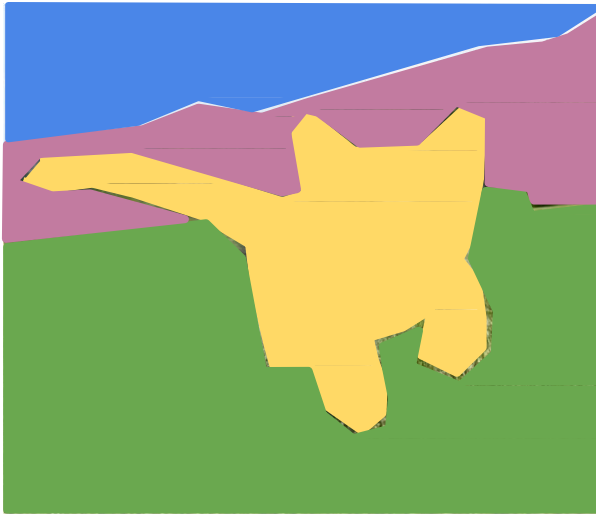
Classification



CAT

No spatial extent

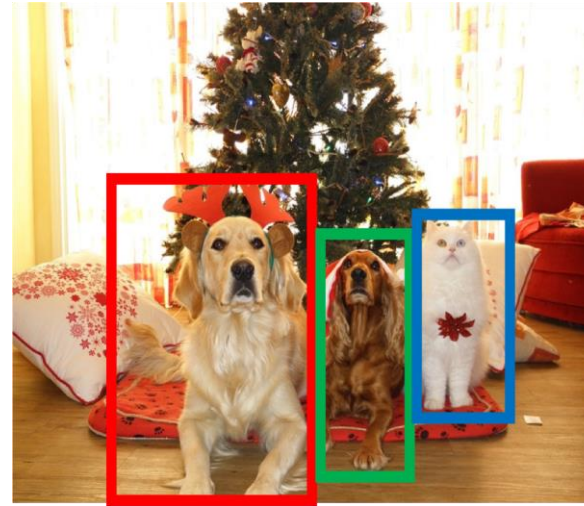
Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation



DOG, DOG, CAT

Part I:

Semantic Segmentation

Structured Prediction Tasks: Semantic Segmentation

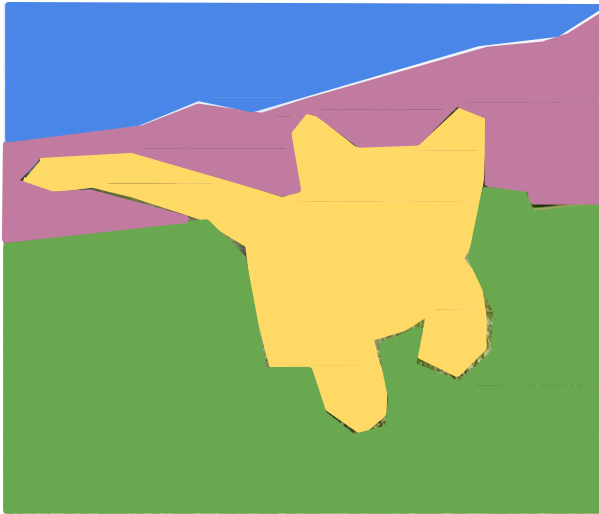
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation

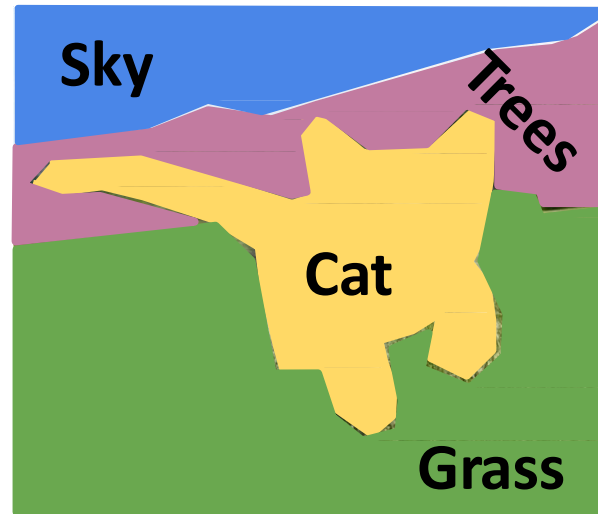


DOG, DOG, CAT

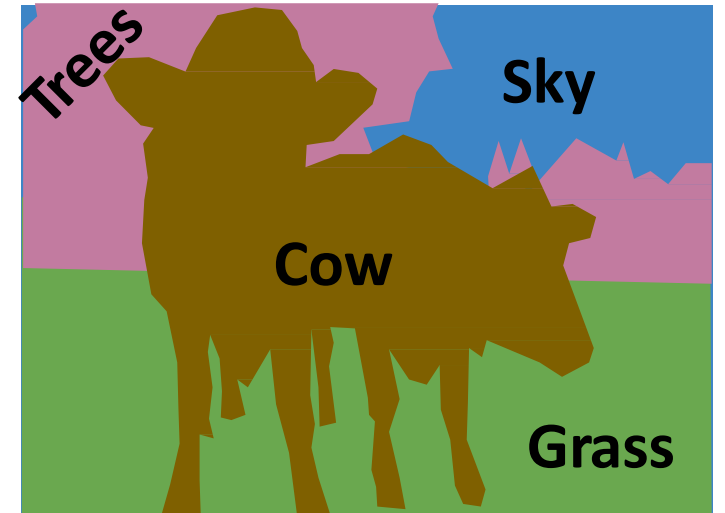
Semantic Segmentation

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels

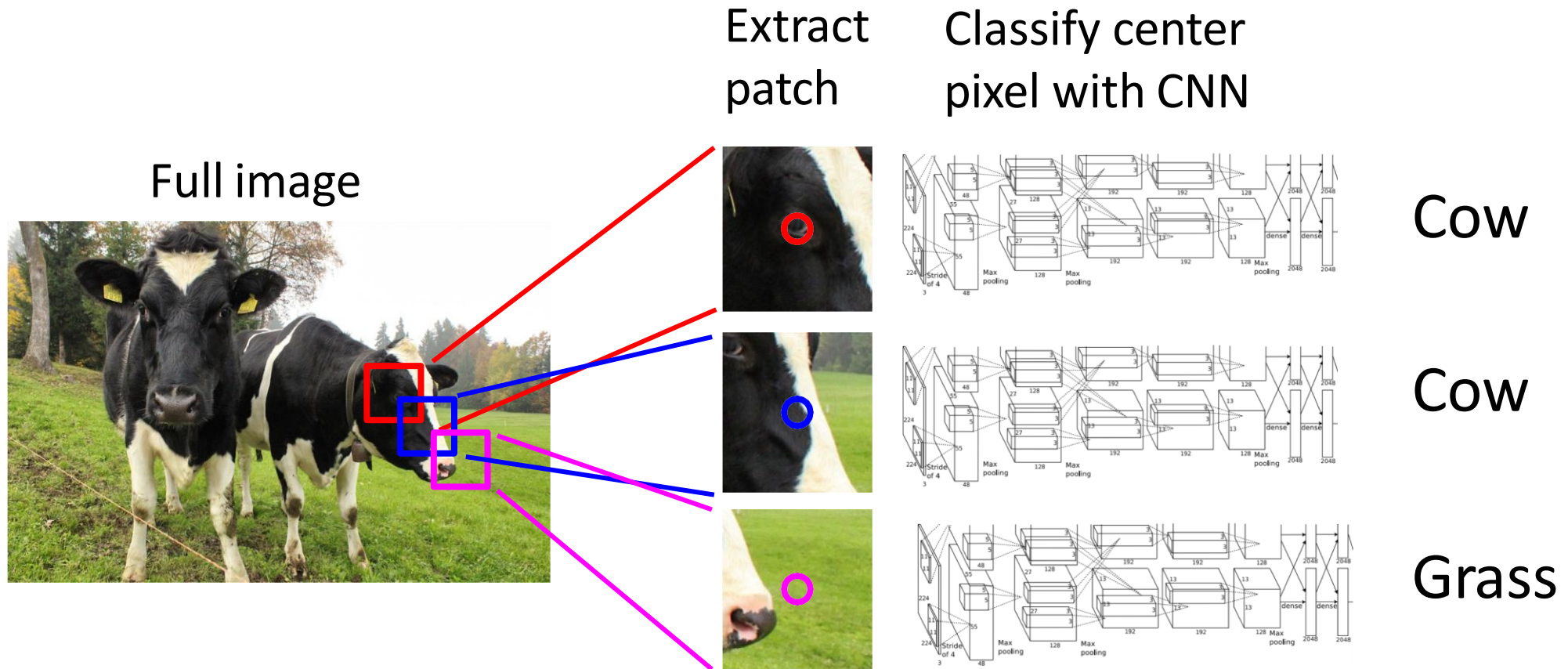


[This image is CC0 public domain](#)



Fully Convolutional Network

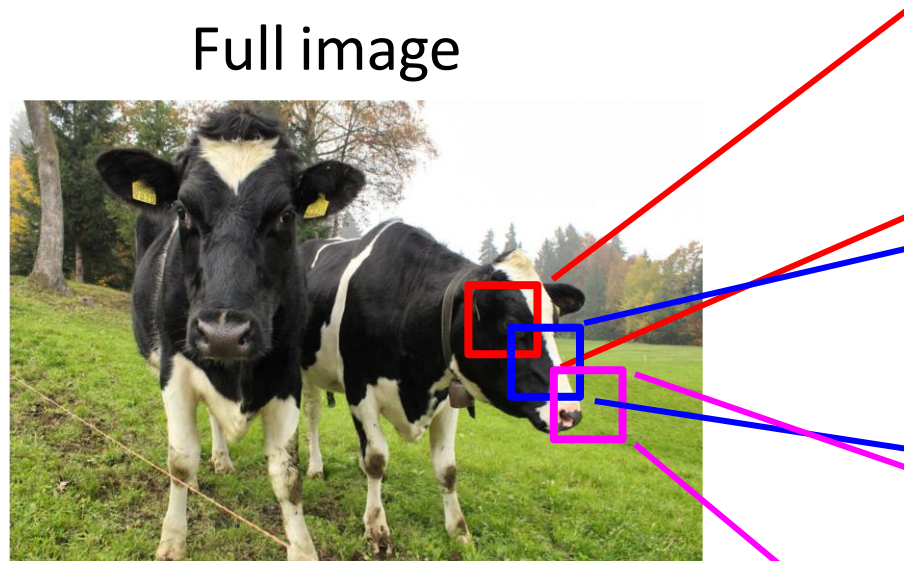
Semantic Segmentation Idea: Sliding Window



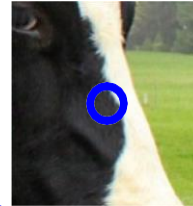
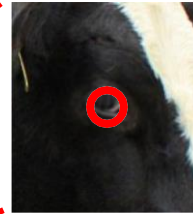
Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013

Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

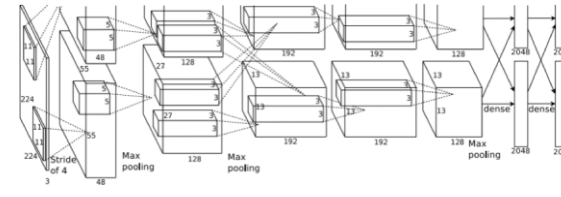
Semantic Segmentation Idea: Sliding Window



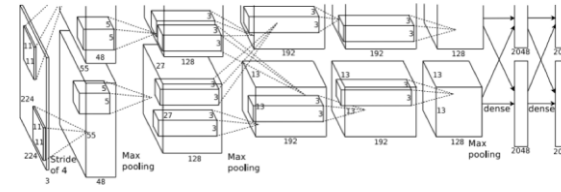
Extract
patch



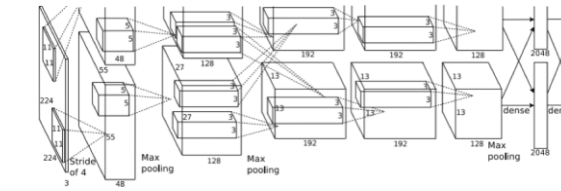
Classify center
pixel with CNN



Cow



Cow



Grass

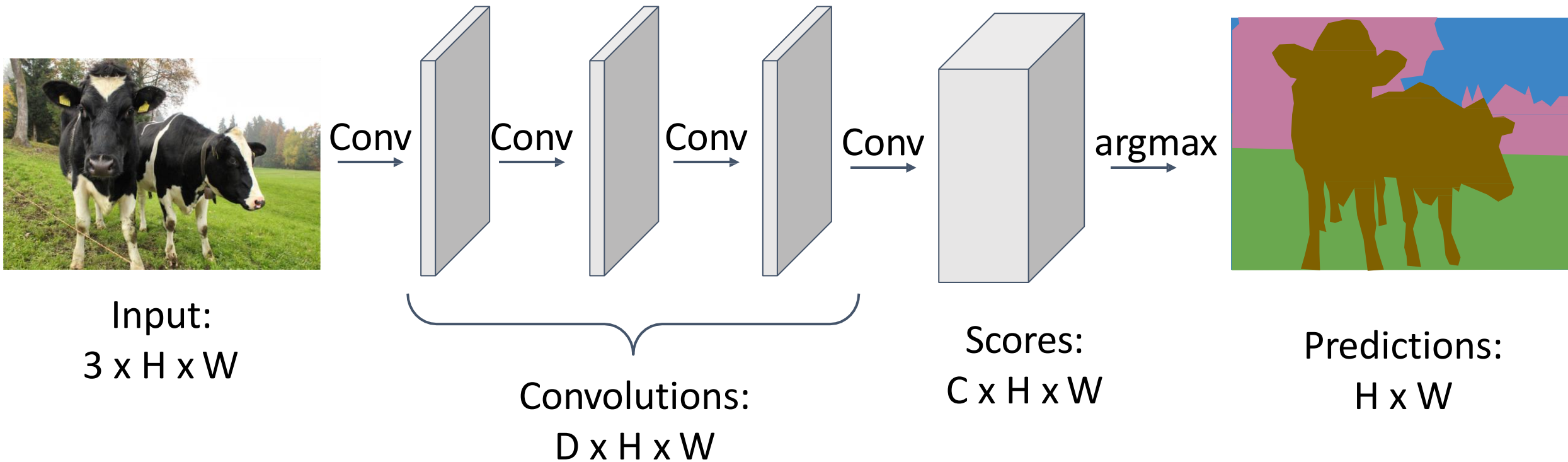
Problem: Very inefficient! Not
reusing shared features
between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

“Fully” Convolution

Fully Convolutional Network

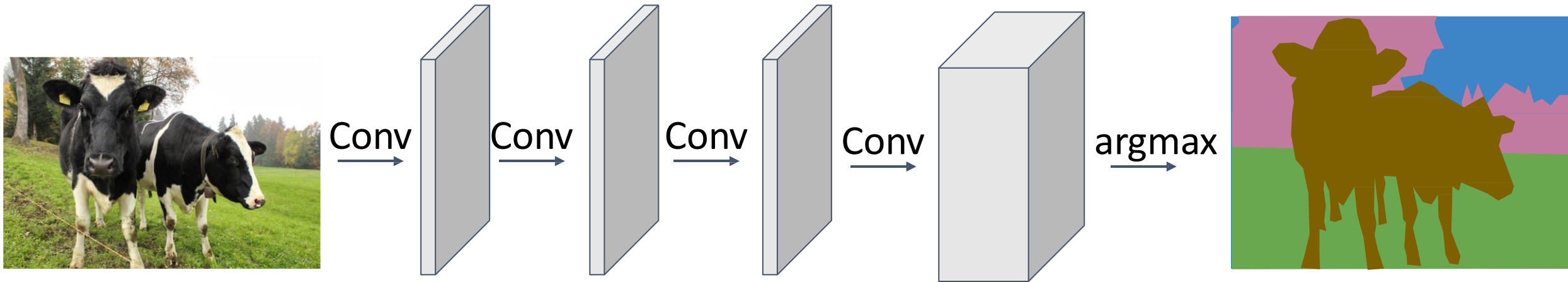
Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Loss function: Per-Pixel cross-entropy

Fully Convolutional Network

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!

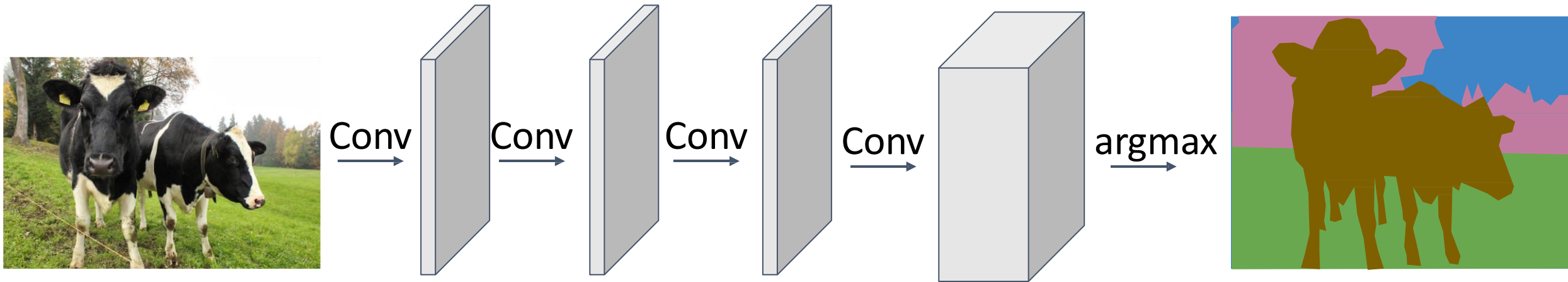


Input:
 $3 \times H \times W$

Problem #1: Effective receptive field size is linear in number of conv layers: With L 3×3 conv layers, receptive field is $1 + 2L$

Fully Convolutional Network

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!

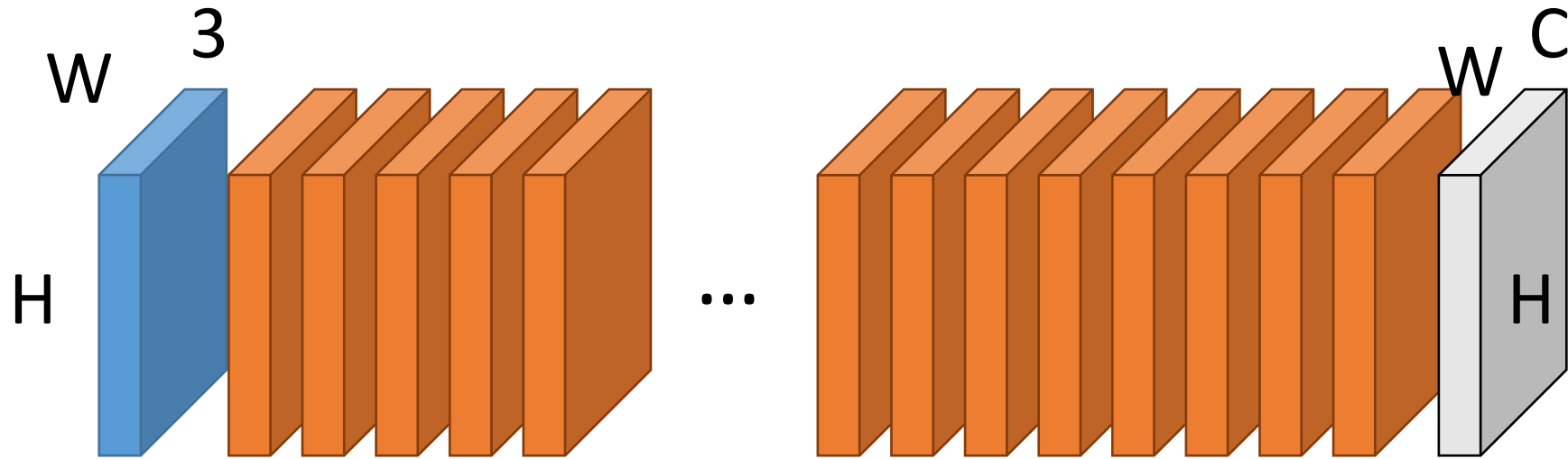


Input:
 $3 \times H \times W$

Problem #1: Effective receptive field size is linear in number of conv layers: With L 3×3 conv layers, receptive field is $1 + 2L$

Problem #2: Convolution on high res images is expensive! Recall ResNet stem aggressively downsamples

Why Not Stack Convolutions?

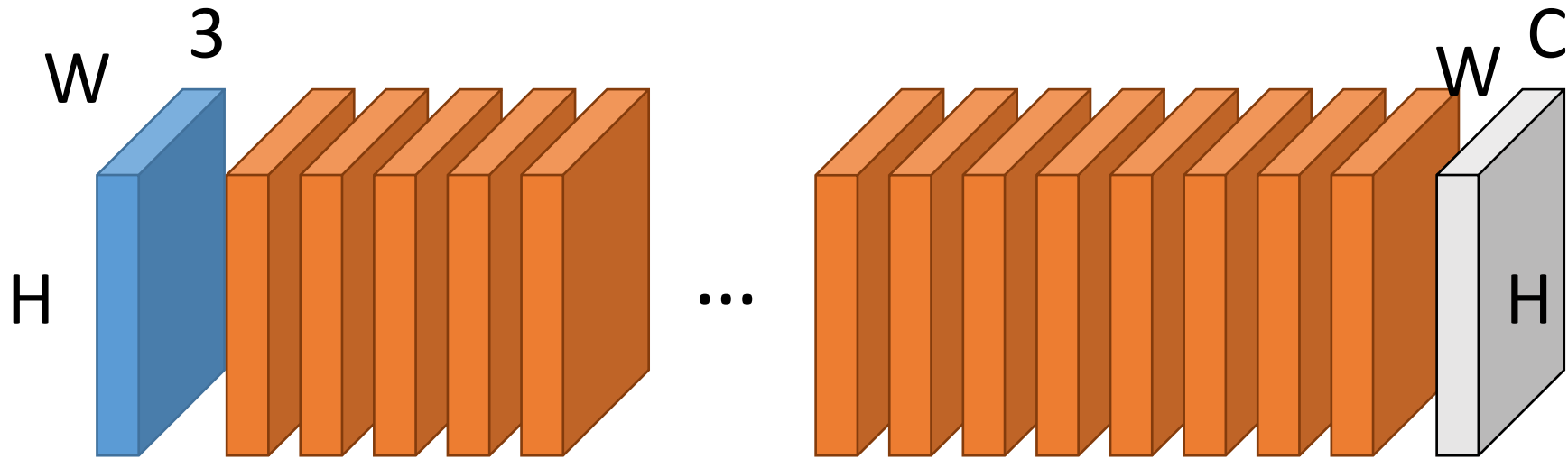


n 3×3 convs have a receptive field of $2n+1$ pixels

How many convolutions until ≥ 200 pixels?

100

Why Not Stack Convolutions?



Suppose 200 3x3 filters/layer, $H=W=400$

Storage/layer/image: $200 * 400 * 400 * 4 \text{ bytes} = 122\text{MB}$

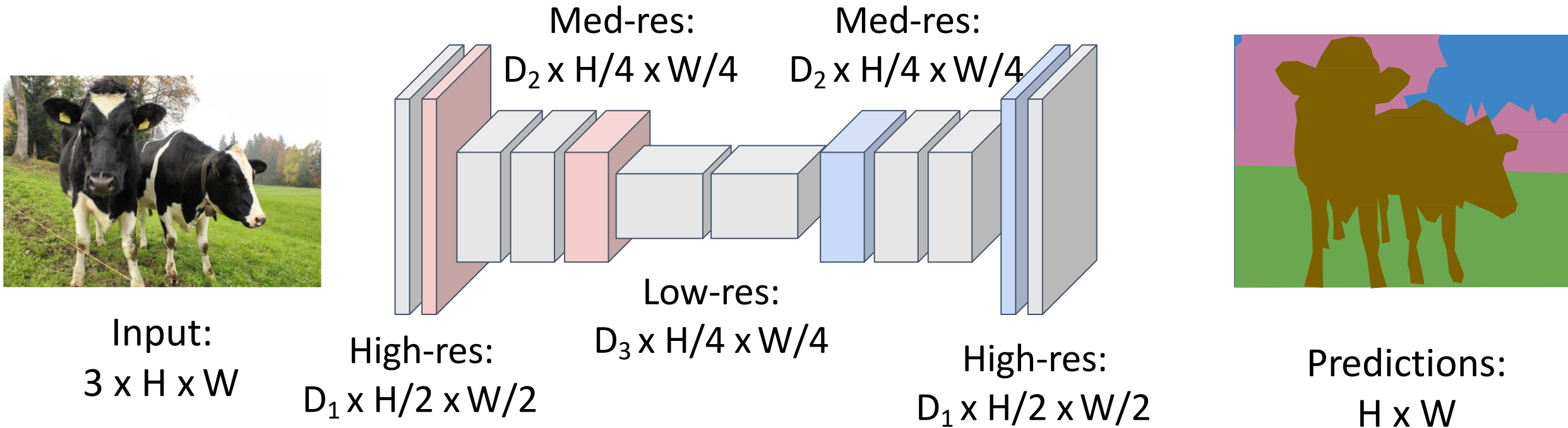
Uh oh!*

*100 layers, batch size of 20 = 238GB of memory!

Downsampling and Upsampling

Fully Convolutional Network

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Fully Convolutional Network

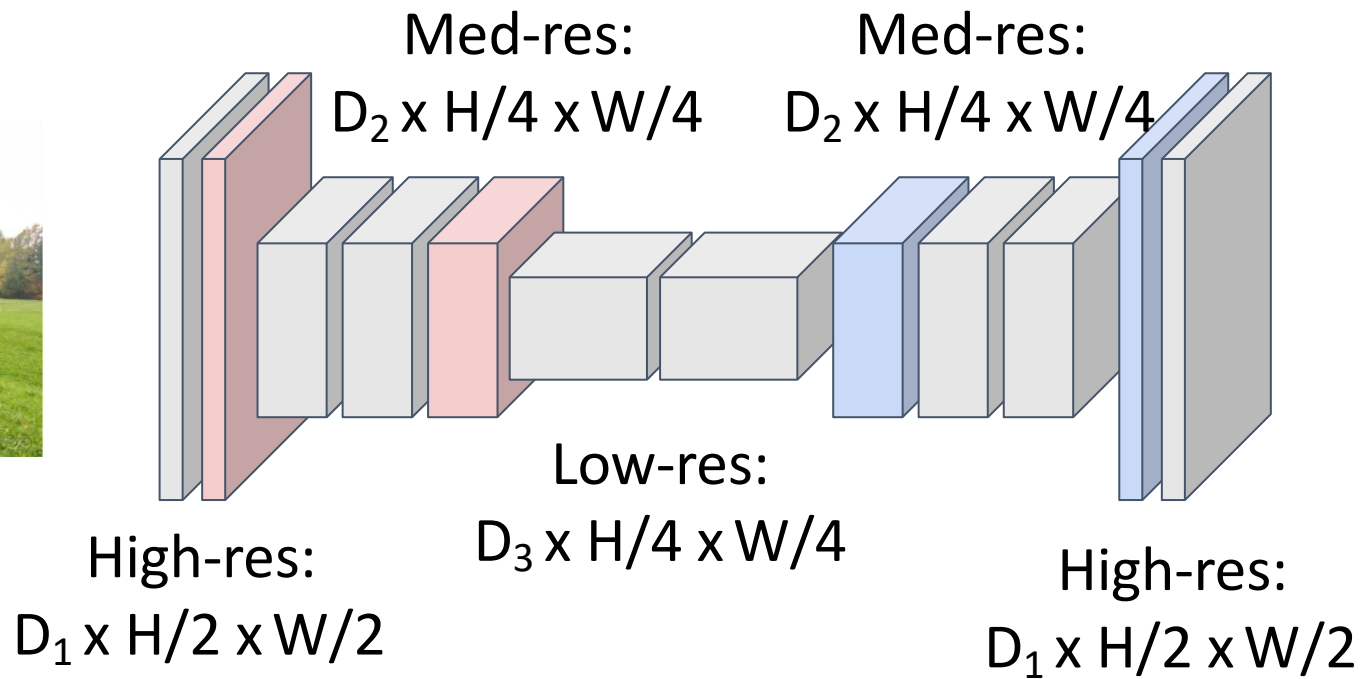
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling:
???



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

In-Network Upsampling: “Unpooling”

Bed of Nails

1	2
3	4



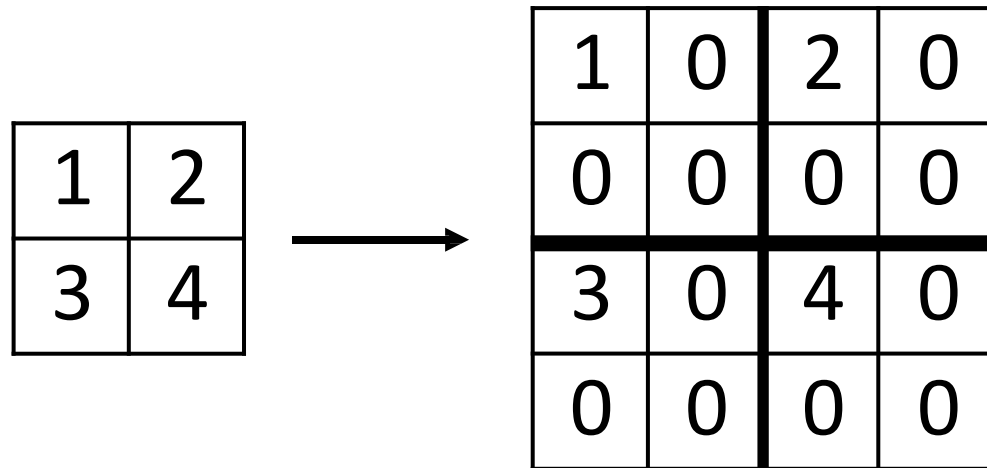
1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

In-Network Upsampling: “Unpooling”

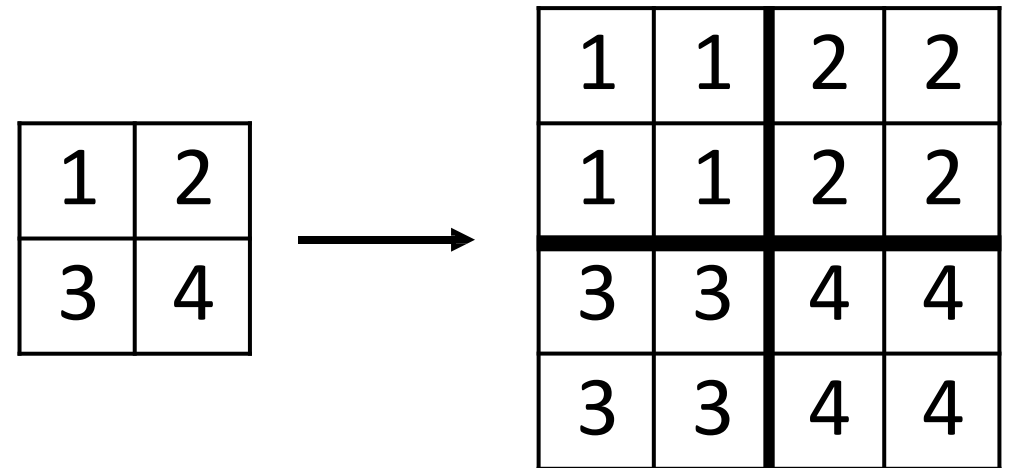
Bed of Nails



Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

Nearest Neighbor



Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

In-Network Upsampling: Bilinear Interpolation

1	
2	
3	
4	



1.00	1.25	1.75	2.00
1.50	1.75	2.25	2.50
2.50	2.75	3.25	3.50
3.00	3.25	3.75	4.00

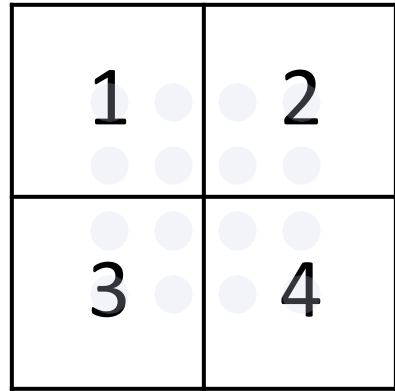
Input: C x 2 x 2

Output: C x 4 x 4

$$f_{x,y} = \sum_{i,j} f_{i,j} \max(0, 1 - |x - i|) \max(0, 1 - |y - j|) \quad \begin{array}{l} i \in \{\lfloor x \rfloor - 1, \dots, \lceil x \rceil + 1\} \\ j \in \{\lfloor y \rfloor - 1, \dots, \lceil y \rceil + 1\} \end{array}$$

Use two closest neighbors in x and y
to construct linear approximations

In-Network Upsampling: Bicubic Interpolation



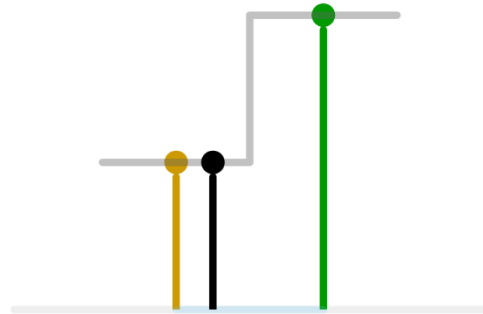
0.68	1.02	1.56	1.89
1.35	1.68	2.23	2.56
2.44	2.77	3.32	3.65
3.11	3.44	3.98	4.32

Input: $C \times 2 \times 2$

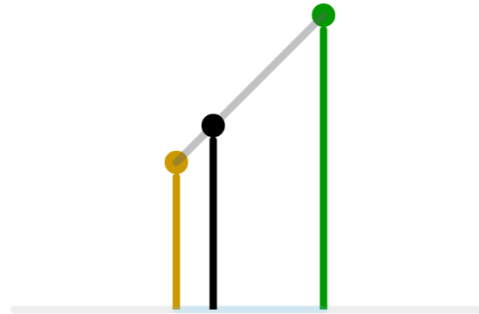
Output: $C \times 4 \times 4$

Use **three** closest neighbors in x and y to
construct **cubic** approximations
(This is how we normally resize images!)

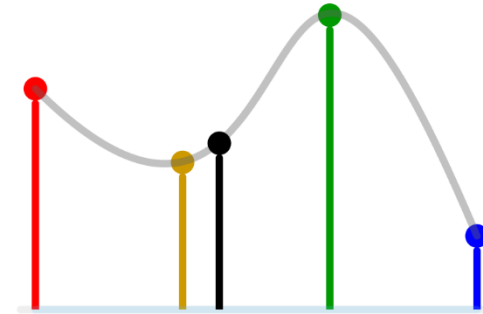
In-Network Upsampling



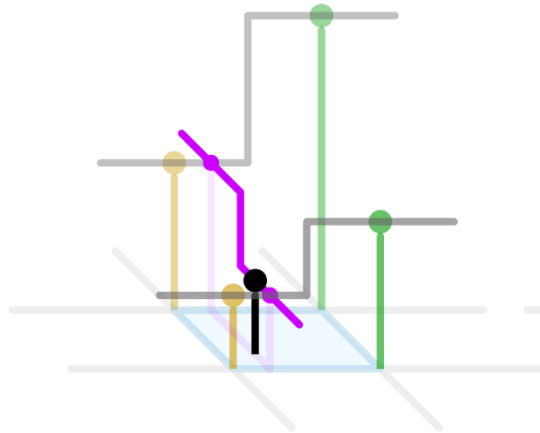
1D nearest-neighbour



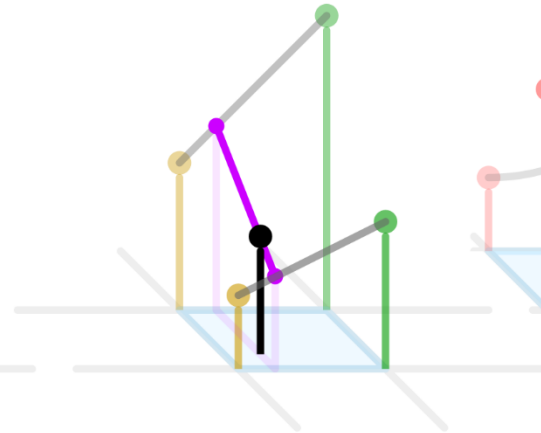
Linear



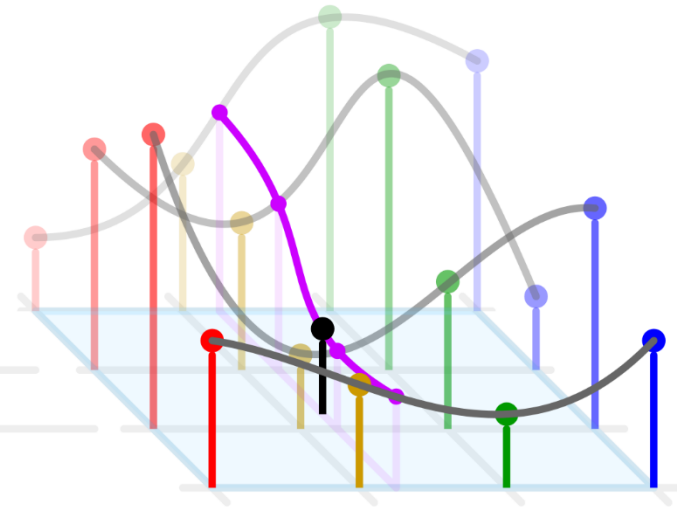
Cubic



2D nearest-neighbour



Bilinear



Bicubic

In-Network Upsampling: “Max Unpooling”

Max Pooling: Remember which position had the max

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8



5	6
7	8



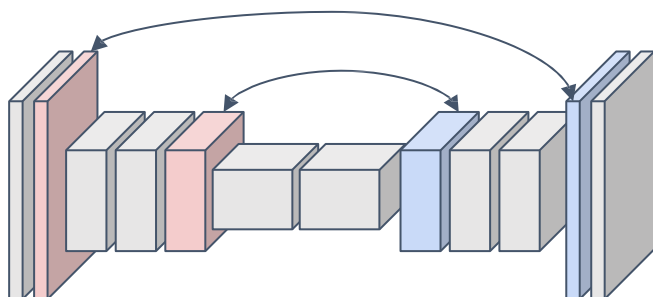
Rest
of
net



1	2
3	4



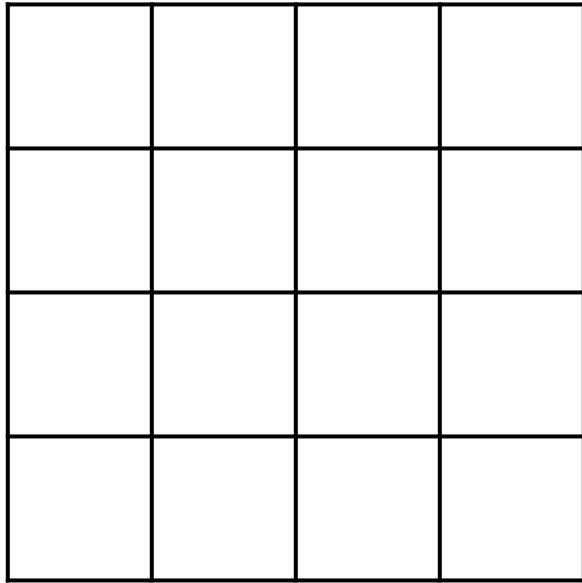
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4



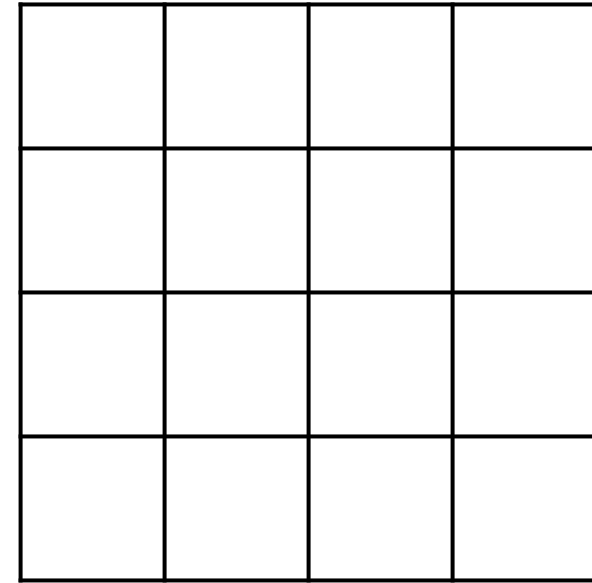
Pair each downsampling layer
with an upsampling layer

Learnable Upsampling: Transposed Convolution

Recall: Normal 3 x 3 convolution, stride 1, pad 1



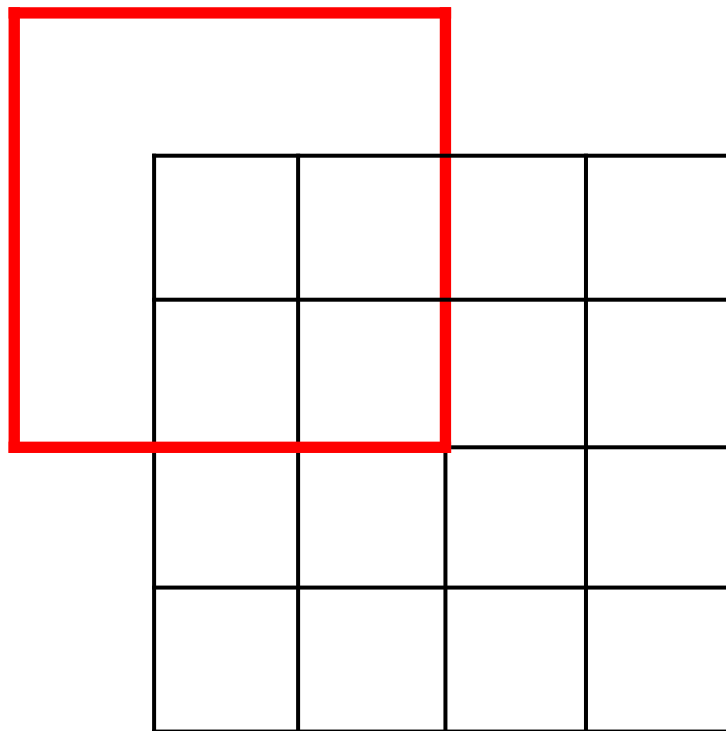
Input: 4 x 4



Output: 4 x 4

Learnable Upsampling: Transposed Convolution

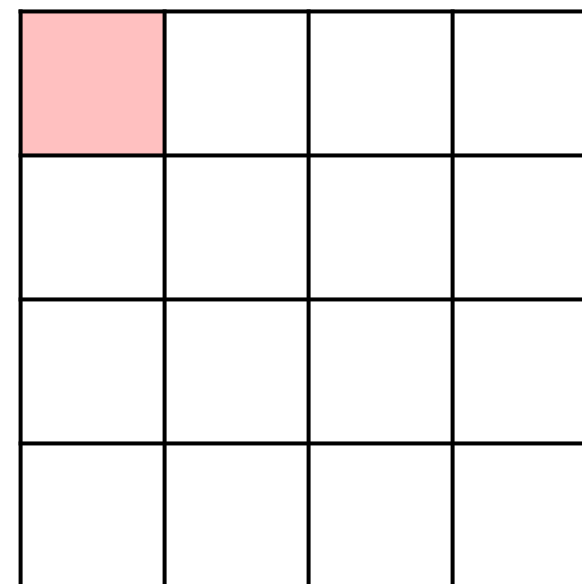
Recall: Normal 3 x 3 convolution, stride 1, pad 1



Input: 4 x 4



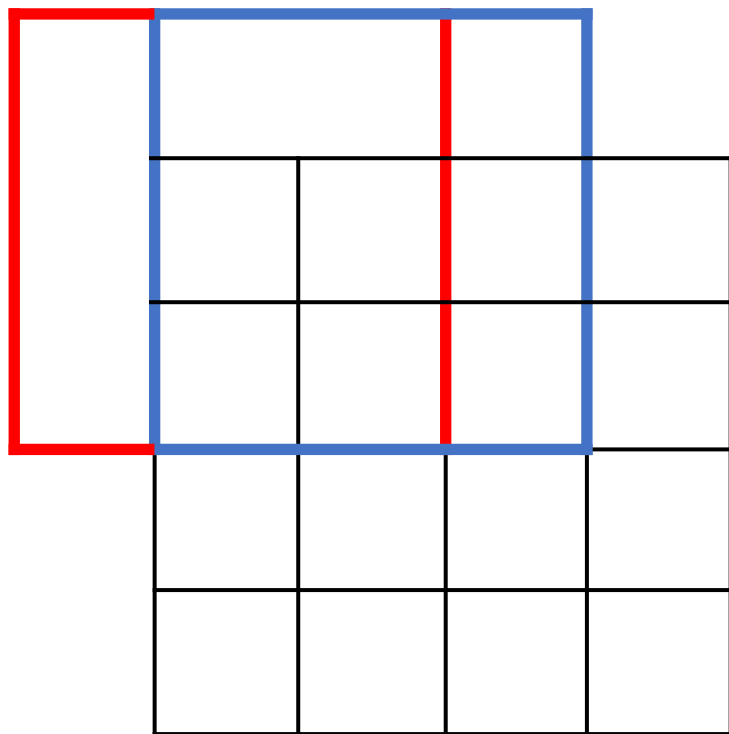
Dot product
between input
and filter



Output: 4 x 4

Learnable Upsampling: Transposed Convolution

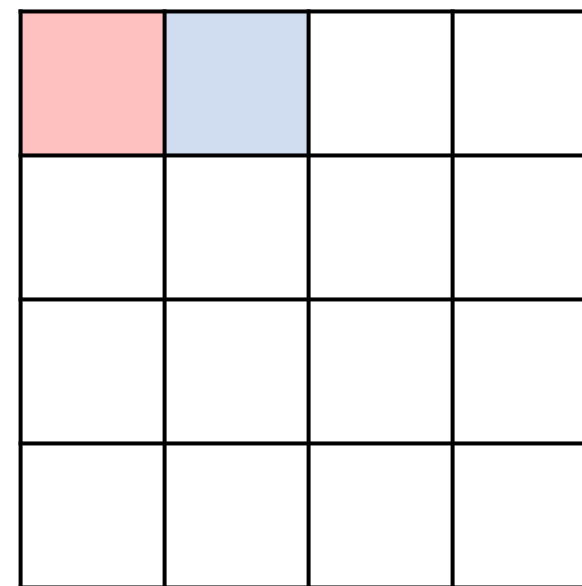
Recall: Normal 3 x 3 convolution, stride 1, pad 1



Input: 4 x 4



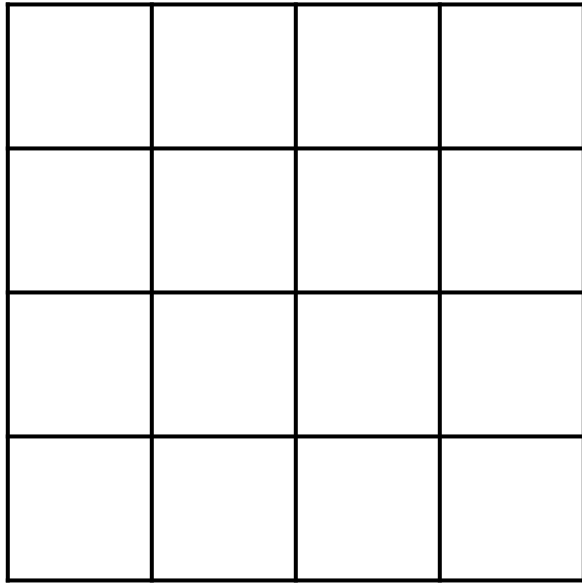
Dot product
between input
and filter



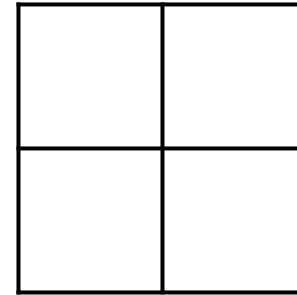
Output: 4 x 4

Learnable Upsampling: Transposed Convolution

Recall: Normal 3 x 3 convolution, stride 2, pad 1



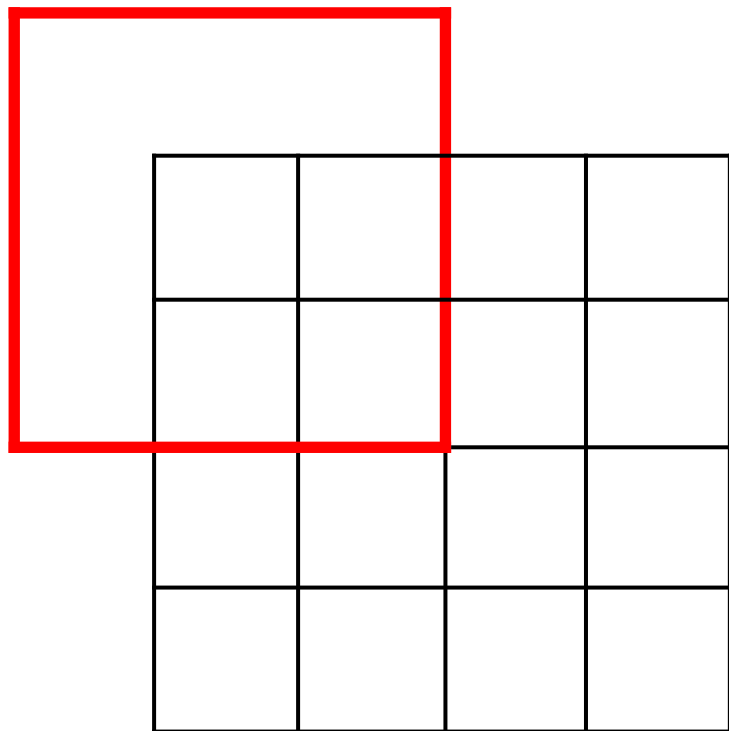
Input: 4 x 4



Output: 2 x 2

Learnable Upsampling: Transposed Convolution

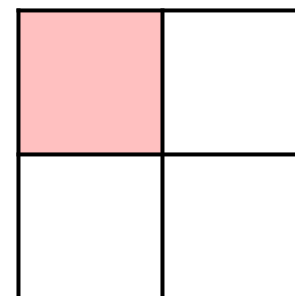
Recall: Normal 3 x 3 convolution, stride 2, pad 1



Input: 4 x 4



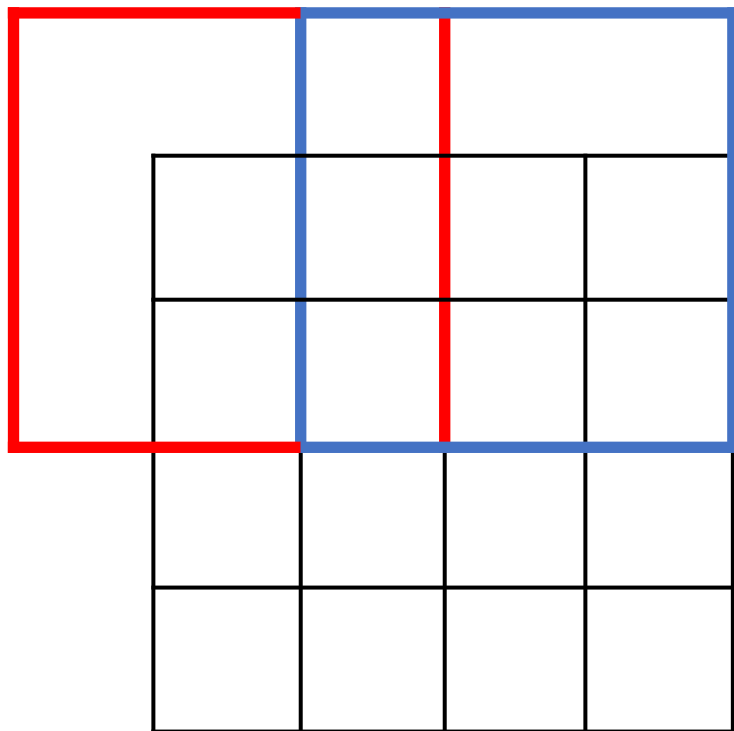
Dot product
between input
and filter



Output: 2 x 2

Learnable Upsampling: Transposed Convolution

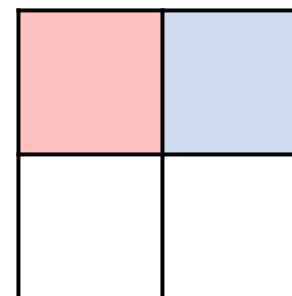
Recall: Normal 3 x 3 convolution, stride 2, pad 1



Input: 4 x 4



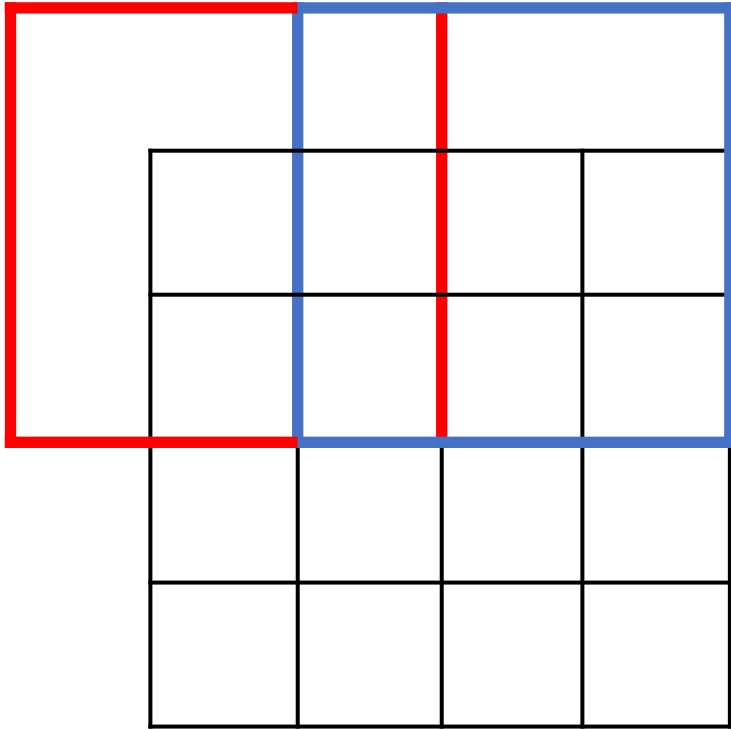
Dot product
between input
and filter



Output: 2 x 2

Learnable Upsampling: Transposed Convolution

Recall: Normal 3 x 3 convolution, stride 2, pad 1

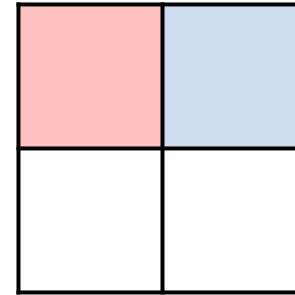


Input: 4 x 4

Convolution with stride > 1 is “Learnable Downsampling”
Can we use stride < 1 for “Learnable Upsampling”?



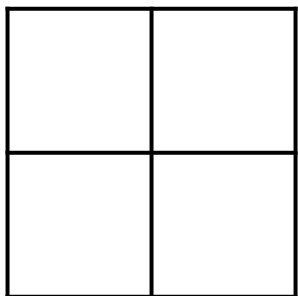
Dot product
between input
and filter



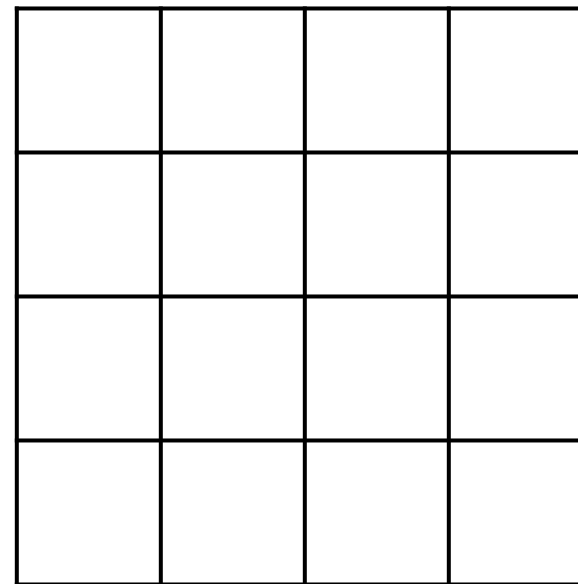
Output: 2 x 2

Learnable Upsampling: Transposed Convolution

3 x 3 convolution transpose, stride 2



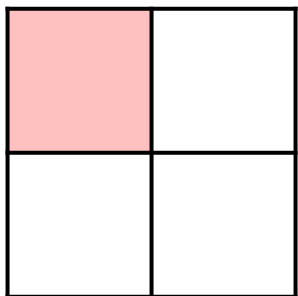
Input: 2 x 2



Output: 4 x 4

Learnable Upsampling: Transposed Convolution

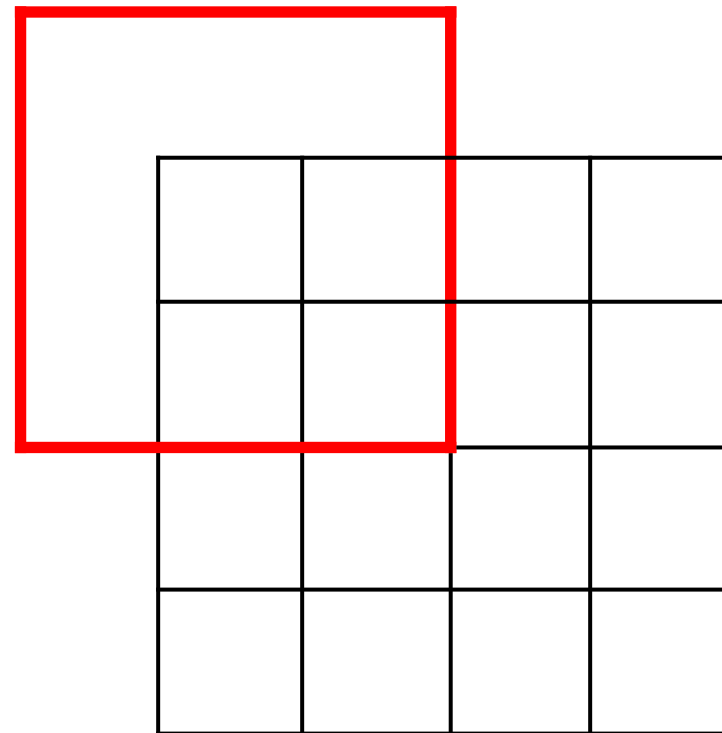
3 x 3 convolution transpose, stride 2



Input: 2 x 2



Weight filter by
input value and
copy to output

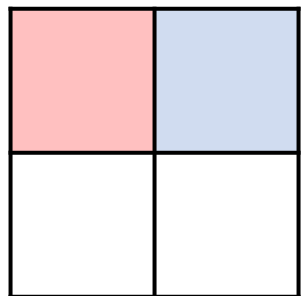


Output: 4 x 4

Learnable Upsampling: Transposed Convolution

3 x 3 **convolution transpose**, stride 2

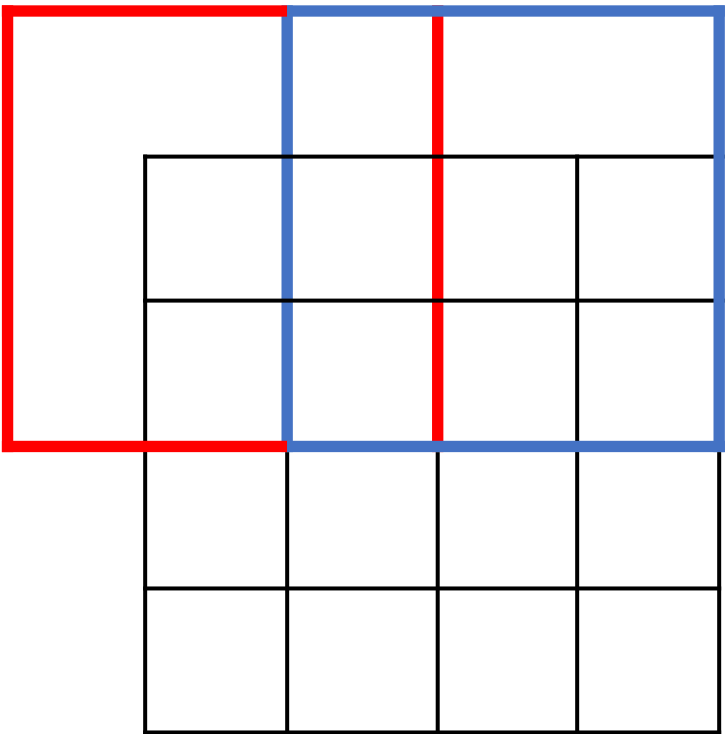
Filter moves 2 pixels in output
for every 1 pixel in input



Input: 2 x 2



Weight filter by
input value and
copy to output

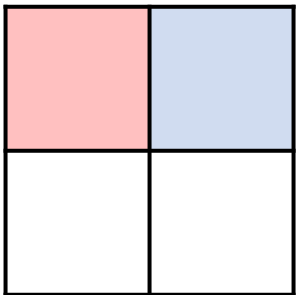


Output: 4 x 4

Learnable Upsampling: Transposed Convolution

3 x 3 **convolution transpose**, stride 2

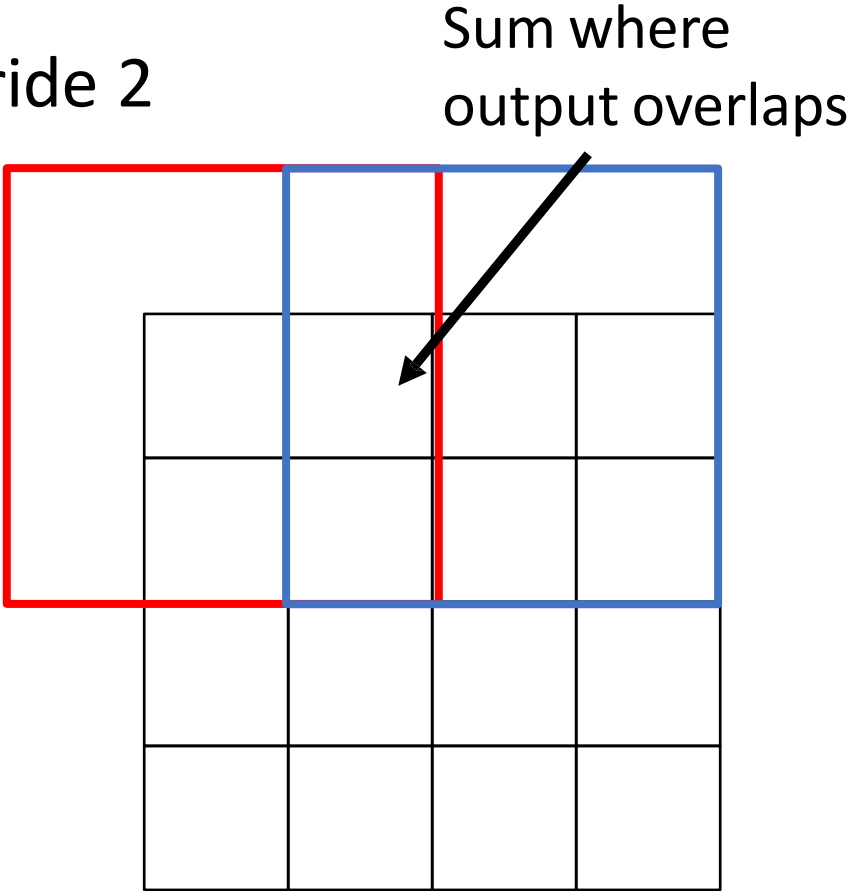
Filter moves 2 pixels in output
for every 1 pixel in input



Input: 2 x 2



Weight filter by
input value and
copy to output

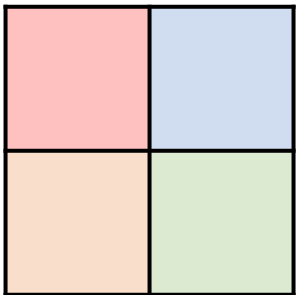


Output: 4 x 4

Learnable Upsampling: Transposed Convolution

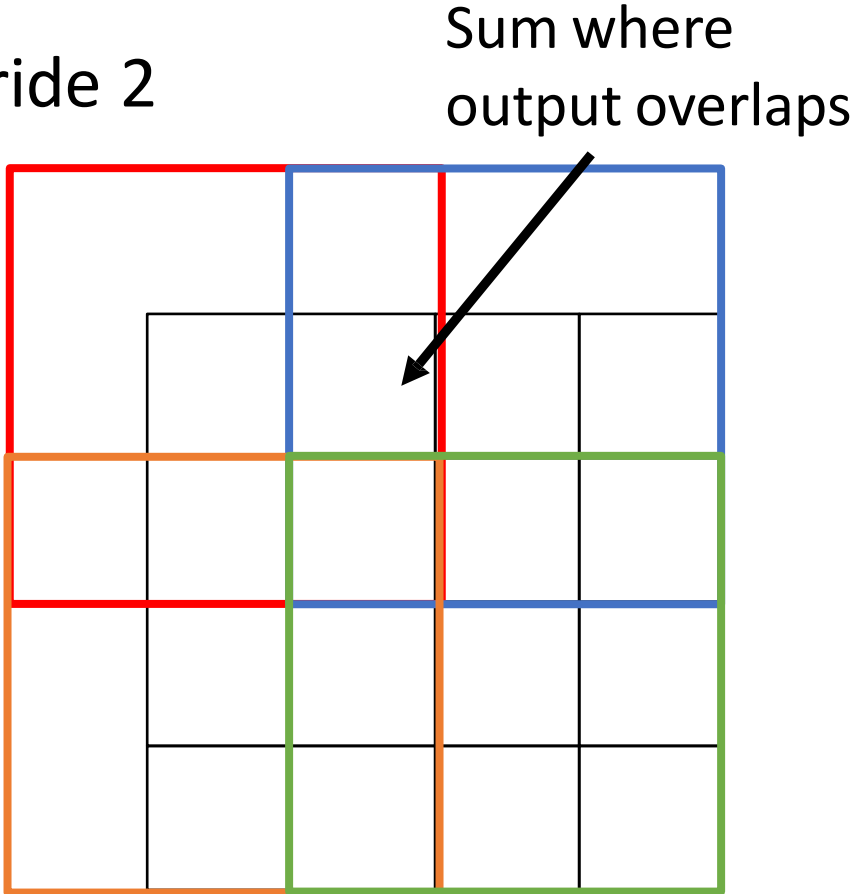
3 x 3 **convolution transpose**, stride 2

This gives 5x5 output – need to trim one pixel from top and left to give 4x4 output



Input: 2 x 2

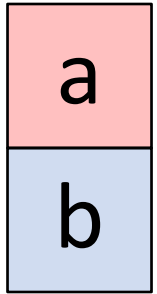
→
Weight filter by
input value and
copy to output



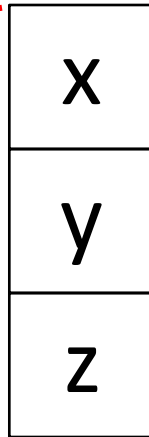
Output: 4 x 4

Transposed Convolution: 1D example

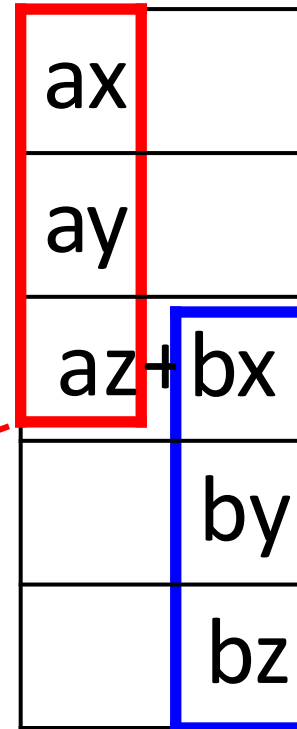
Input



Filter



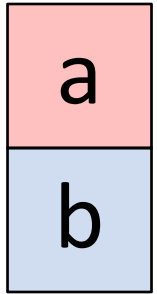
Output



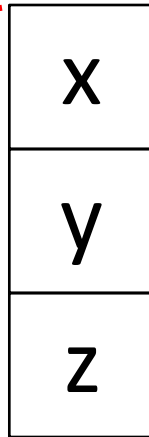
- Output has copies of filter weighted by input
- Stride 2: Move 2 pixels output for each pixel in input
- Sum at overlaps

Transposed Convolution: 1D example

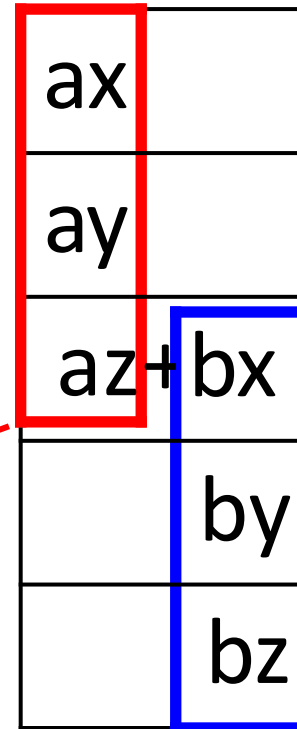
Input



Filter



Output



This has many names:

- Deconvolution (bad)!
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution
- Transposed Convolution (best name)

Transposed Convolution

Convolution

Filter: little lens that looks at a pixel.

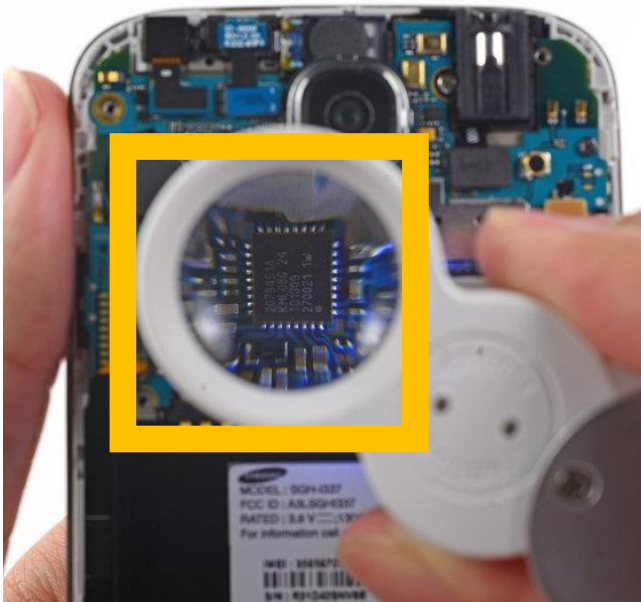


Image credit: ifixit.com, thespruce.com

Transposed Conv.

Filter: tiles used to make image



Fully Convolutional Network

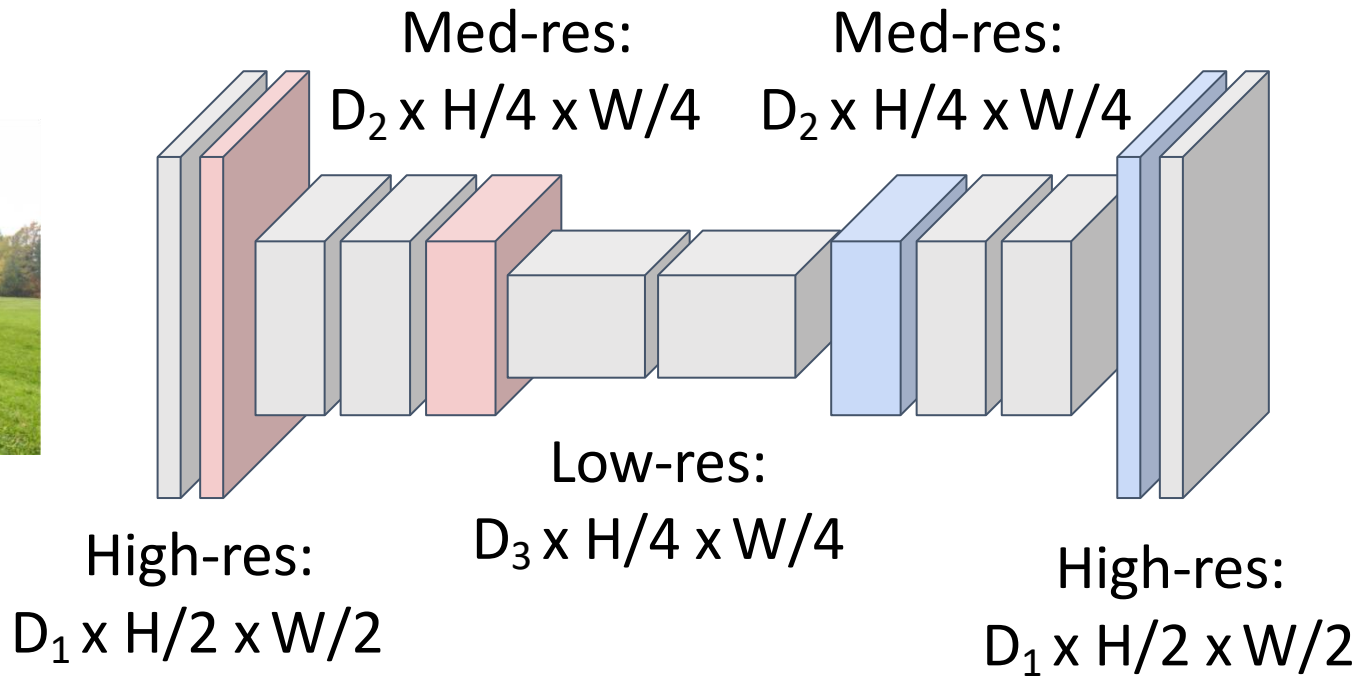
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling:
interpolation,
transposed conv



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

Loss function: Per-Pixel cross-entropy

Skip Connections

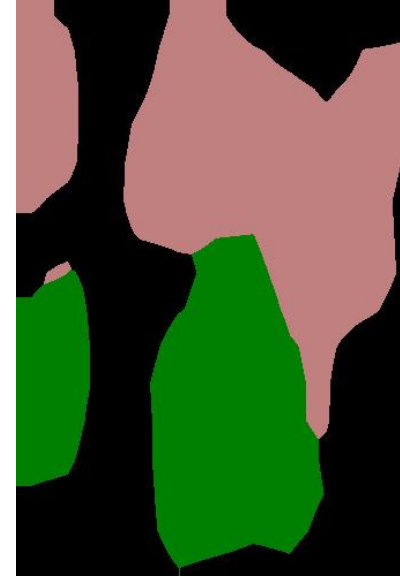
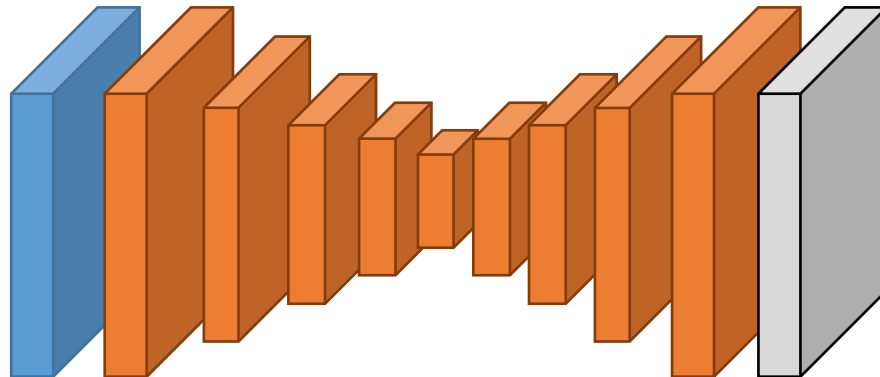
Retaining Spatial Details

While the output *is* HxW, just upsampling often produces results without details/not aligned with the image.

Why?

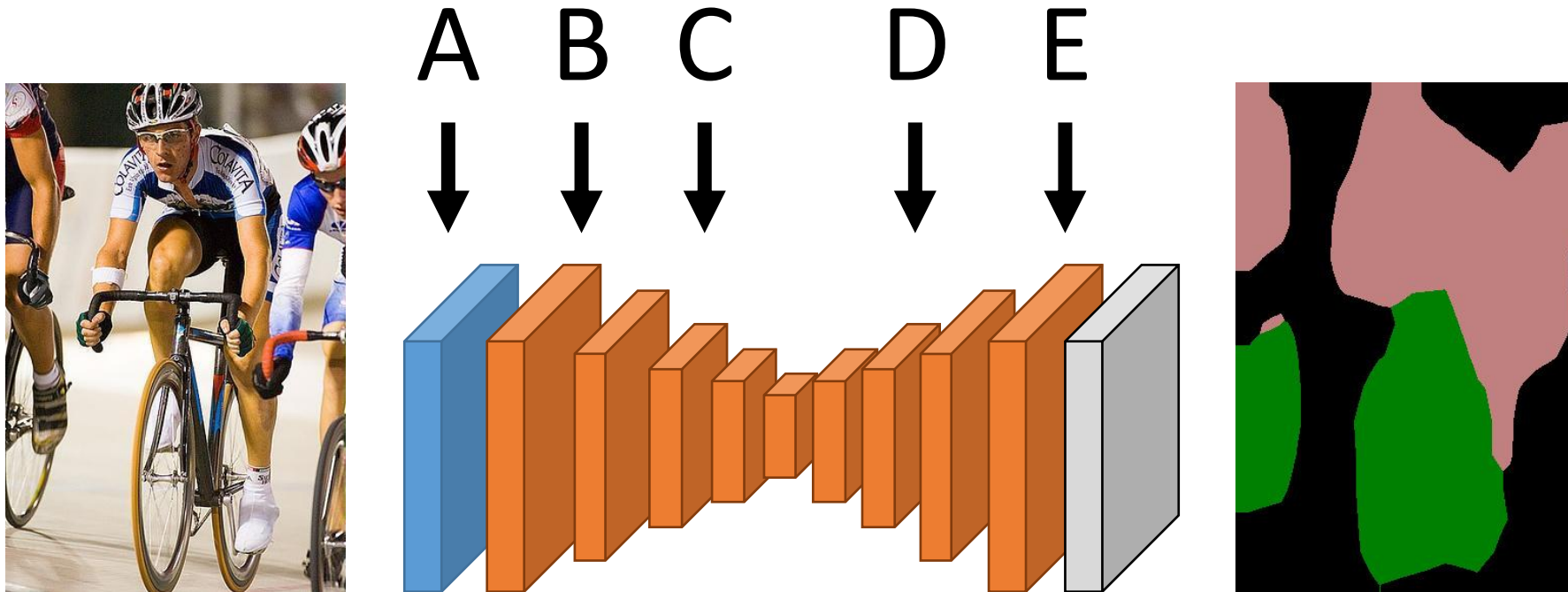


Information about details
lost when downsampling!



Retaining Spatial Details

Where is the useful information about the high-frequency details of the image?

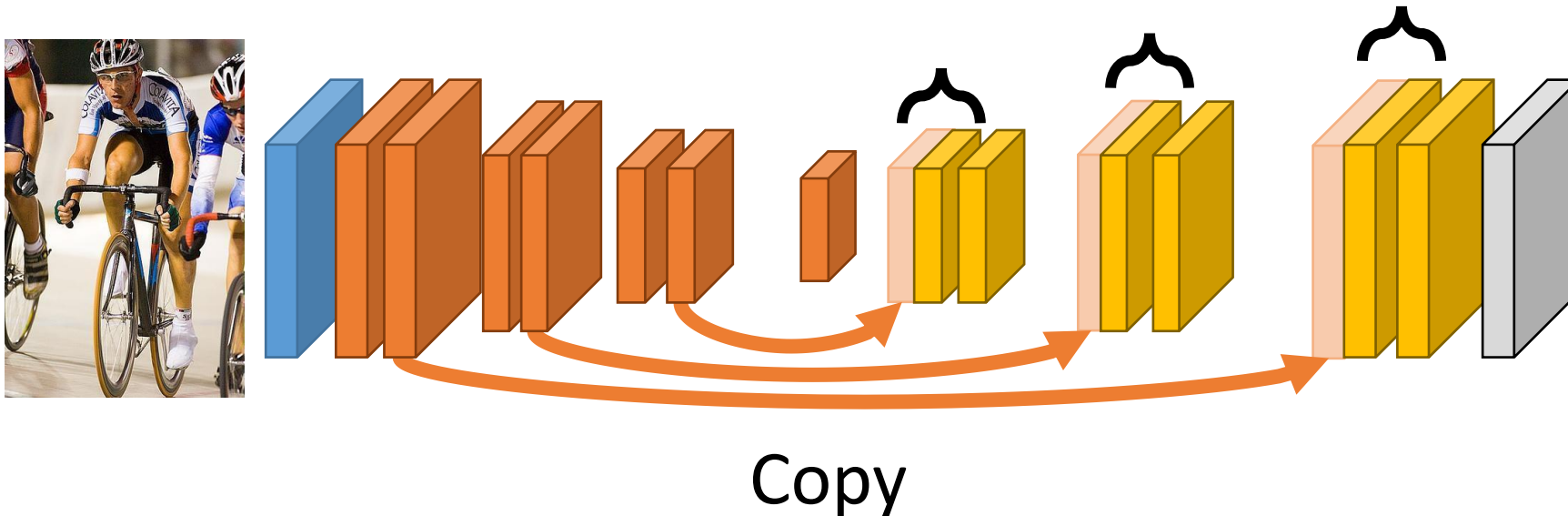


Retaining Spatial Details

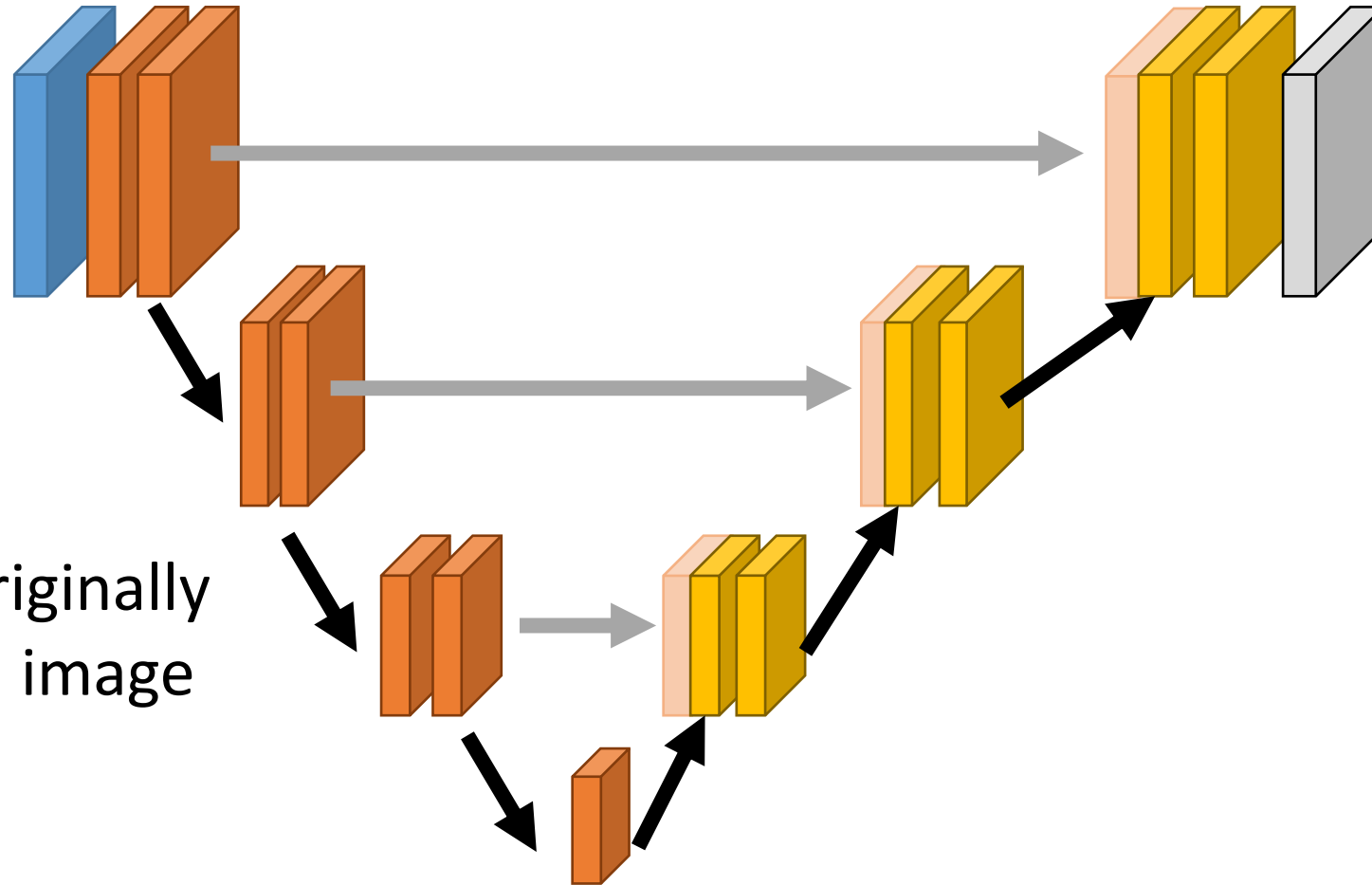
How do you send details forward in the network?

You copy the activations forward.

Subsequent layers at the same resolution figure out how to fuse things.



U-Net



Extremely popular architecture, was originally used for biomedical image segmentation.

Spatial Contexts

The Importance of Spatial Contexts

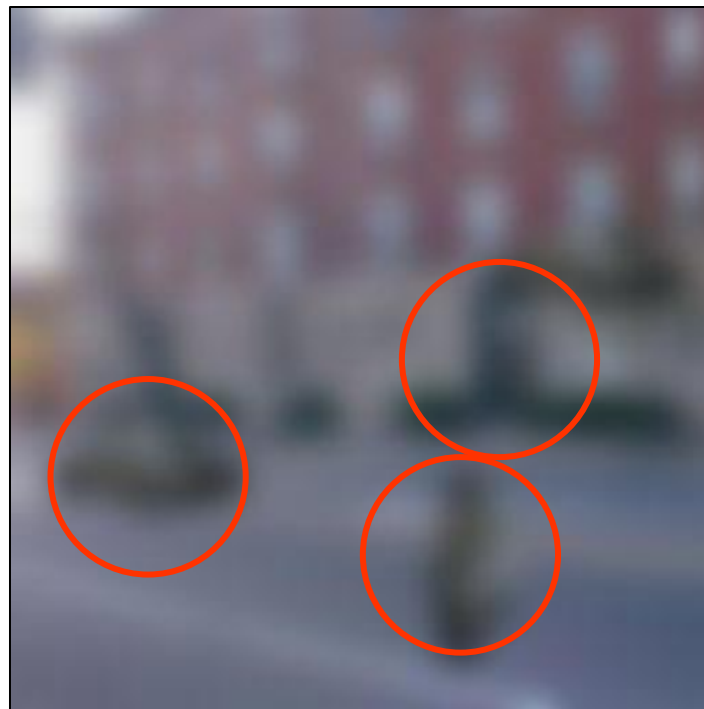
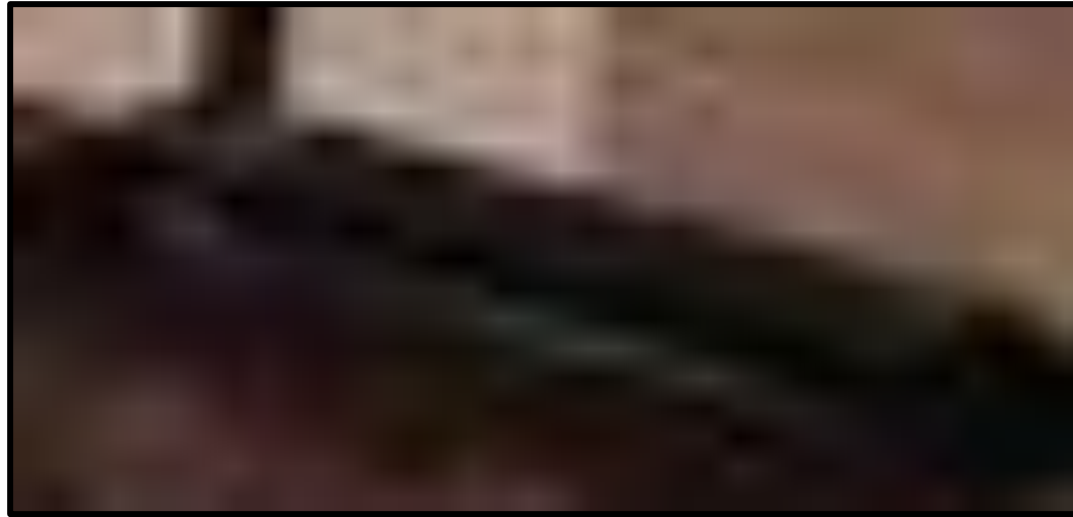


Image credit: A. Torralba

The Importance of Spatial Contexts

What's this? (No Cheating!)



(a) Keyboard?

(c) Old cell phone?

(b) Hammer?

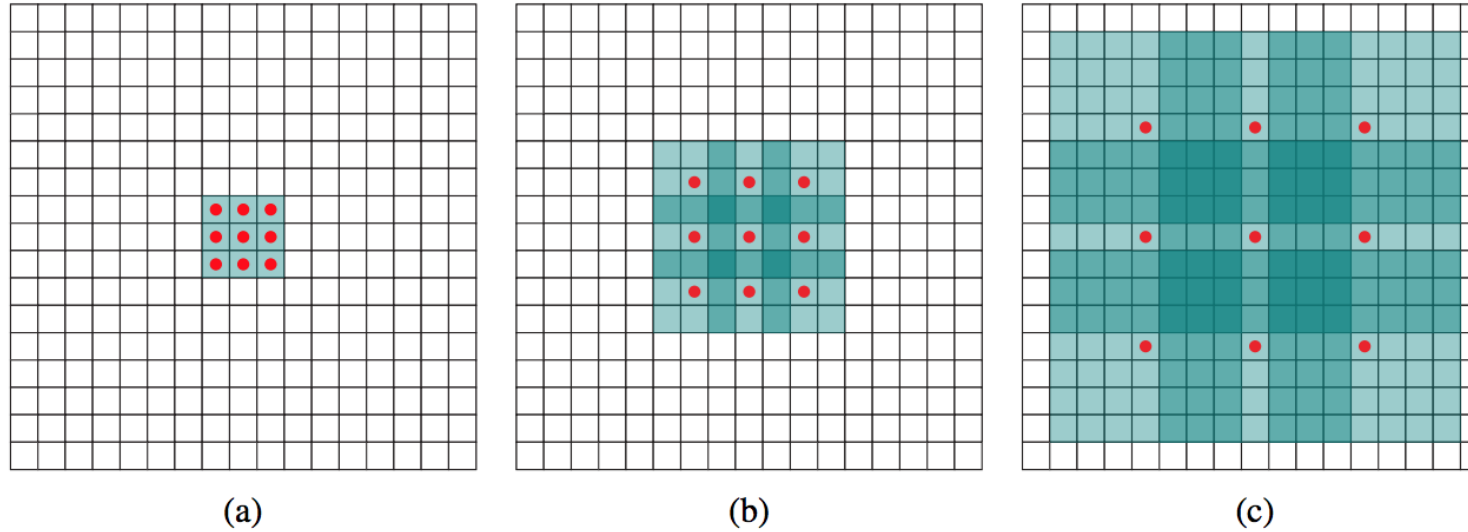
(d) Xbox controller?

The Importance of Spatial Contexts



Image credit: COCO dataset

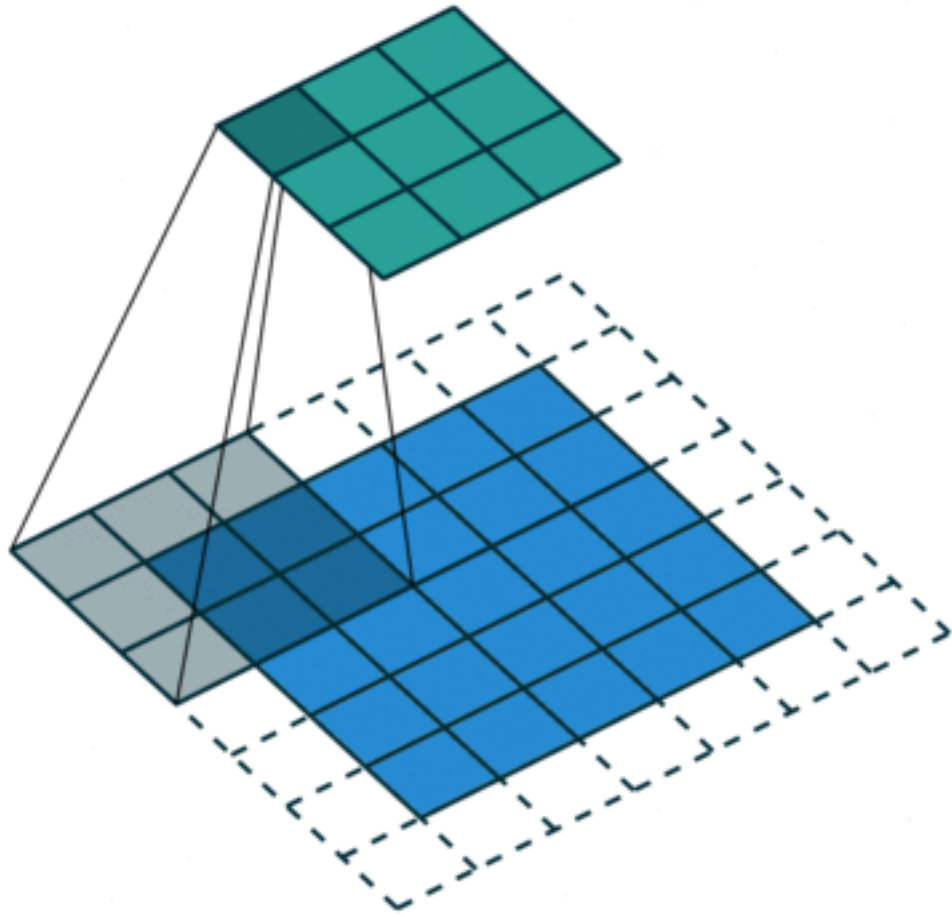
Spatial Contexts: Dilated Convolution



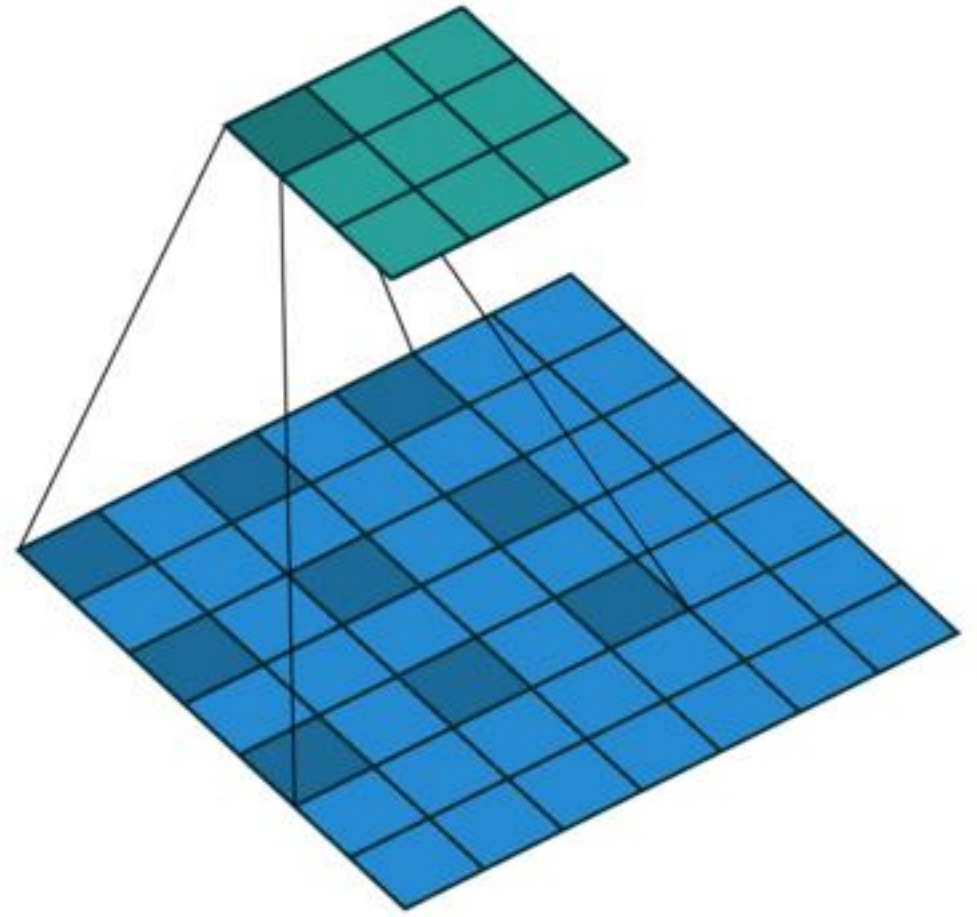
Consecutive 1-Dilated (left), 2-Dilated (middle), 4-Dilated (right) 3×3 Convolution

- Receptive field of an element x in layer $k + 1$ is the set of elements in layer k that influence it
- Resulting receptive field of 2^i -Dilated feature map is size $(2^{i+2} - 1)^2$
- Receptive field grows exponentially while number of parameters is constant

Spatial Contexts: Dilated Convolution

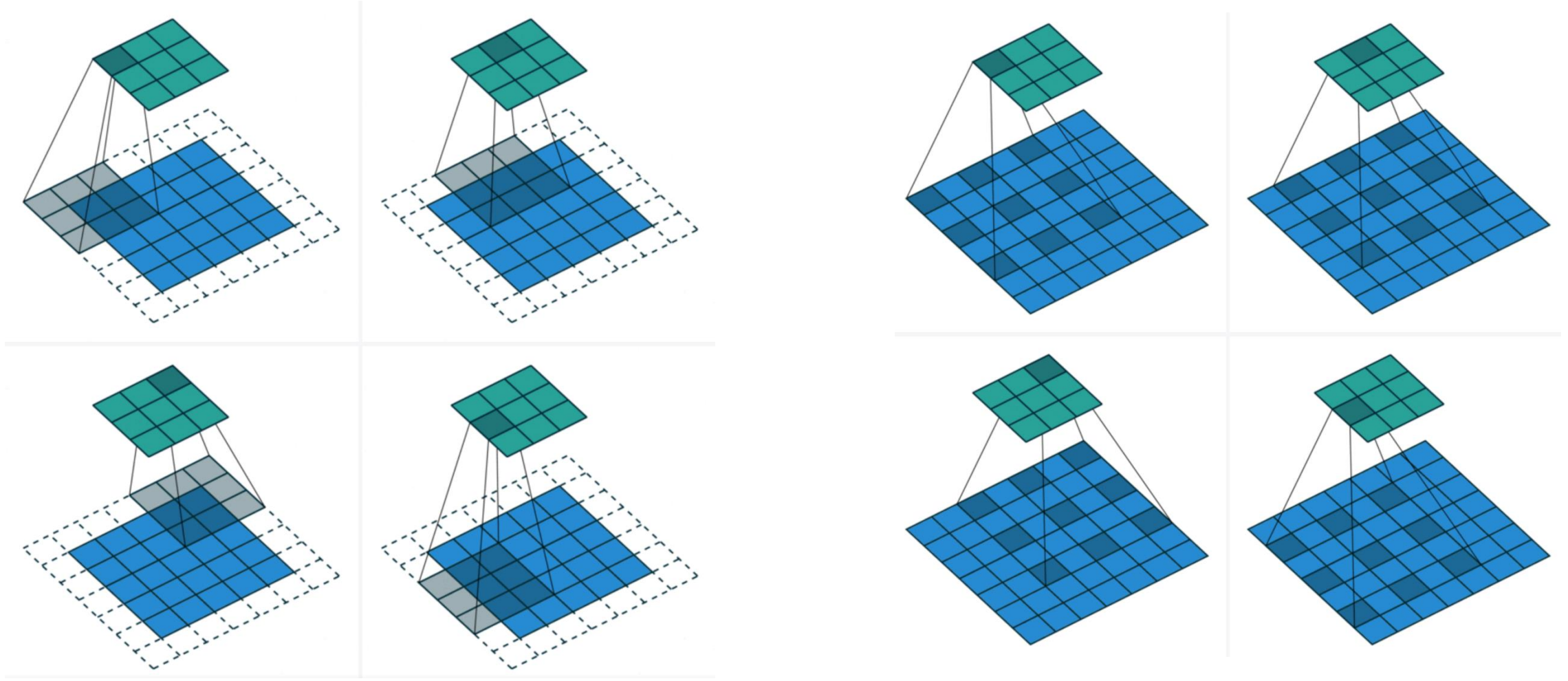


standard convolution



dilated convolution

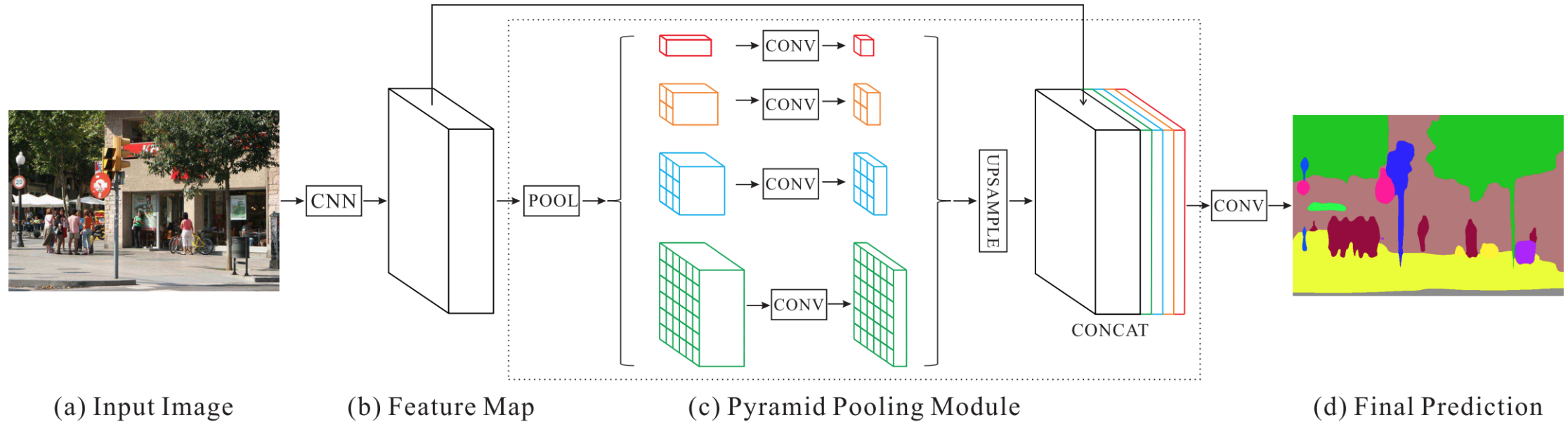
Spatial Contexts: Dilated Convolution



standard convolution

dilated convolution

Spatial Contexts: Pyramid Scene Parsing Net



- Hierarchical global prior, containing information with different scales and varying among different sub-regions
- Pyramid pooling module for global scene prior on the final feature map
- Use 1x1 convolution to reduce the number of channels

Spatial Contexts: Markov Random Field

Energy Function

$$\min E = \textit{Unary} + \textit{Pair}$$

Unary Term

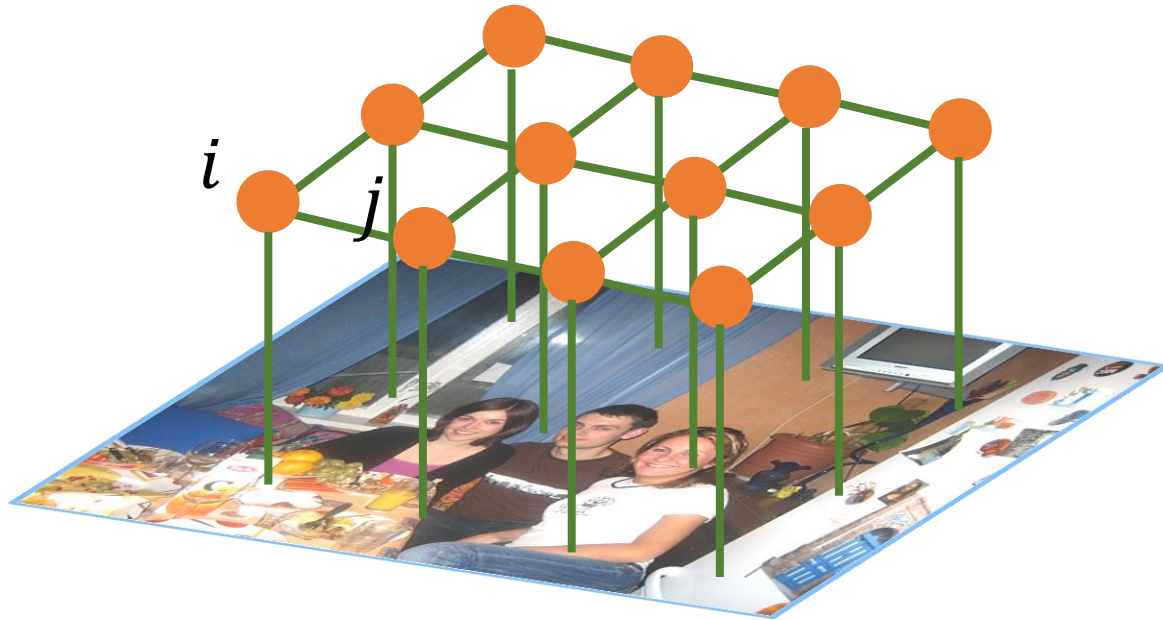
$$\textit{Unary} = - \sum_i \ln p_i(\textit{label})$$

$$p_i(\textit{label} = 'table') = 0.8$$



Spatial Contexts: Markov Random Field

$$diss(i, j) = (\overset{i}{\text{img}}, \overset{j}{\text{img}}) = 0.8$$



Appearance Consistency

Energy Function

$$\min E = \text{Unary} + \text{Pair}$$

Unary Term

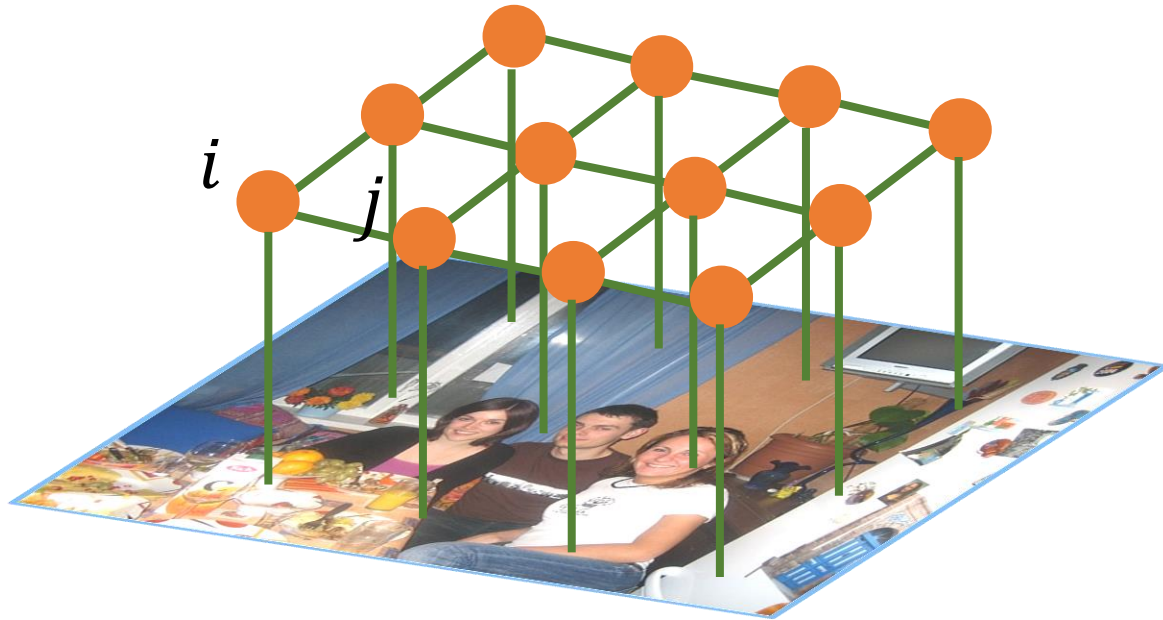
$$\text{Unary} = - \sum_i \ln p_i(\text{label})$$

Pairwise Term

$$\text{Pair} = \sum_{i,j} \text{cost}(i) * diss(i, j)$$

Spatial Contexts: Markov Random Field

$$\text{cost}(i; \text{label} = 'table') = 0.1$$



Label Consistency

Energy Function

$$\min E = \text{Unary} + \text{Pair}$$

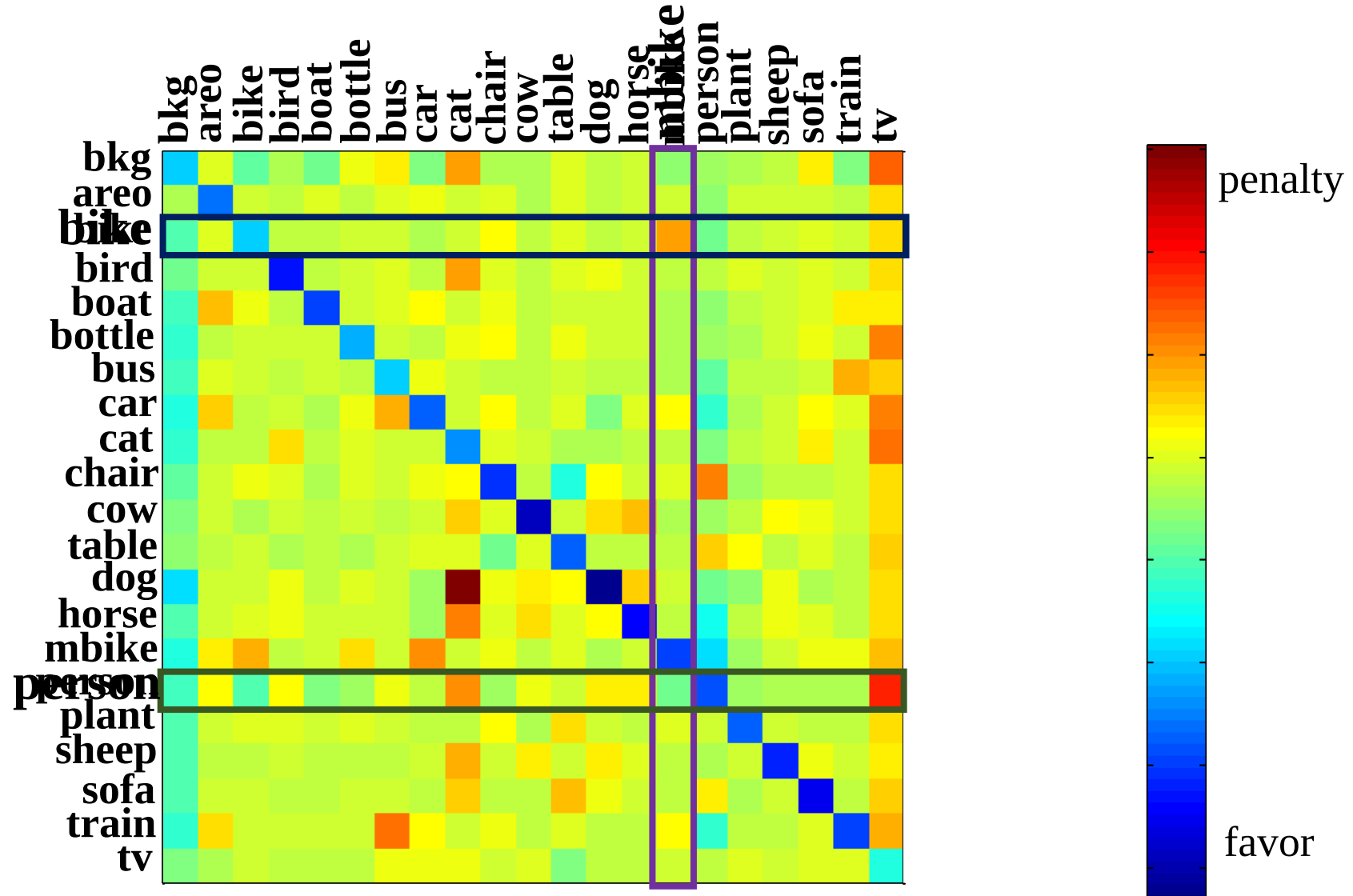
Unary Term

$$\text{Unary} = - \sum_i \ln p_i(\text{label})$$

Pairwise Term

$$\text{Pair} = \sum_{i,j} \text{cost}(i) * \text{diss}(i,j)$$

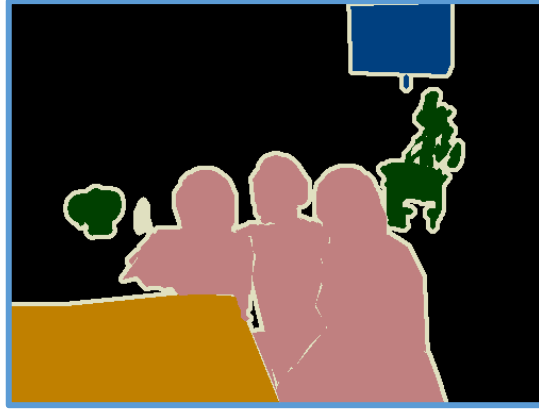
Spatial Contexts: Markov Random Field



Spatial Contexts: Markov Random Field



Original Image



Ground Truth



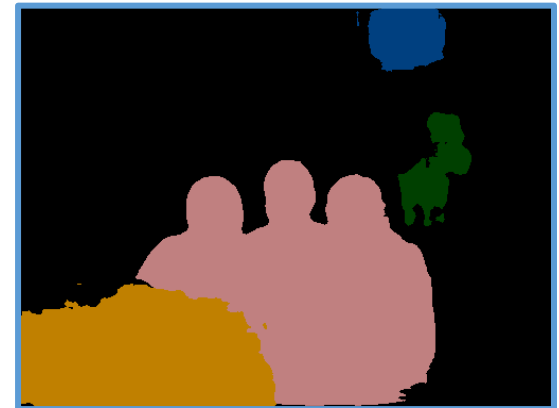
Unary Term



Triple Penalty



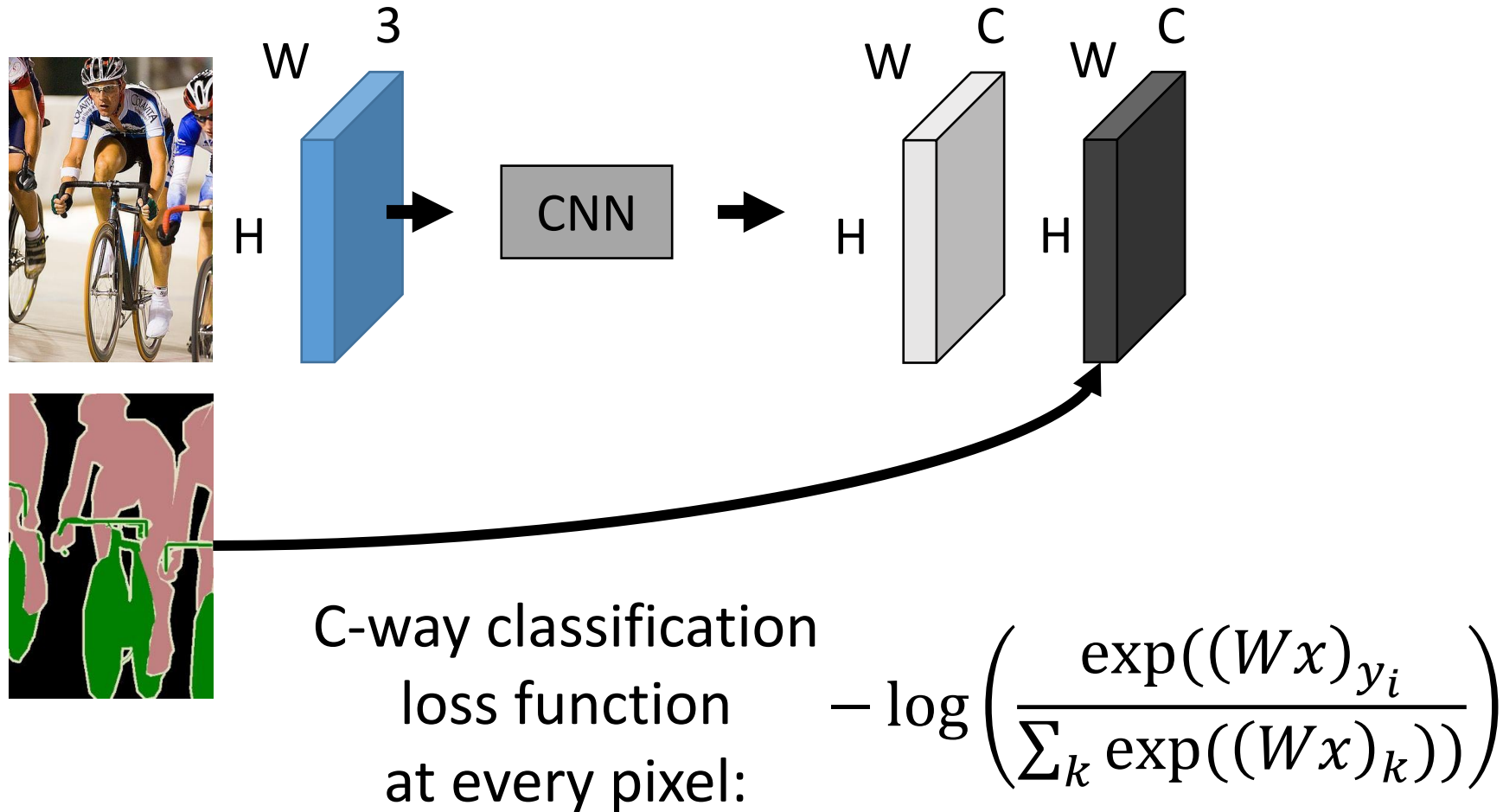
Label Contexts



Joint Tuning

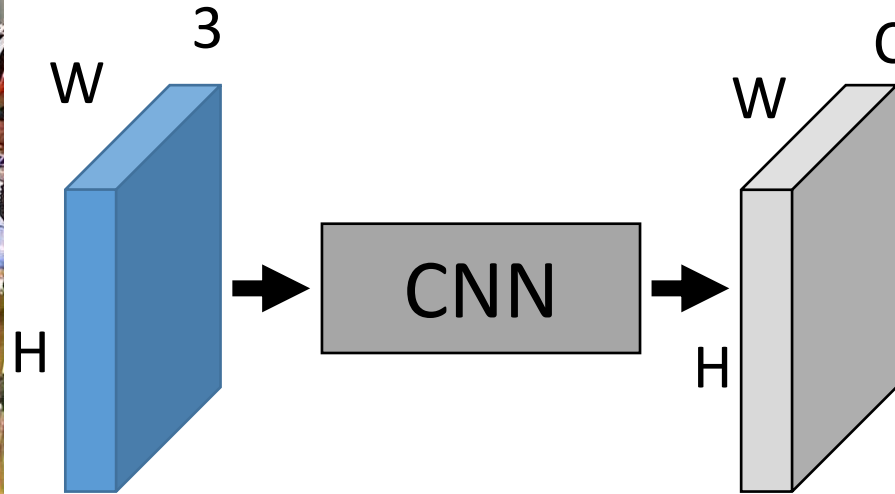
Train and Evaluate FCN

Train Fully Convolutional Network



Evaluate Fully Convolutional Network

Input
Image



Predicted
Classes



How do we convert final $H \times W \times C$ into labels?

argmax over labels

Evaluate Fully Convolutional Network

Input



Prediction ($\hat{y}$)



Ground-Truth (y)

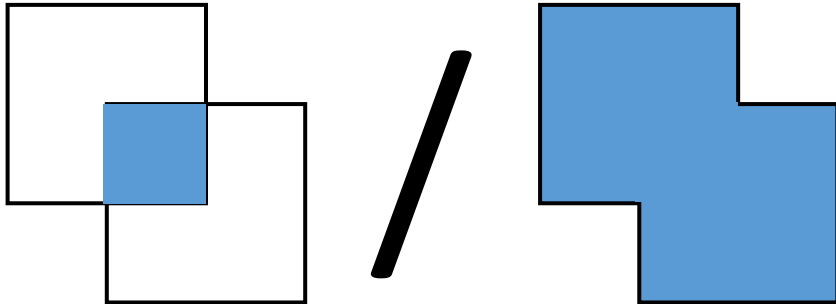


Evaluate Fully Convolutional Network

Prediction and ground-truth are images where each pixel is one of C classes.

Accuracy: $\text{mean}(\hat{y} = y)$

Intersection over union,
averaged over classes



Prediction
($\hat{y}$)

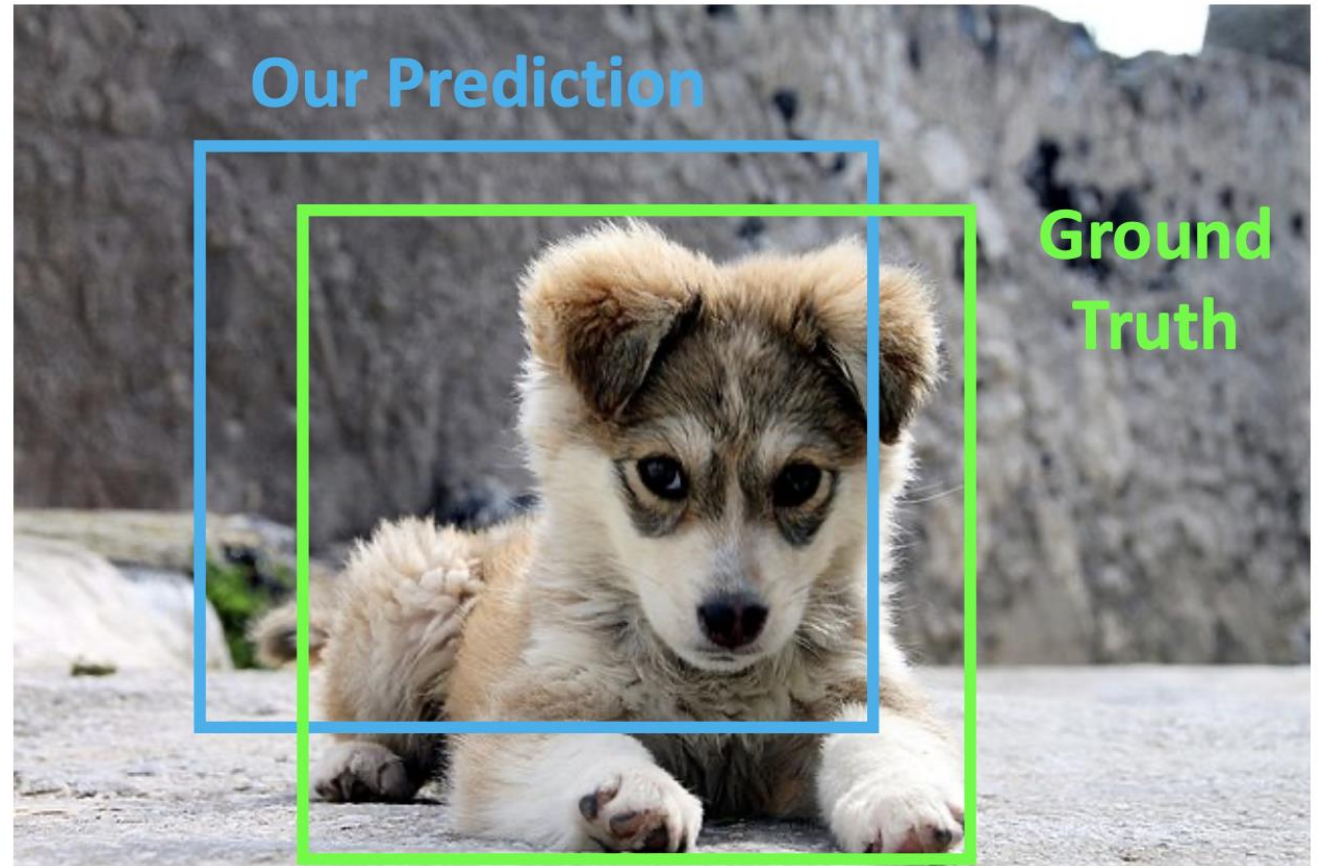


Ground-Truth
(y)



Intersection over Union (IoU)

How can we compare our prediction to the ground-truth bounding box or mask?

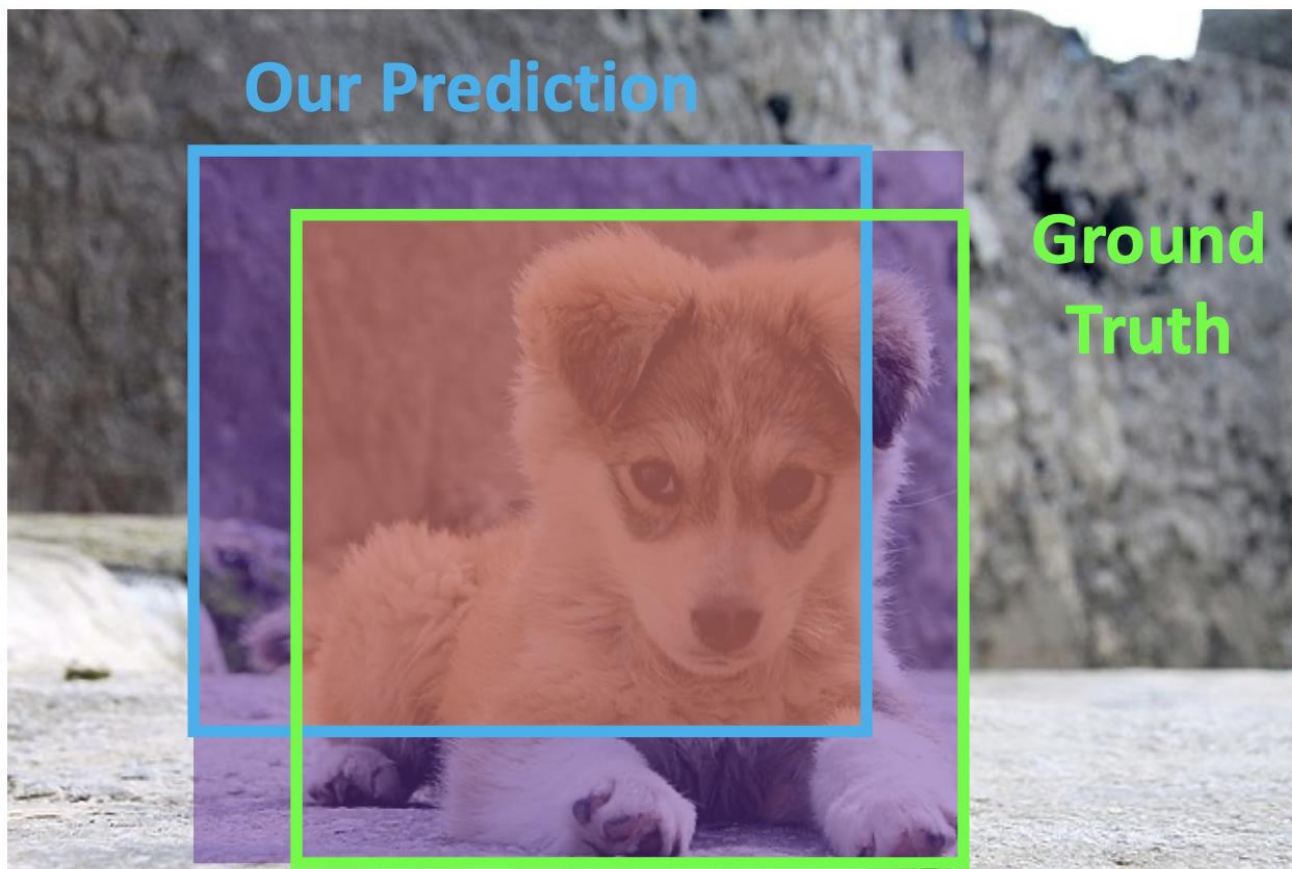


Intersection over Union (IoU)

How can we compare our prediction to the ground-truth bounding box or mask?

Intersection over Union (IoU) (Also called “Jaccard similarity” or “Jaccard index”):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$



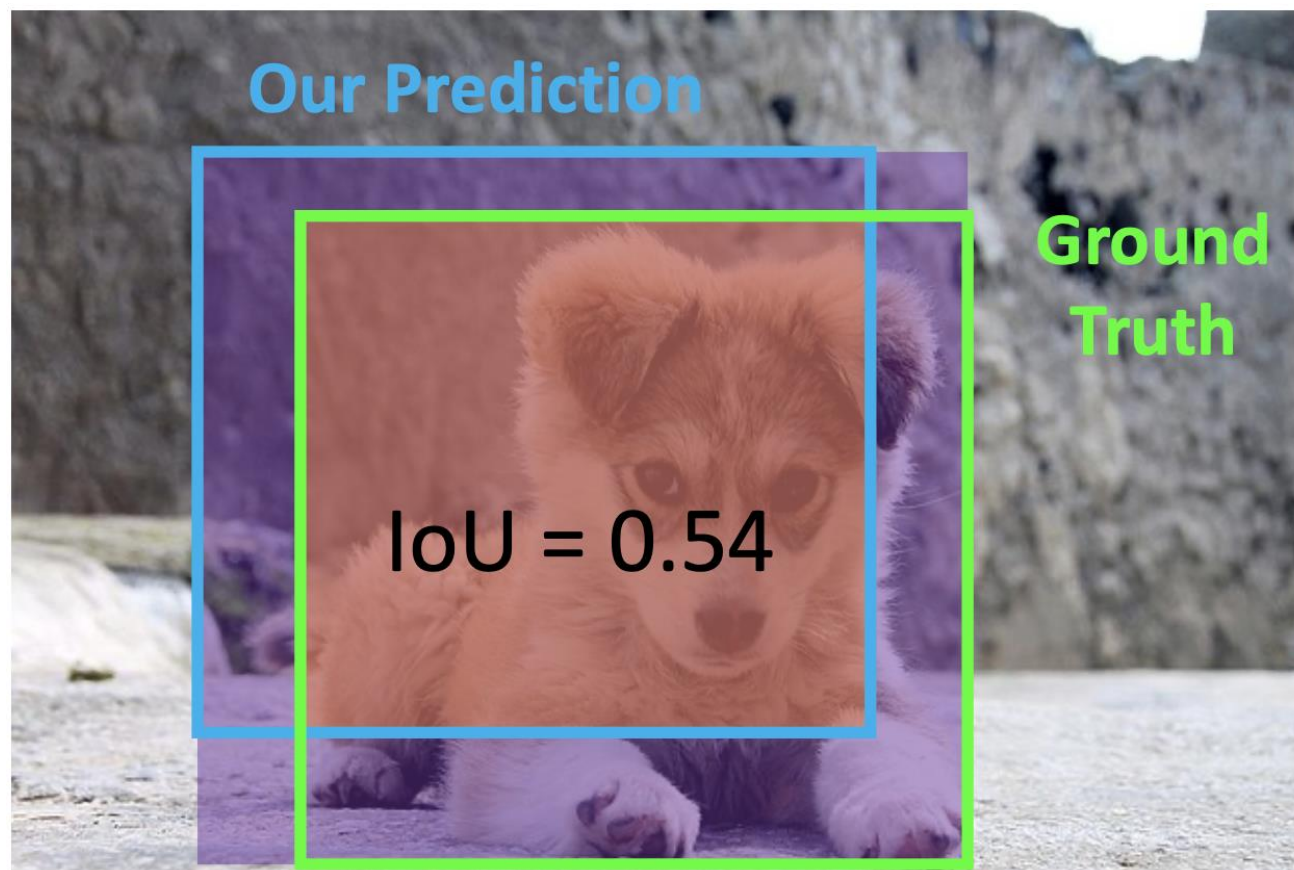
Intersection over Union (IoU)

How can we compare our prediction to the ground-truth bounding box or mask?

Intersection over Union (IoU) (Also called “Jaccard similarity” or “Jaccard index”):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$\text{IoU} > 0.5$ is “decent”



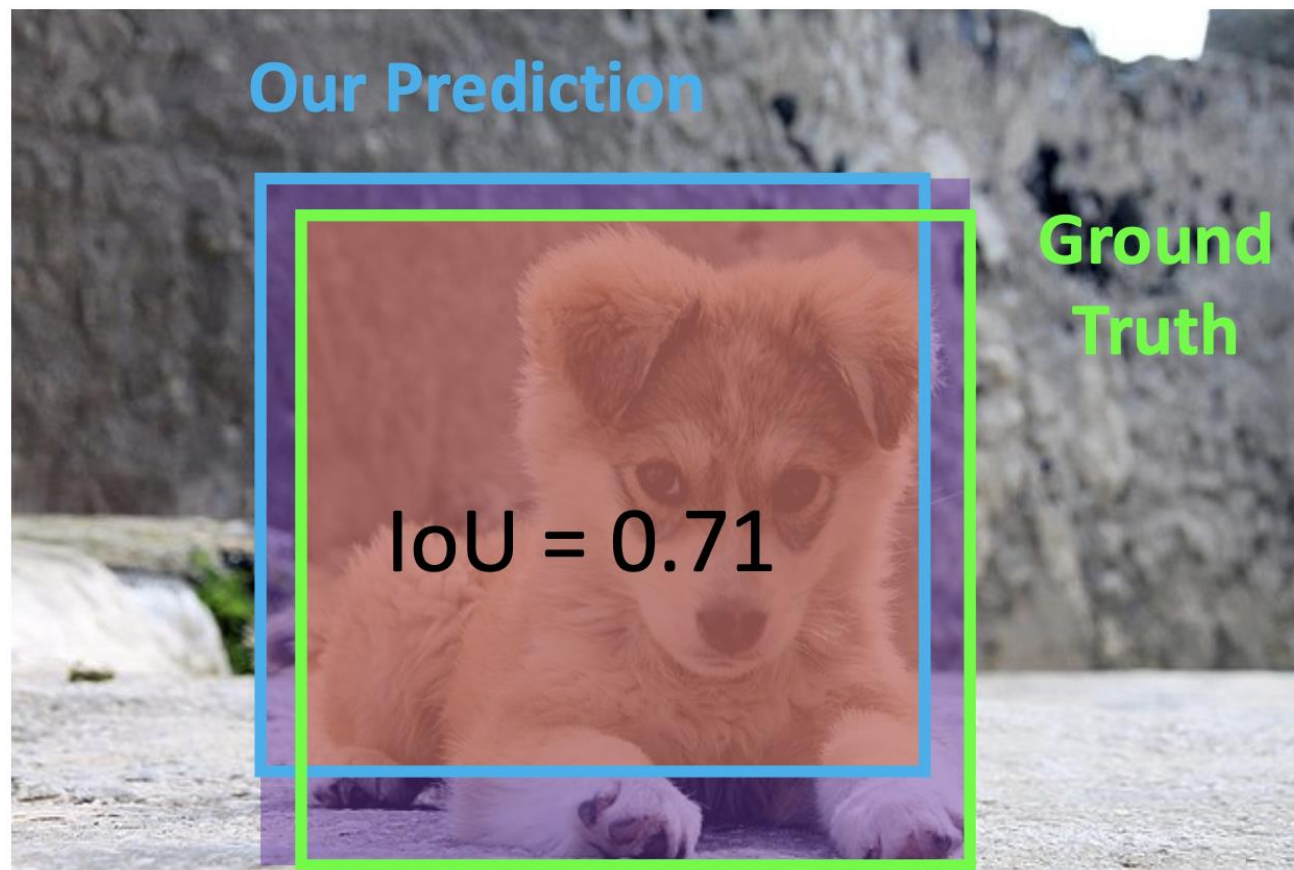
Intersection over Union (IoU)

How can we compare our prediction to the ground-truth bounding box or mask?

Intersection over Union (IoU) (Also called “Jaccard similarity” or “Jaccard index”):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$\text{IoU} > 0.5$ is “decent”,
 $\text{IoU} > 0.7$ is “pretty good”



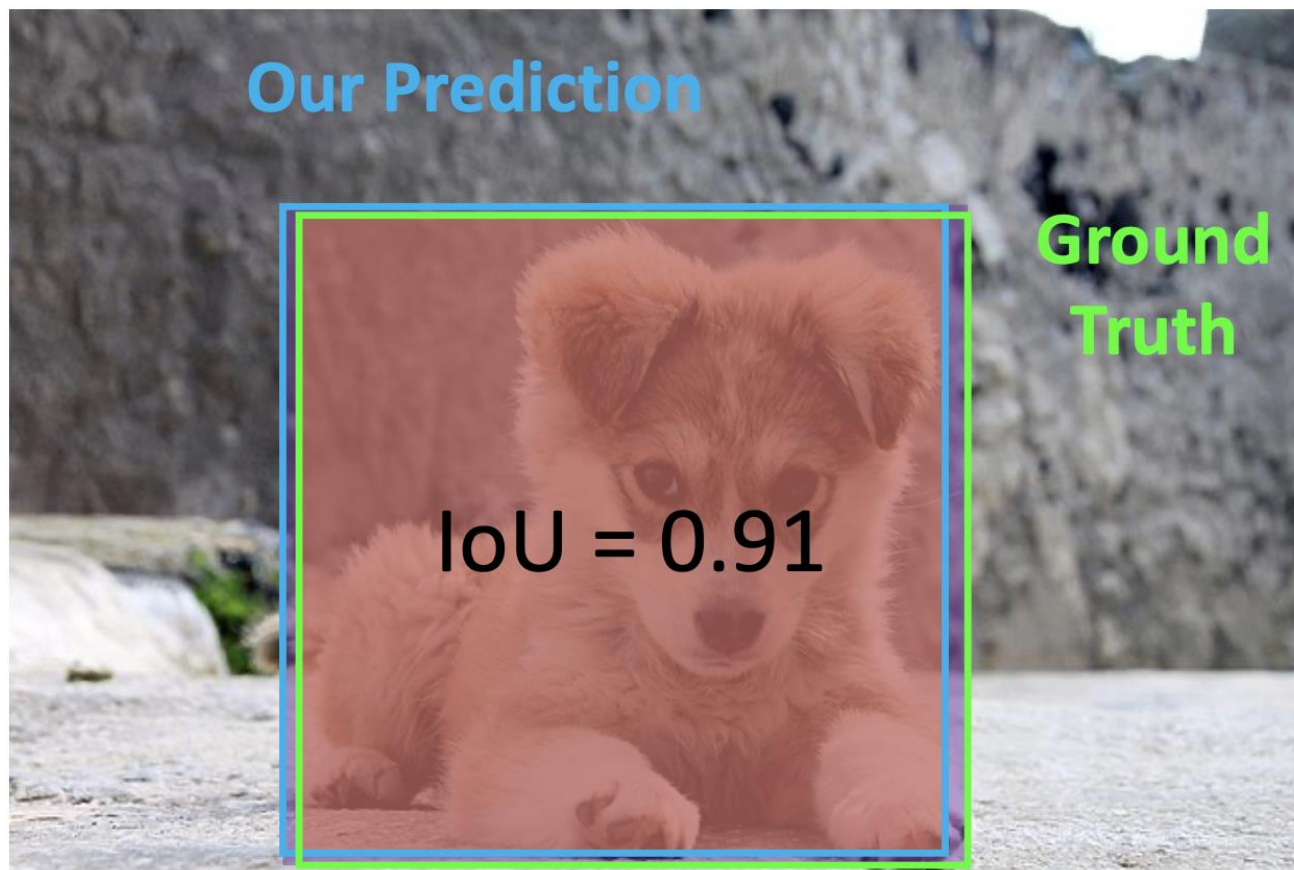
Intersection over Union (IoU)

How can we compare our prediction to the ground-truth bounding box or mask?

Intersection over Union (IoU) (Also called “Jaccard similarity” or “Jaccard index”):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

IoU > 0.5 is “decent”,
IoU > 0.7 is “pretty good”,
IoU > 0.9 is “almost perfect”



Part II:

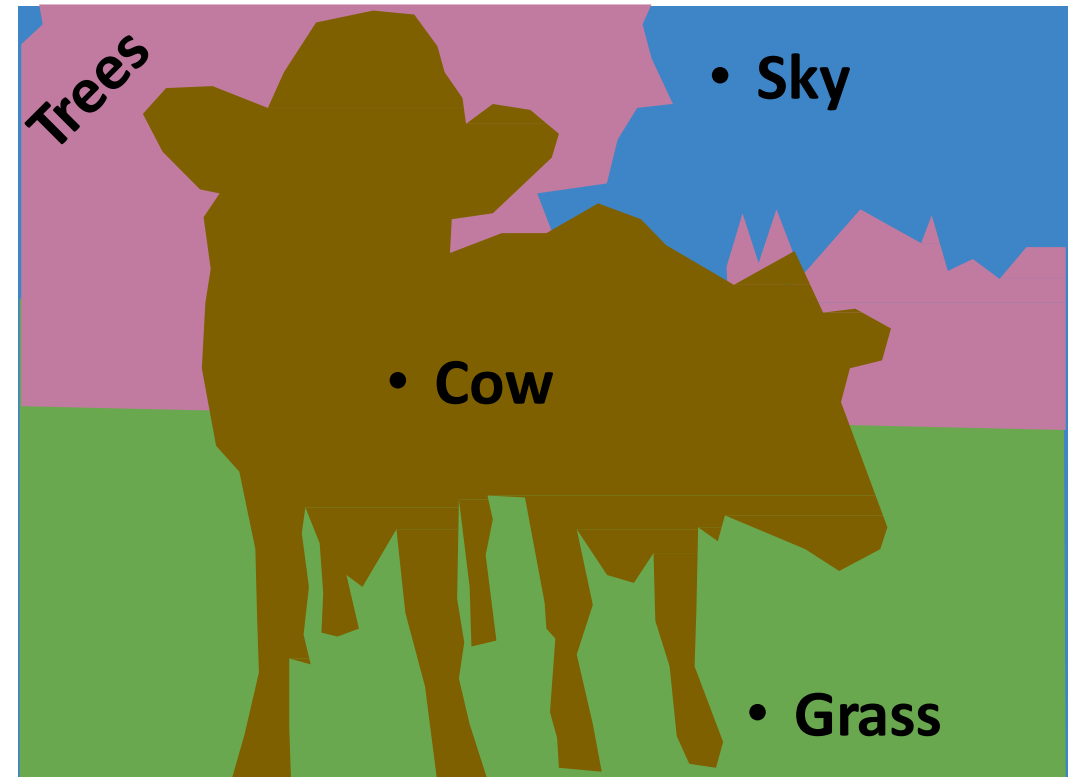
Instance Segmentation

Structured Prediction Tasks

Object Detection: Detects individual object instances, but only gives box



Semantic Segmentation: Gives per-pixel labels, but merges instances



Things and Stuff

Things and Stuff

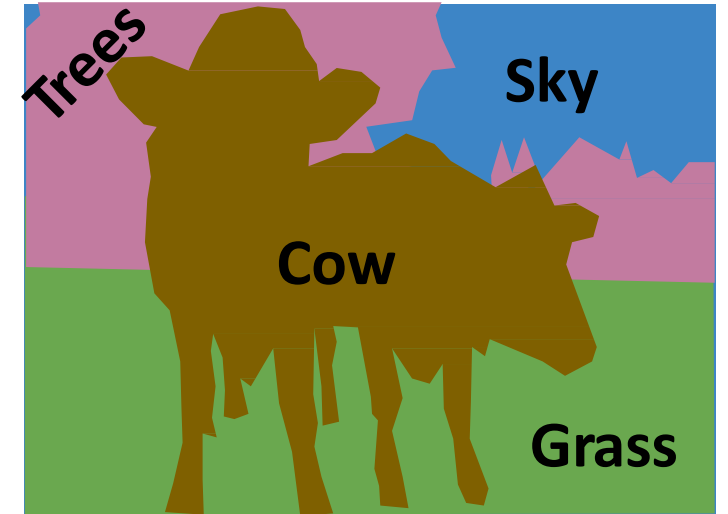
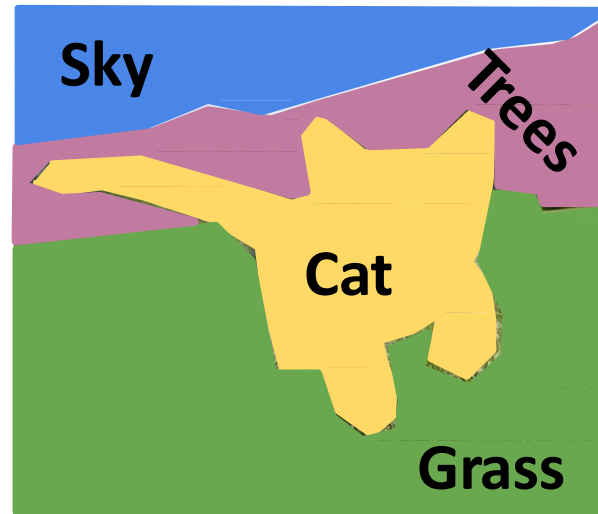
Things: Object categories that can be separated into object instances (e.g. cats, cars, person)



[This image is CC0 public domain](#)



Stuff: Object categories that cannot be separated into instances (e.g. sky, grass, water, trees)



Things and Stuff

- Things



- Stuff

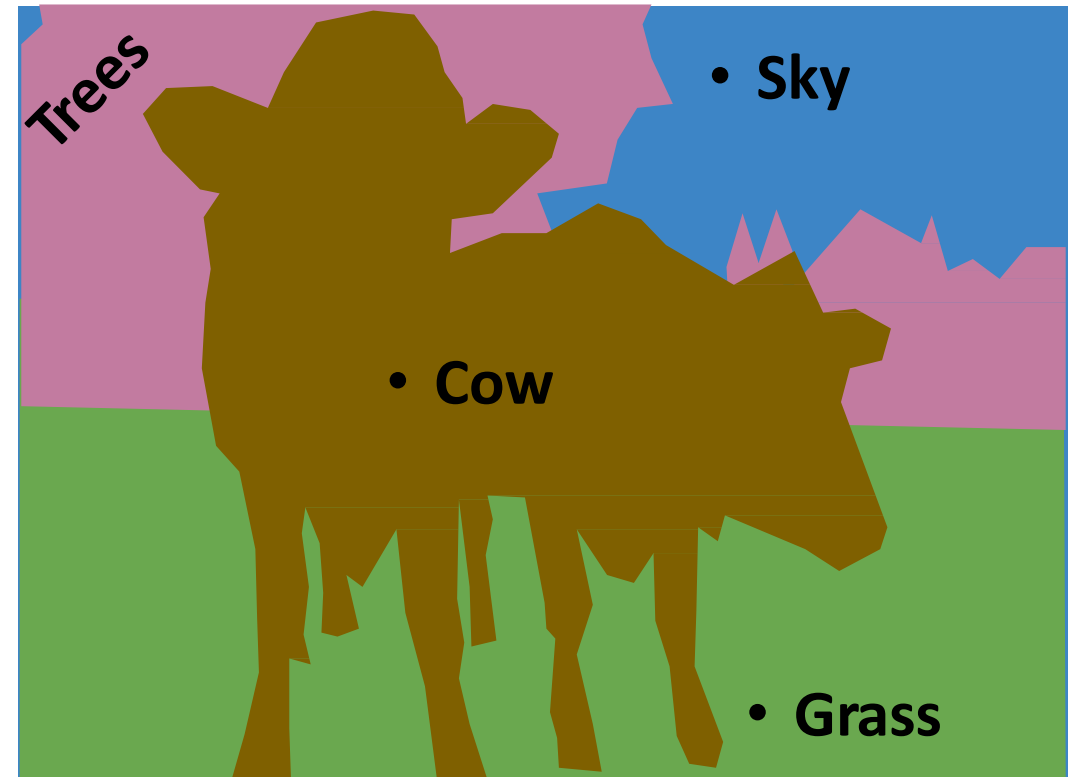


Structured Prediction Tasks

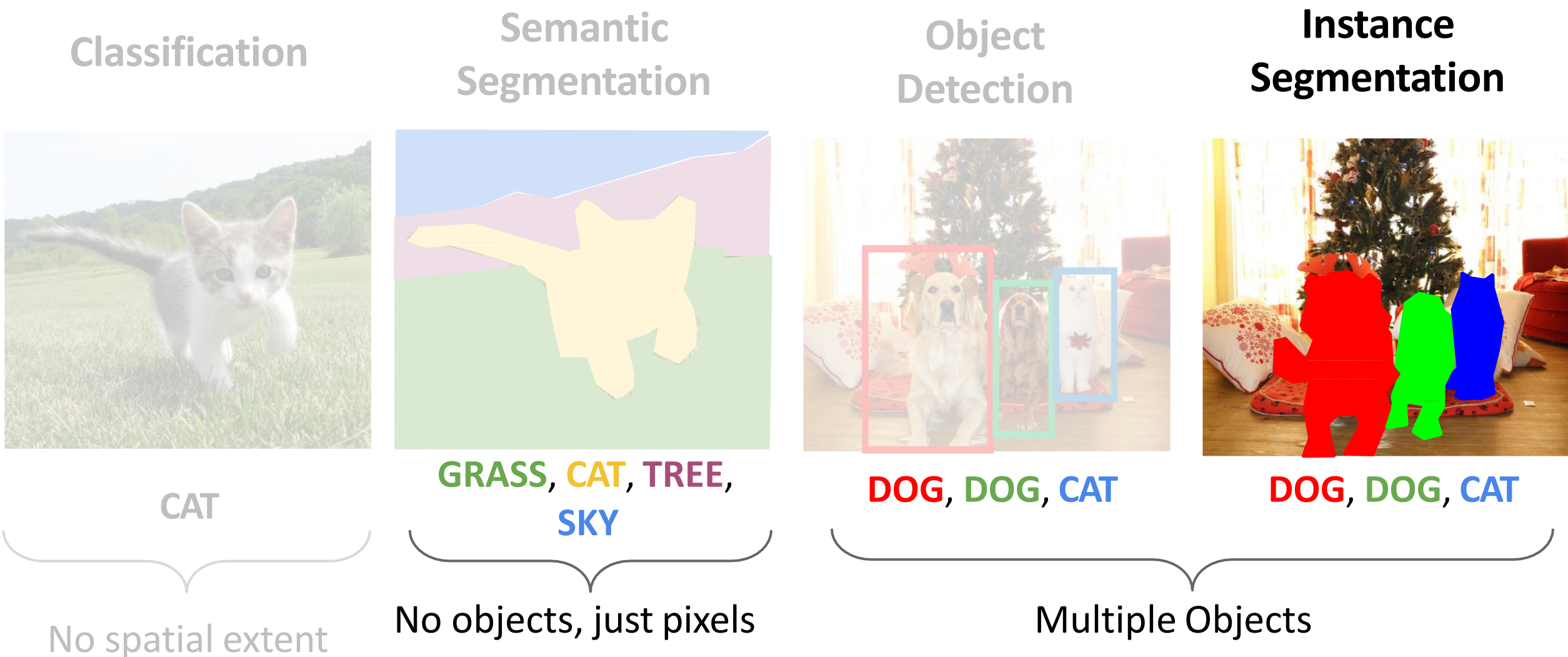
Object Detection: Detects individual object instances, but only gives box (Only things!)



Semantic Segmentation: Gives per-pixel labels, but merges instances (Both things and stuff)



Structured Prediction Tasks: Instance Segmentation



Semantic or Instance Segmentation?



Face Parsing



Eyeglasses



Earring



Cloth



Hat



Necklace



Hair

Semantic Segmentation



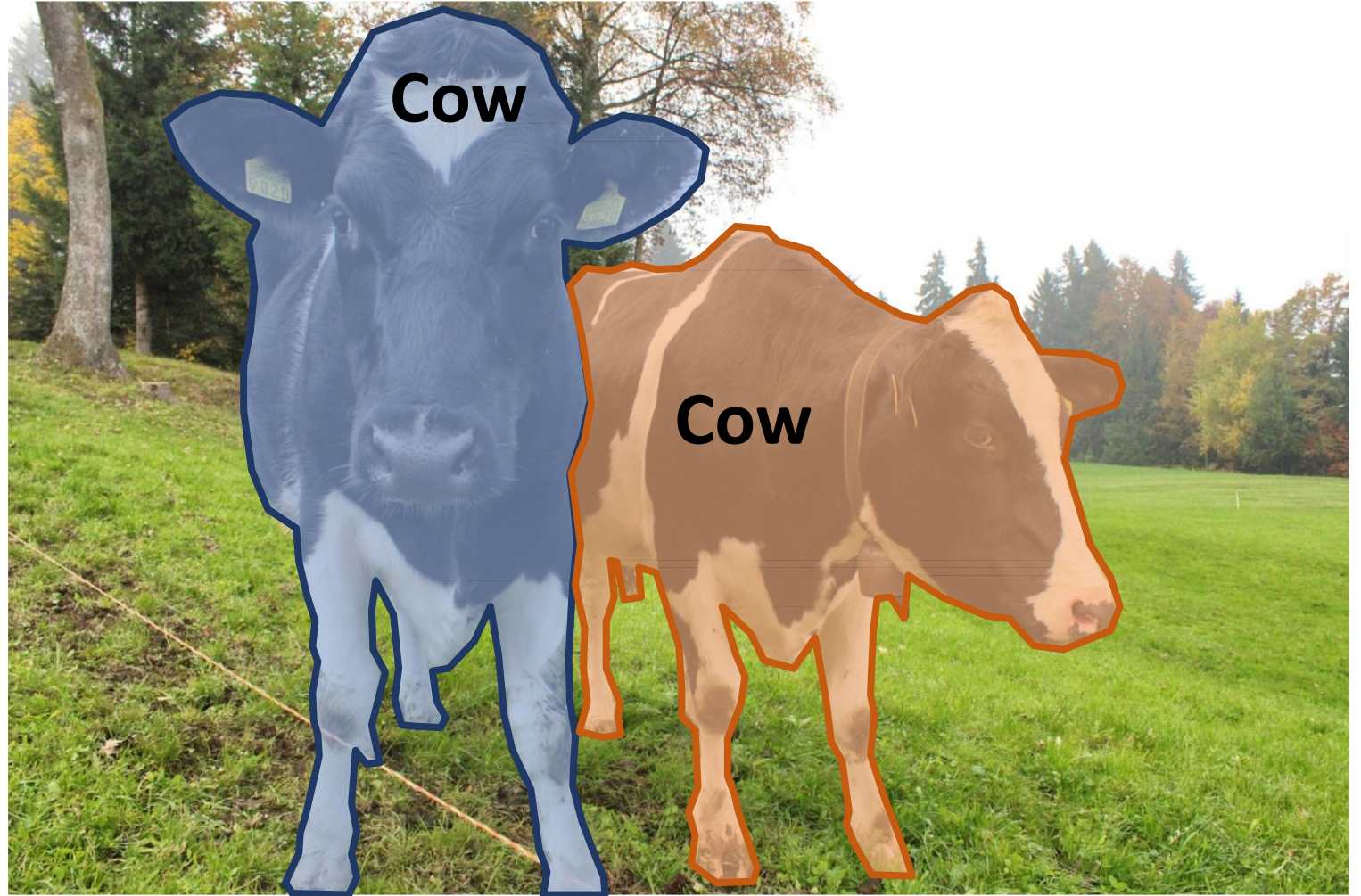
Instance Segmentation



Instance Segmentation

Instance Segmentation:

Detect all objects in the image, and identify the pixels that belong to each object (Only things!)

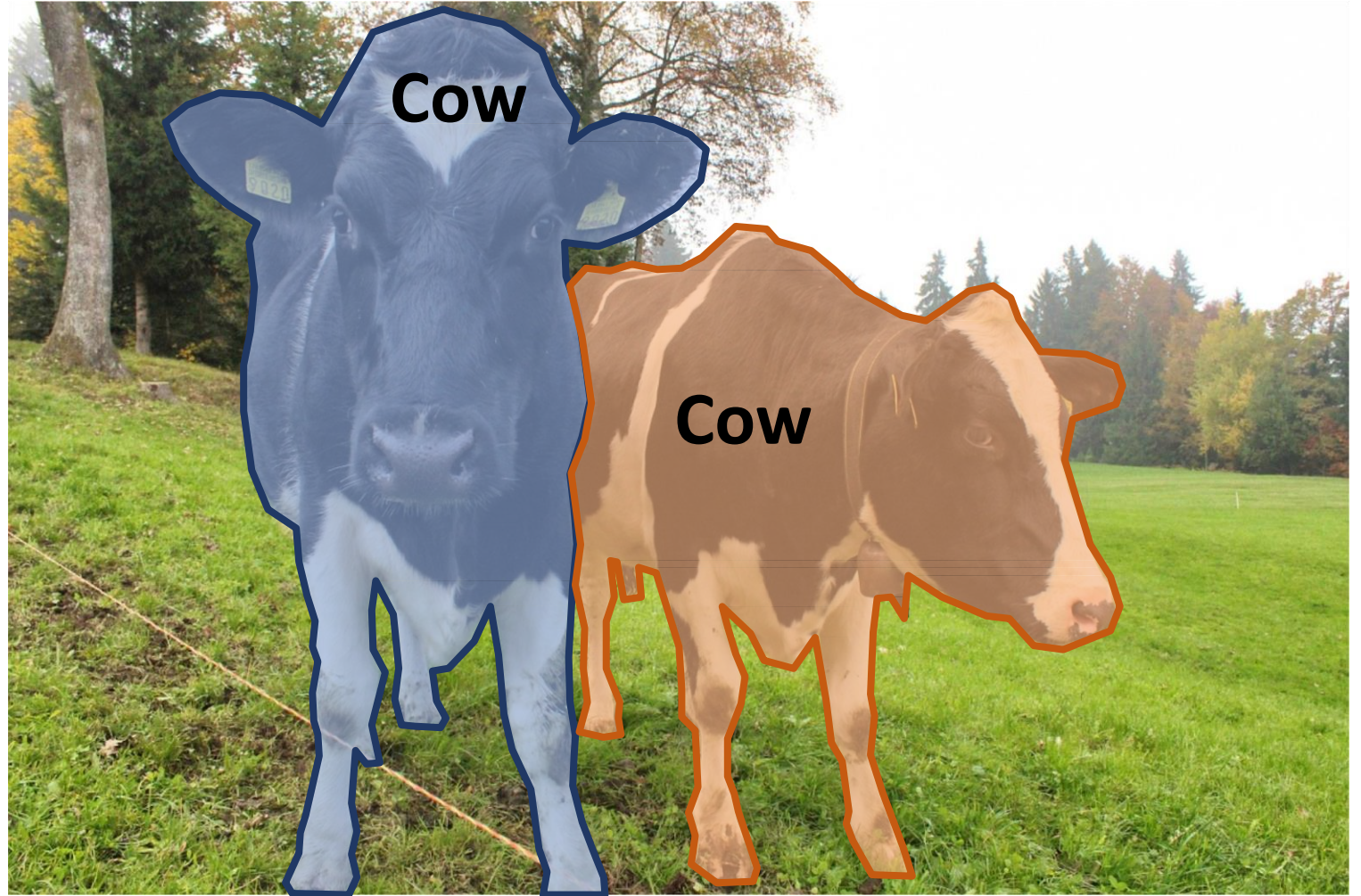


Instance Segmentation

Instance Segmentation:

Detect all objects in the image, and identify the pixels that belong to each object (Only things!)

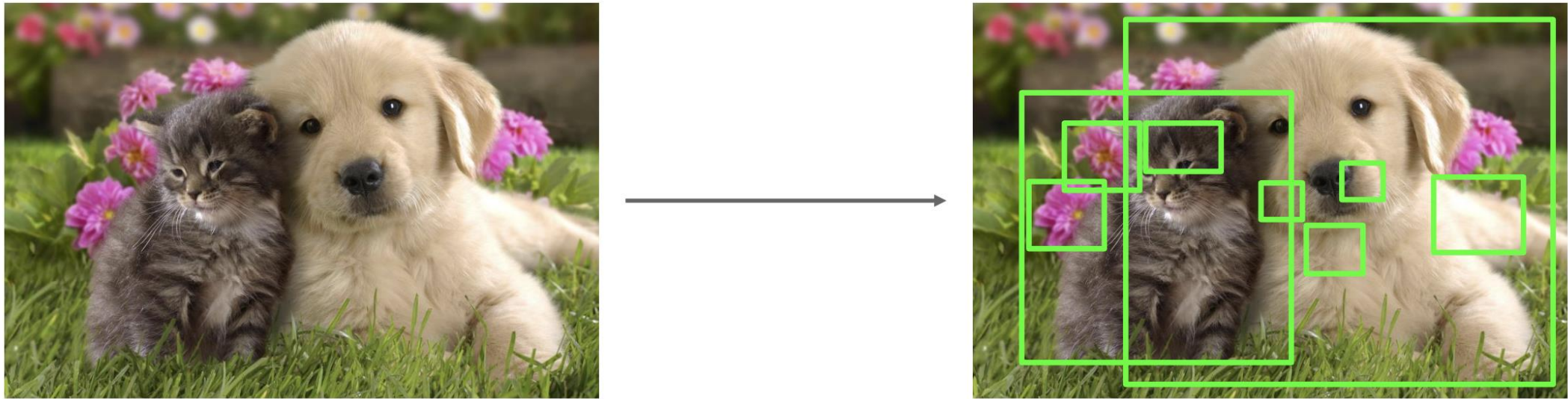
Approach: Perform object detection, then predict a segmentation mask for each object!



Object Detection

Region proposals

- Find a small set of boxes that are likely to cover all objects
- Often based on heuristics: e.g. look for “blob-like” image regions
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012

Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013

Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014 Zitnick and Dollar, “Edge

boxes: Locating object proposals from edges”, ECCV 2014

Region-based CNN

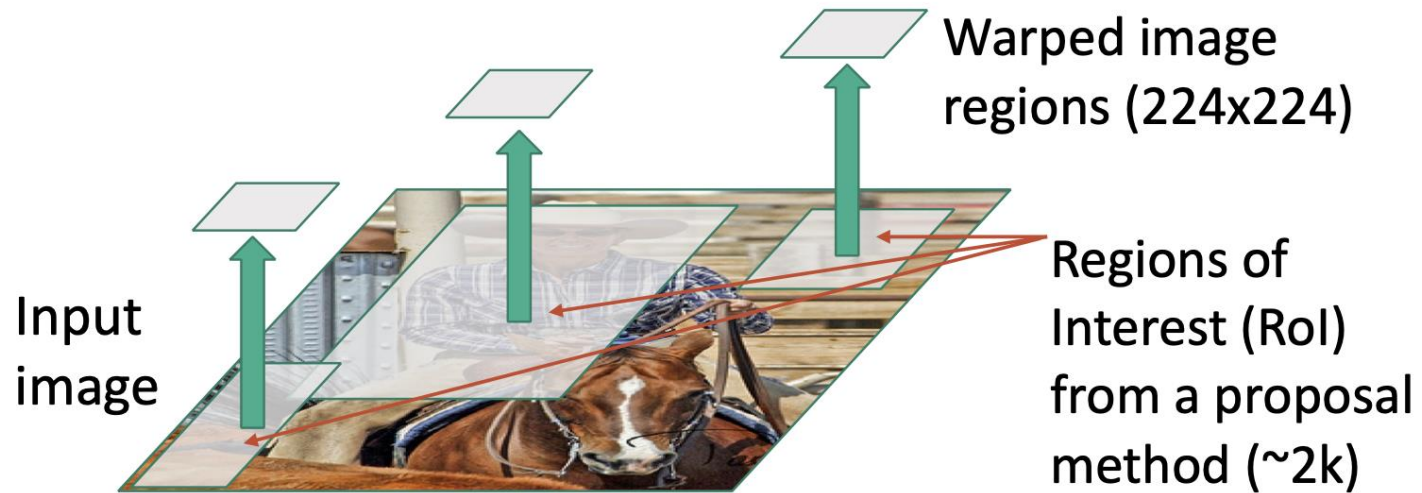
Input
image



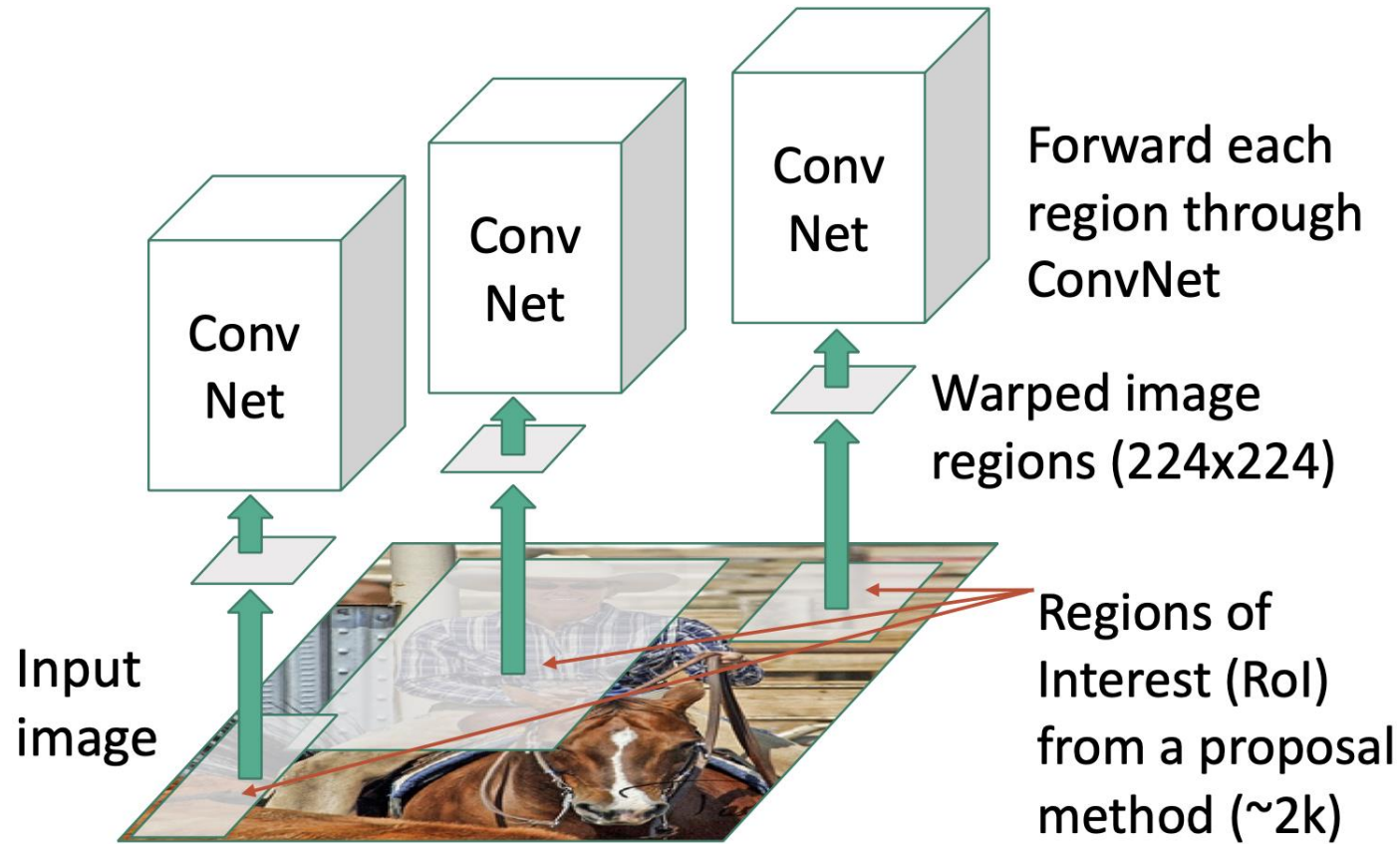
Region-based CNN



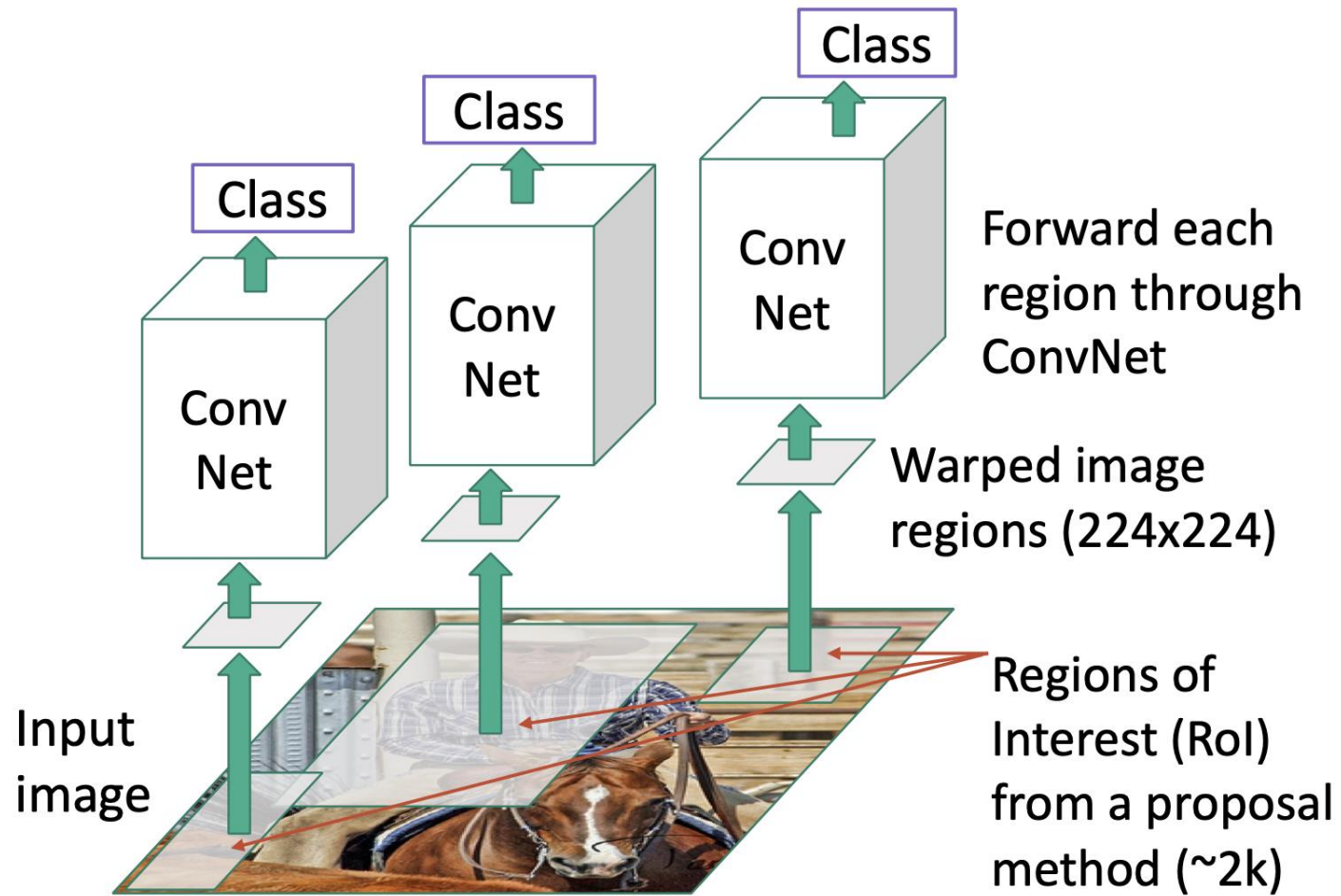
Region-based CNN



Region-based CNN

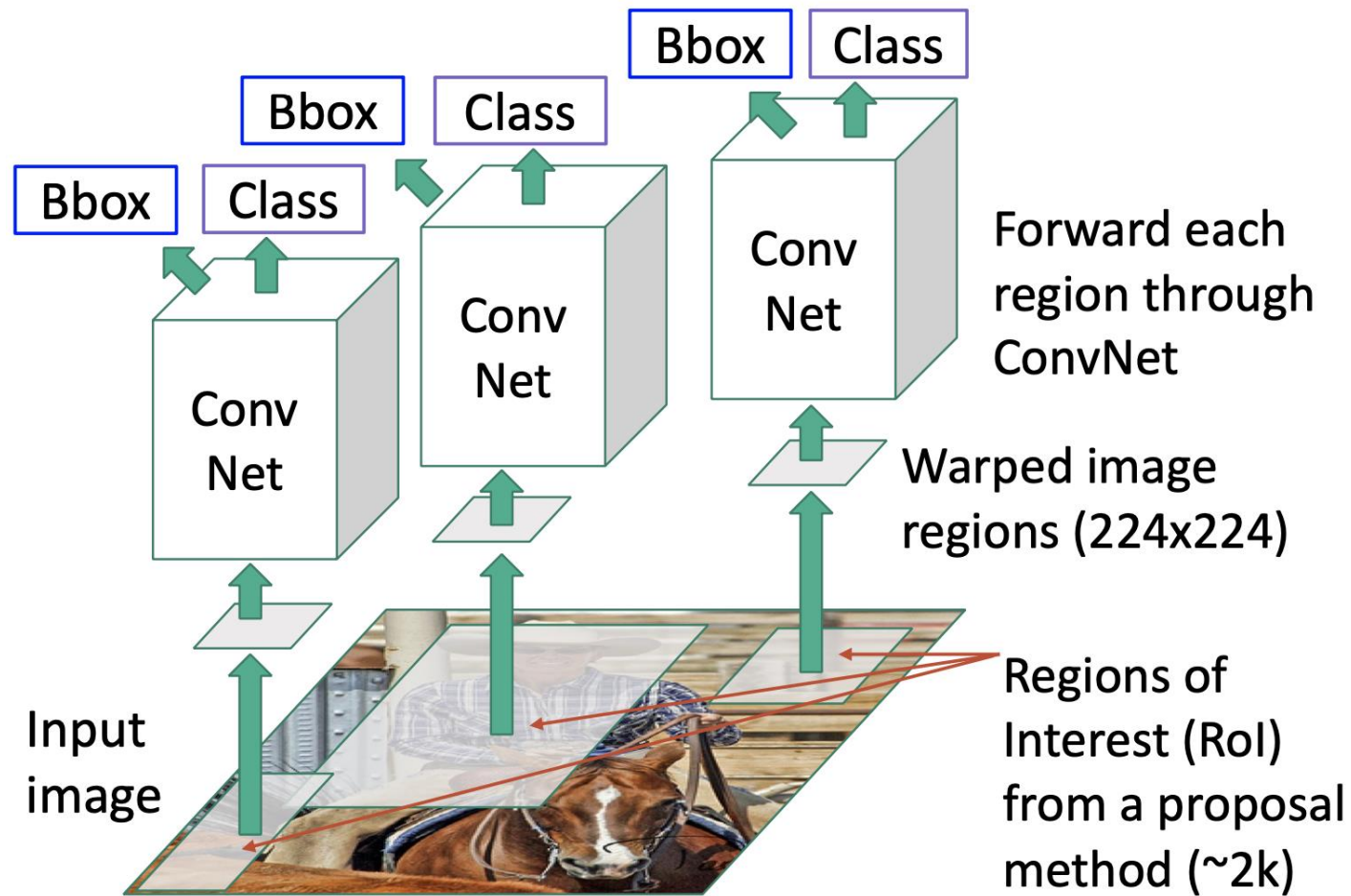


Region-based CNN



Classify each region

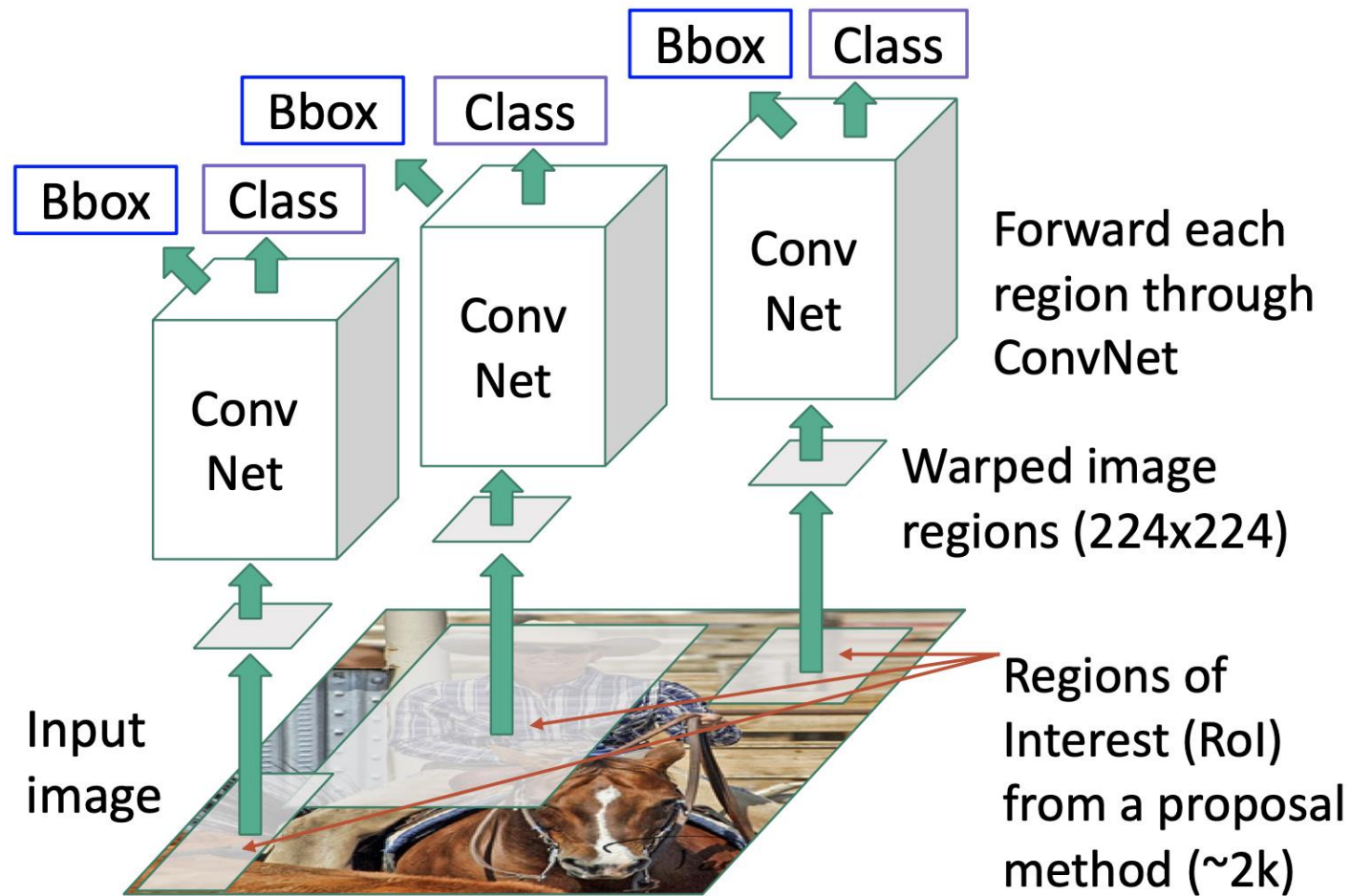
Region-based CNN



Classify each region

Bounding box regression:
Predict “transform” to correct the
RoI: 4 numbers (t_x, t_y, t_h, t_w)

Region-based CNN (R-CNN)



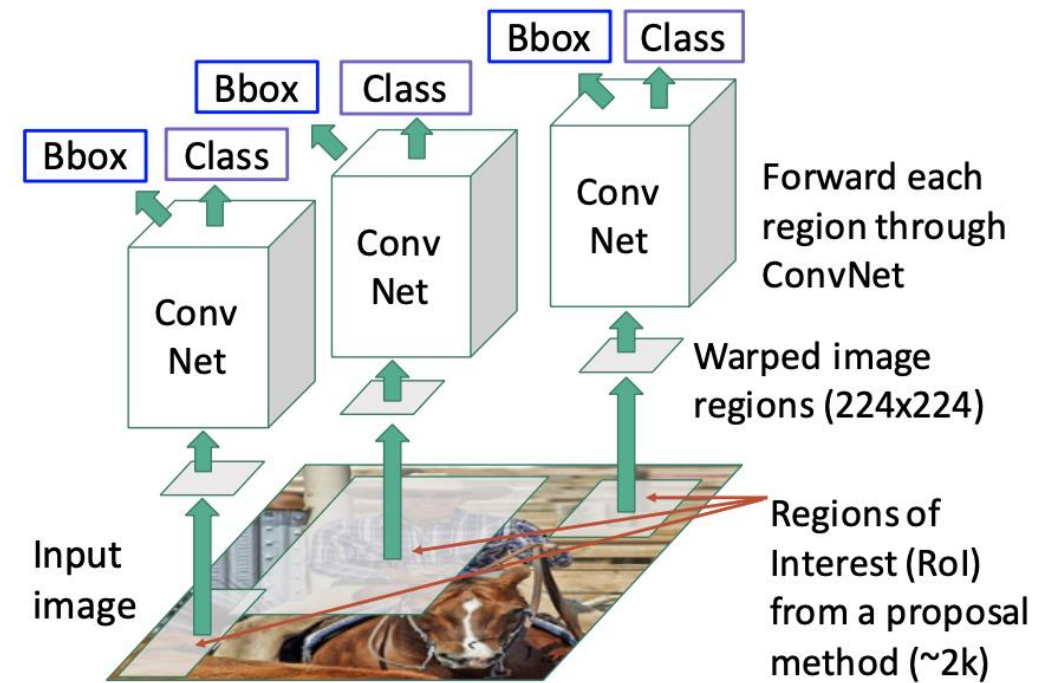
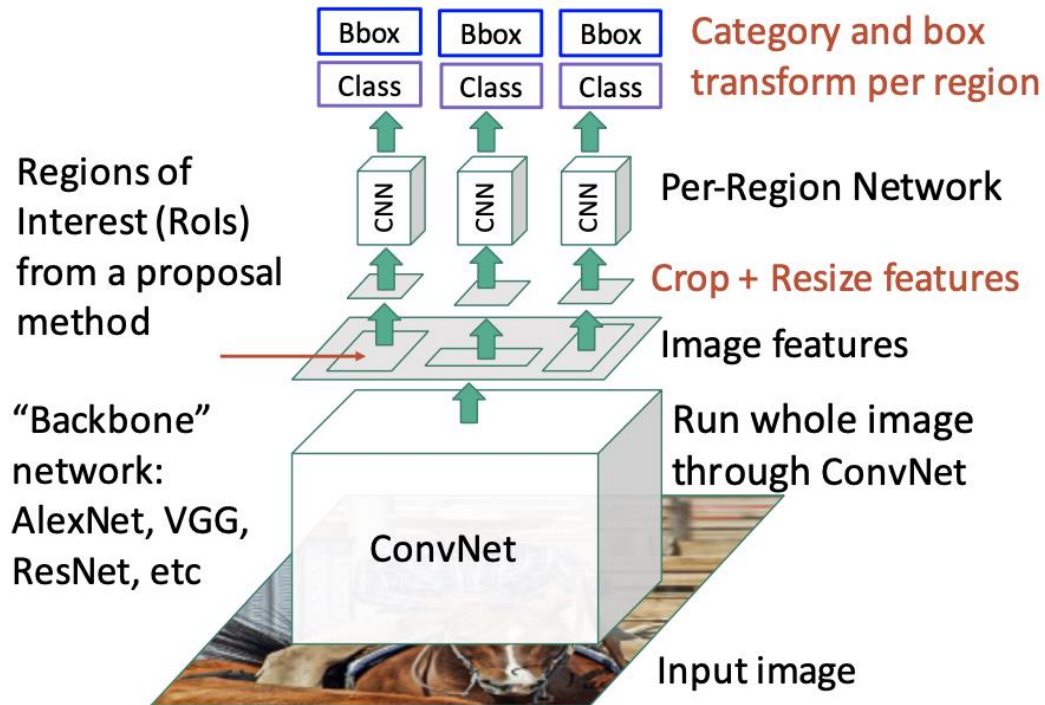
Problem: Very slow! Need to do ~2k forward passes for each image!

Solution: Run CNN **before** warping!

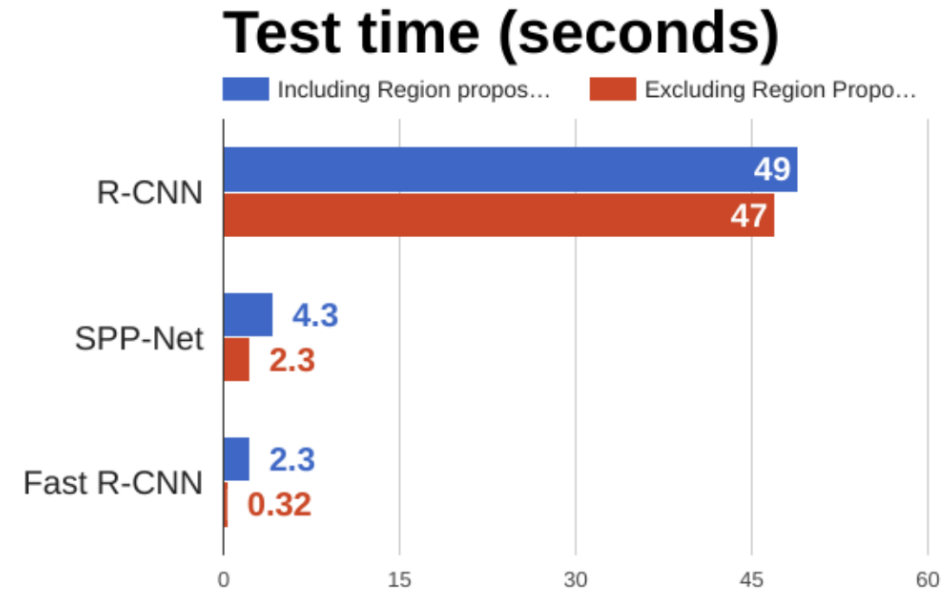
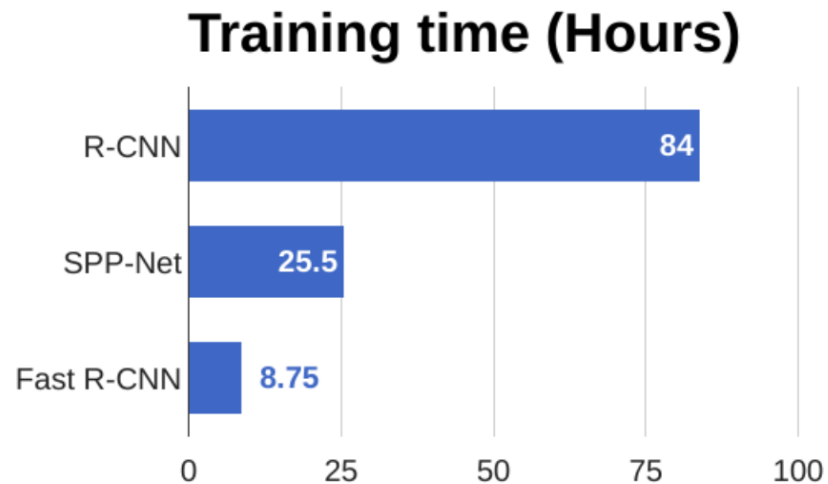
Fast R-CNN vs “Slow” R-CNN

Fast R-CNN: Apply differentiable cropping to shared image features

“Slow” R-CNN: Run CNN independently for each region

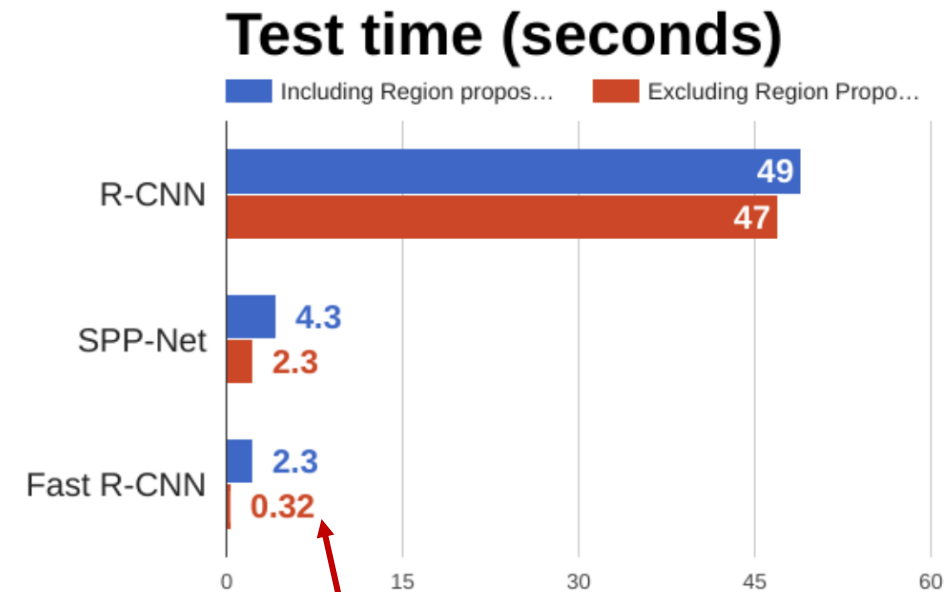
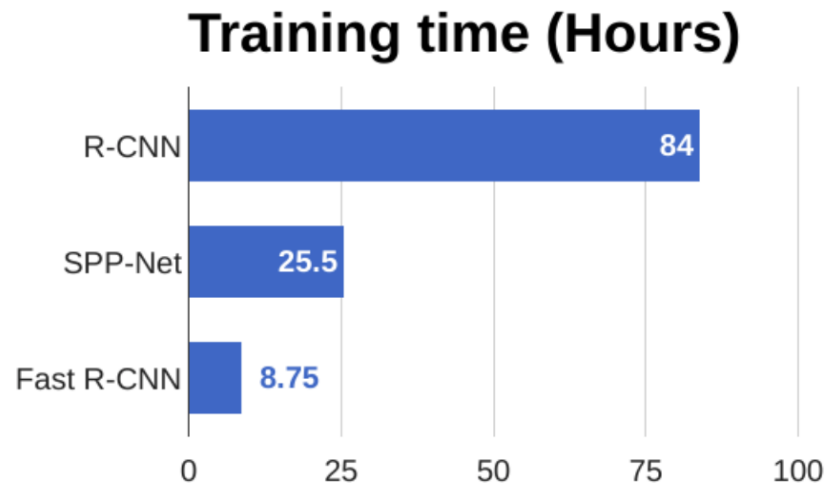


Fast R-CNN vs “Slow” R-CNN



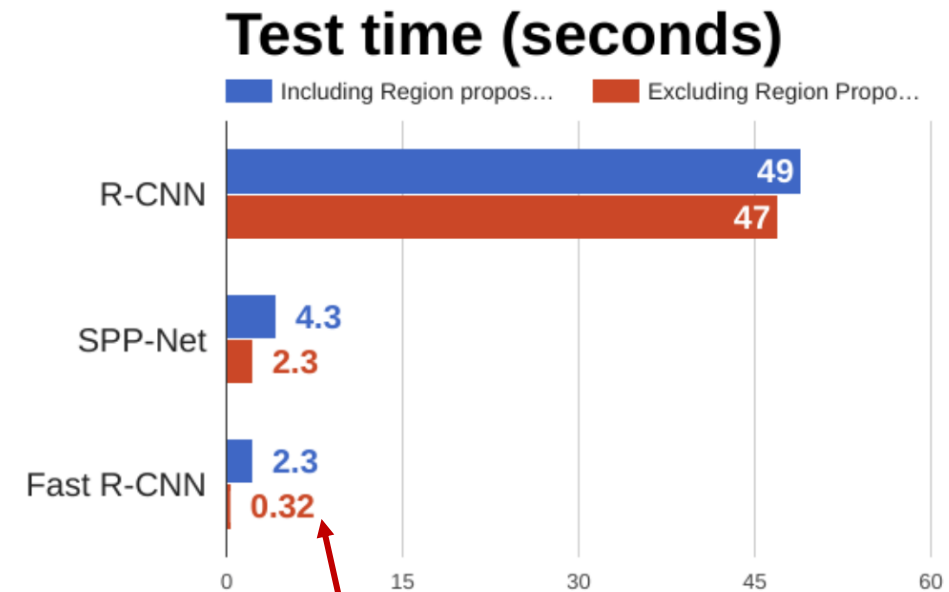
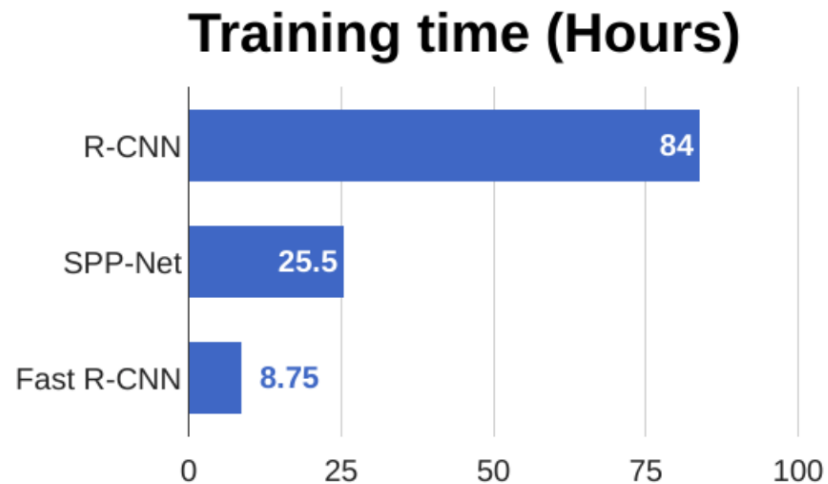
Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014. He et al, “Spatial pyramid pooling in deep convolutional networks for visual recognition”, ECCV 2014
Girshick, “Fast R-CNN”, ICCV 2015

Fast R-CNN vs “Slow” R-CNN



Problem: Runtime dominated by region proposals!

Fast R-CNN vs “Slow” R-CNN



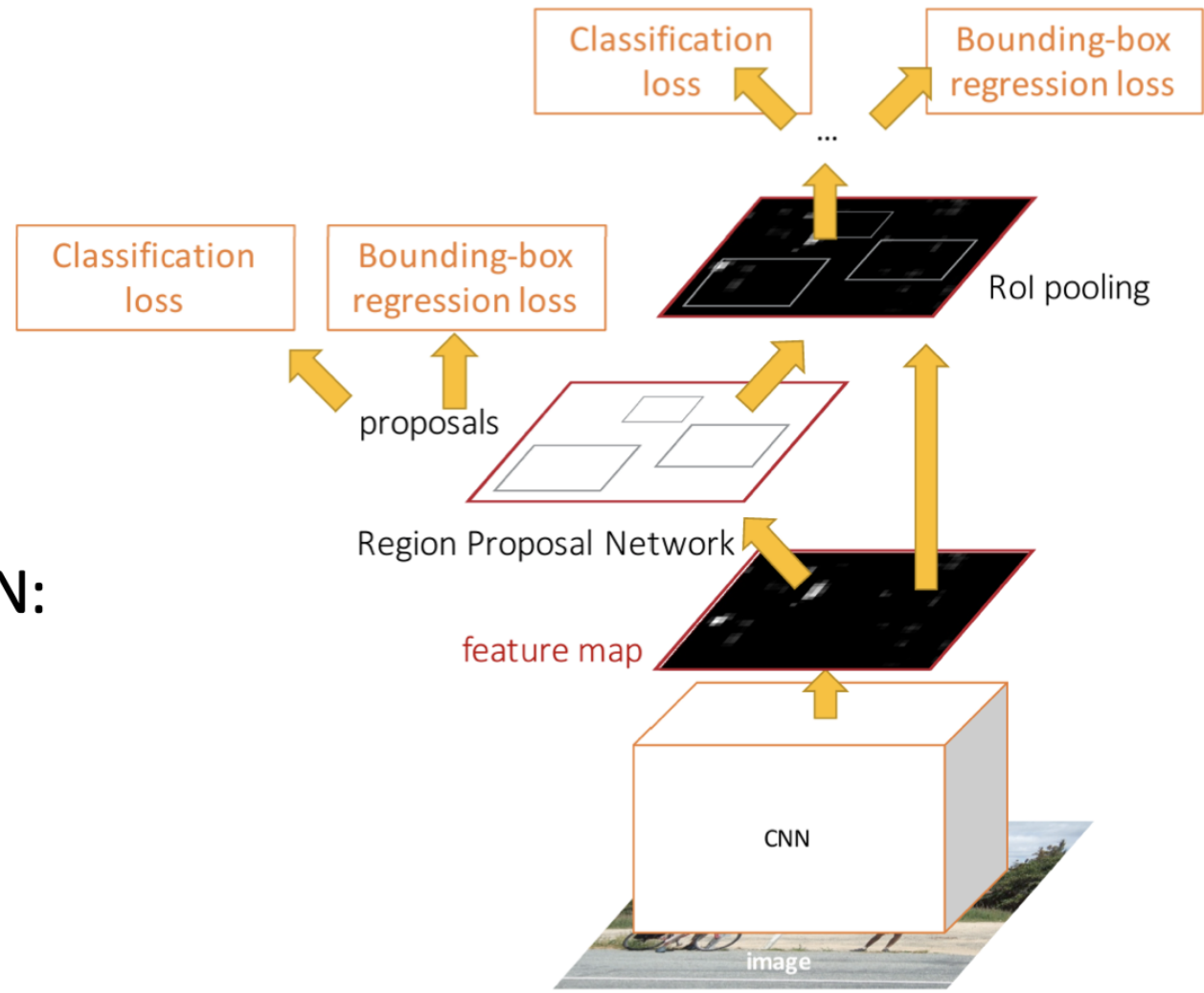
Recall: Region proposals computed by heuristic “Selective Search” algorithm on CPU -- let’s learn them with a CNN instead!

Problem: Runtime dominated by region proposals!

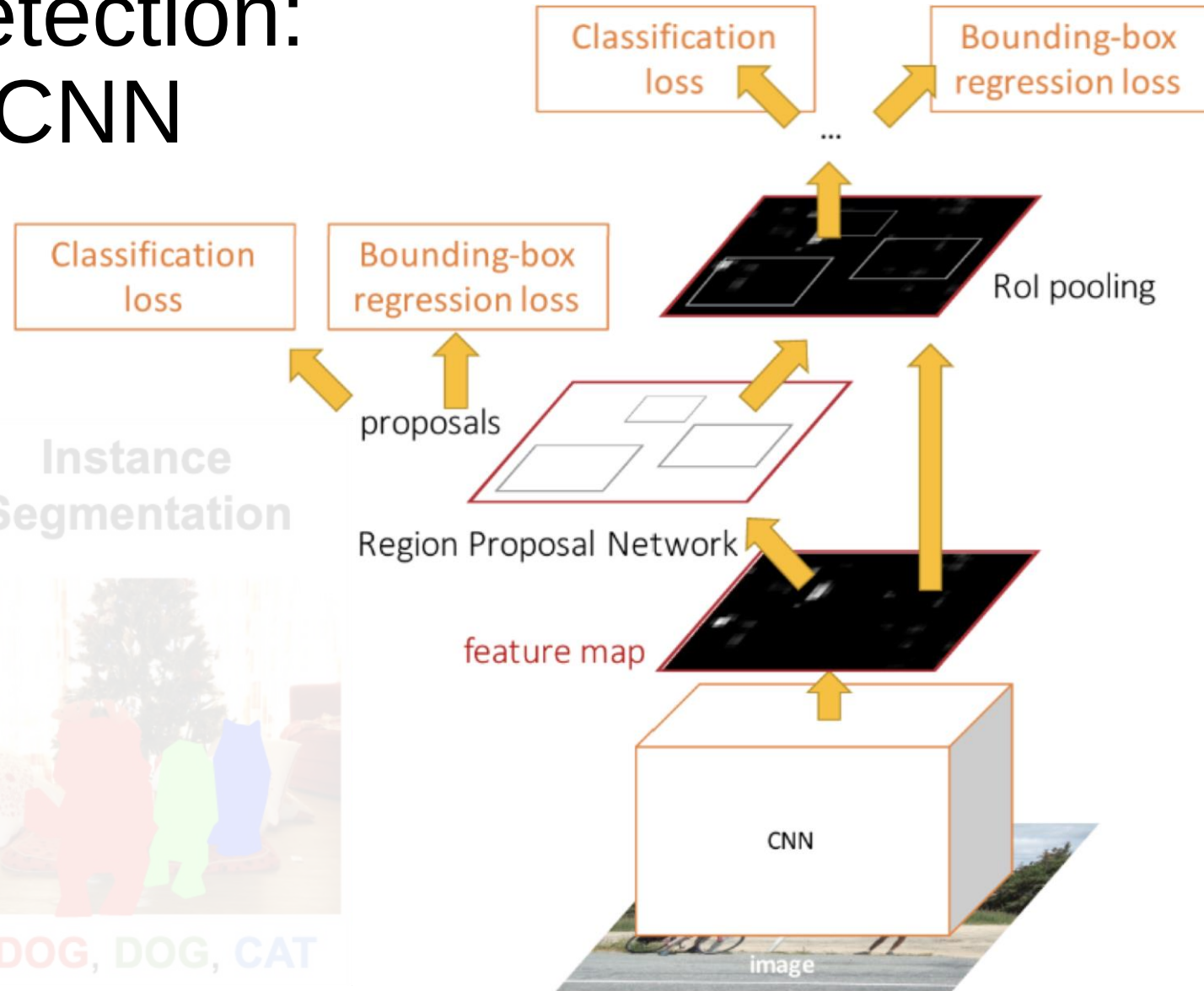
Faster R-CNN: Learnable Region Proposals

Insert **Region Proposal Network (RPN)** to predict proposals from features

Otherwise same as Fast R-CNN:
Crop features for each proposal, classify each one

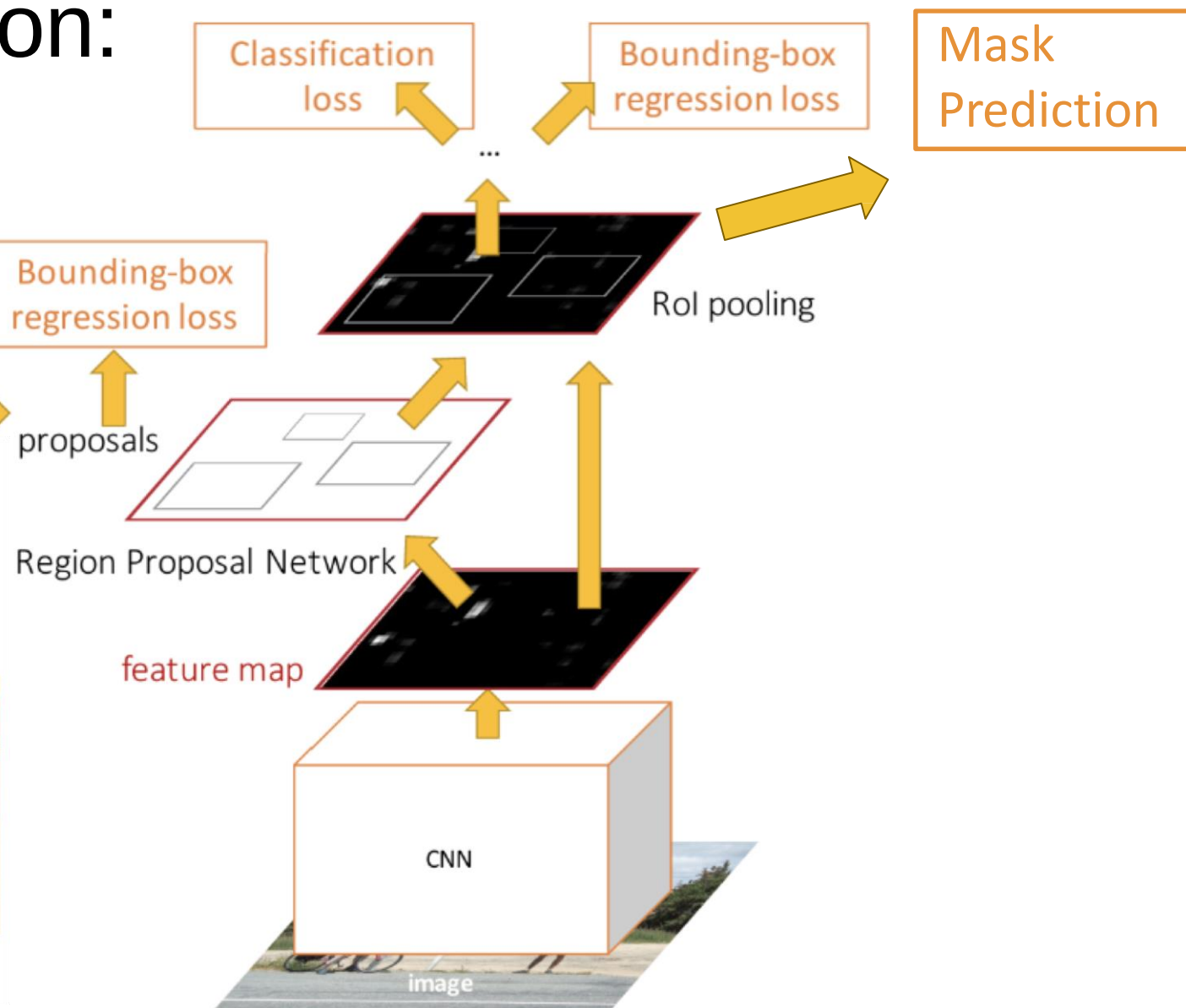


Object Detection: Faster R-CNN

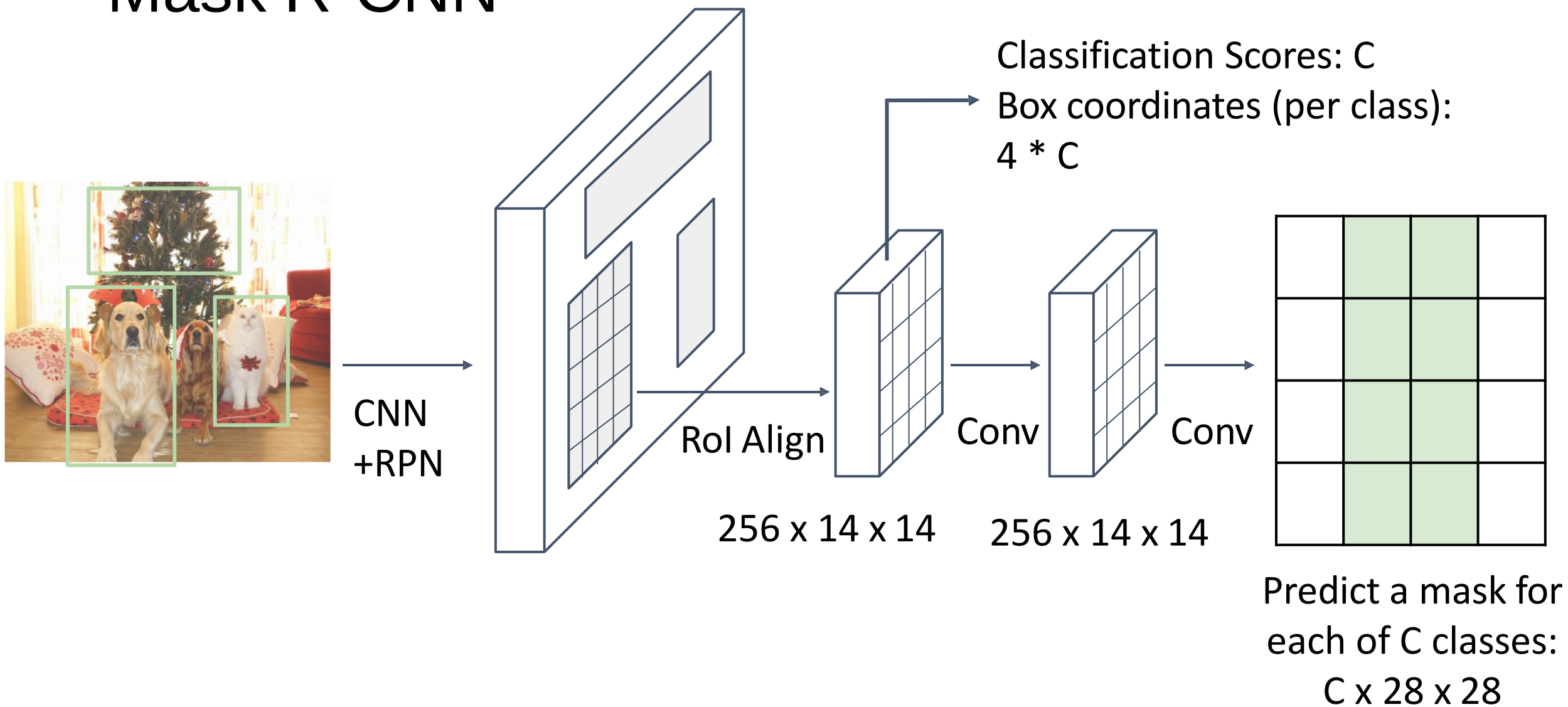


Mask R-CNN

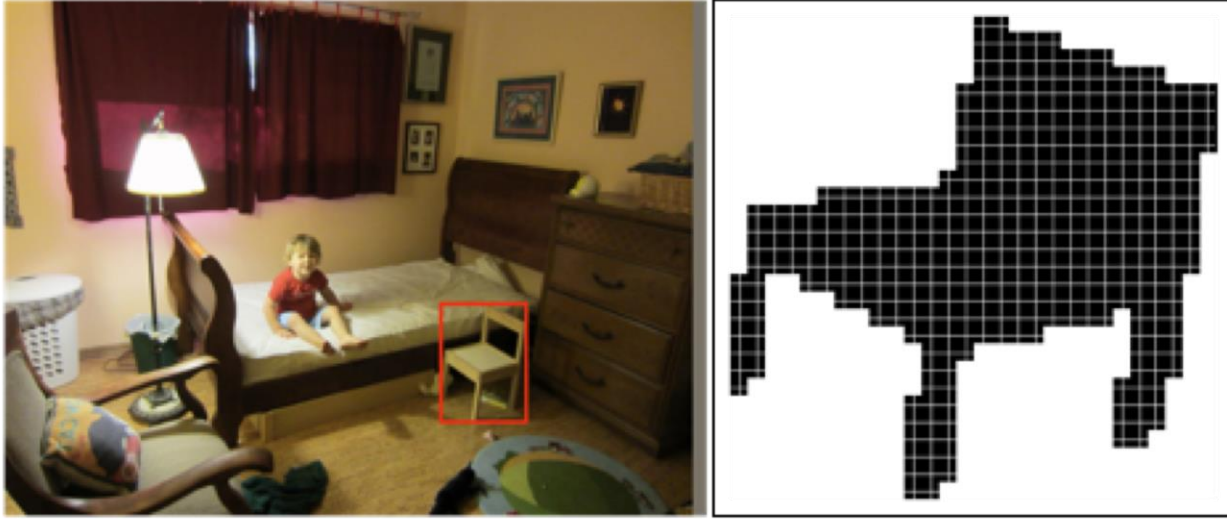
Instance Segmentation: Mask R-CNN



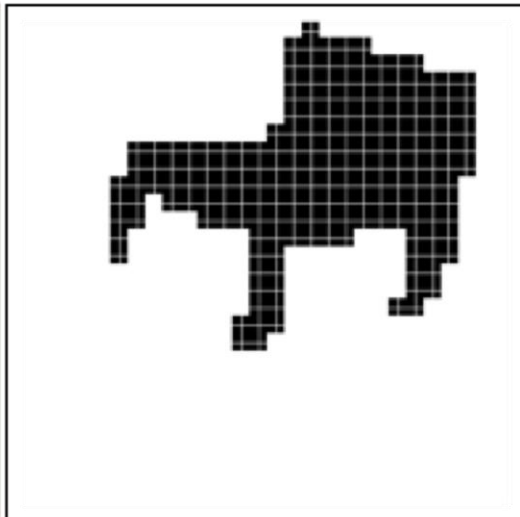
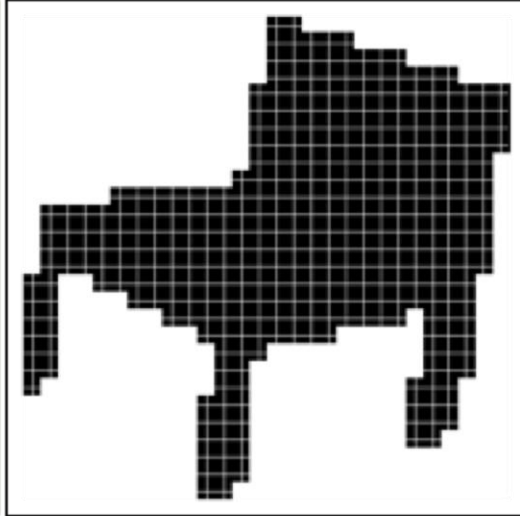
Mask R-CNN



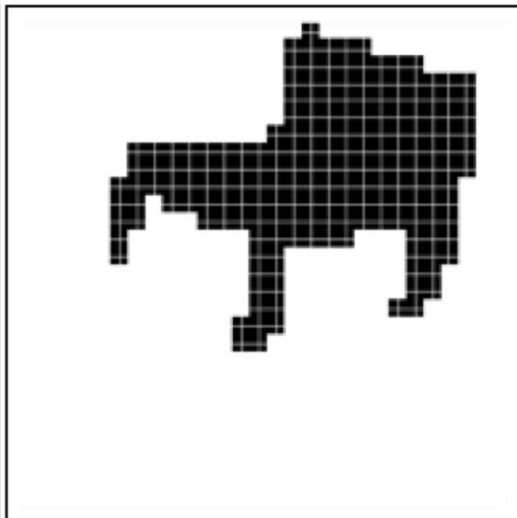
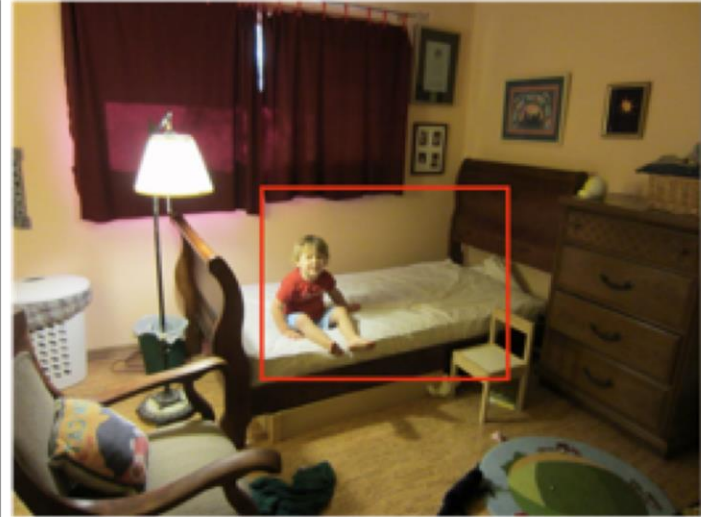
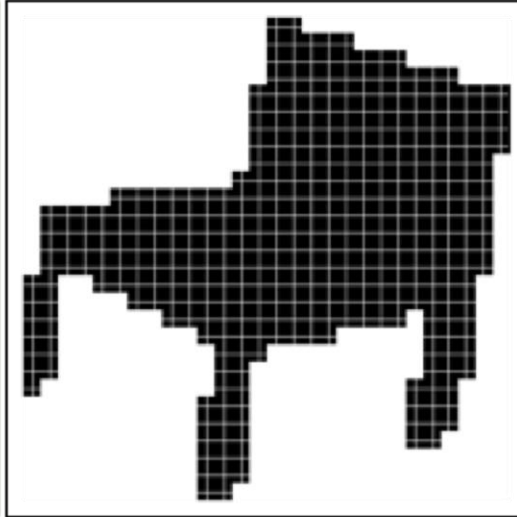
Mask R-CNN: Example Training Targets



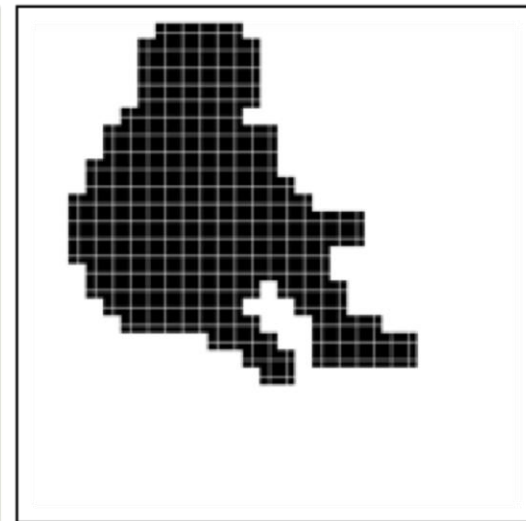
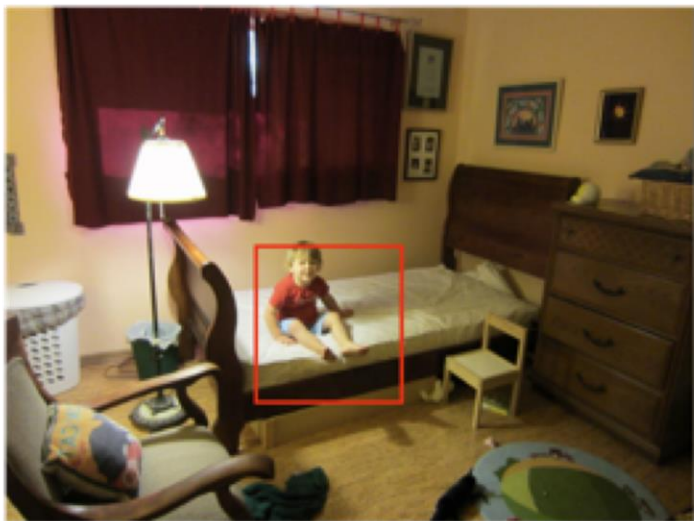
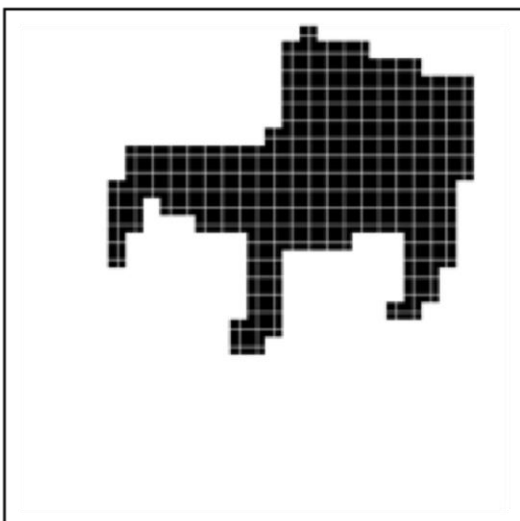
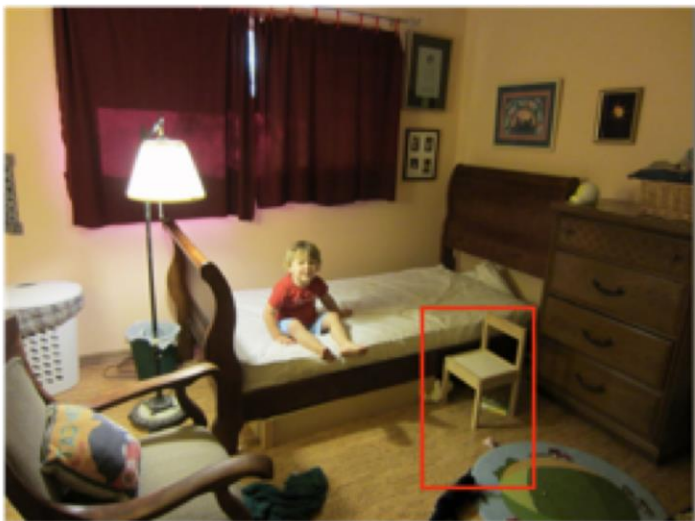
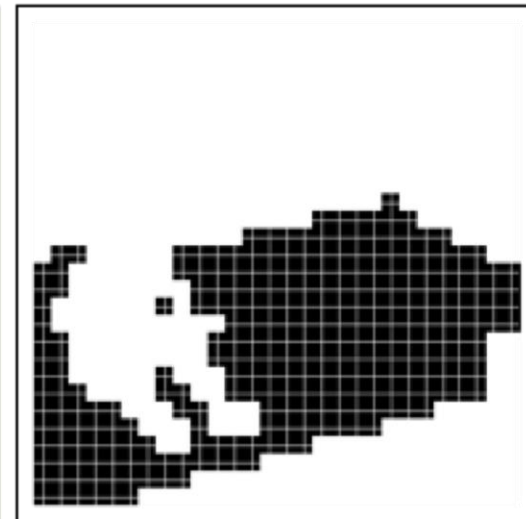
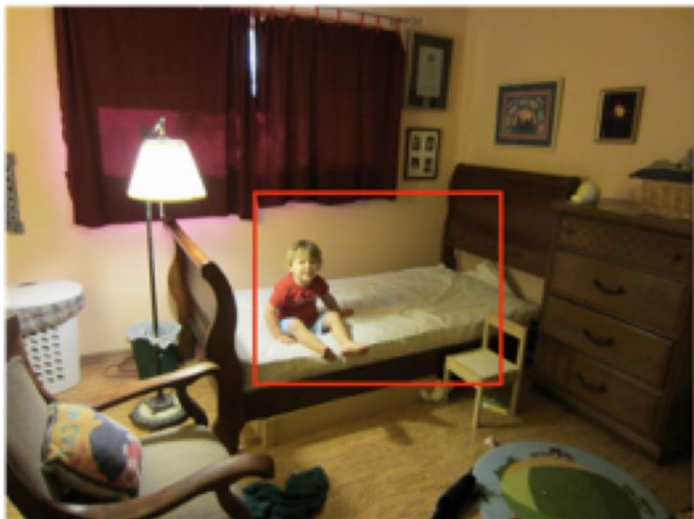
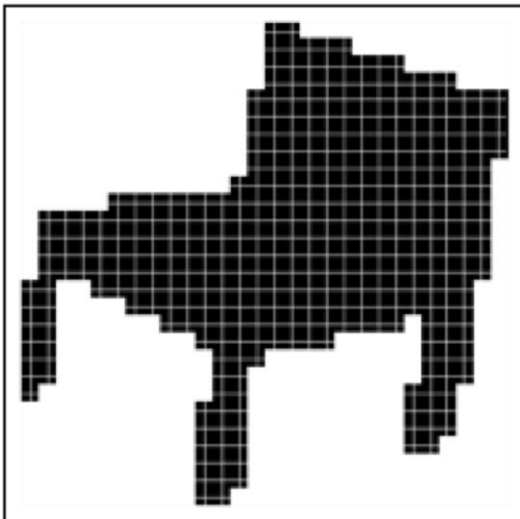
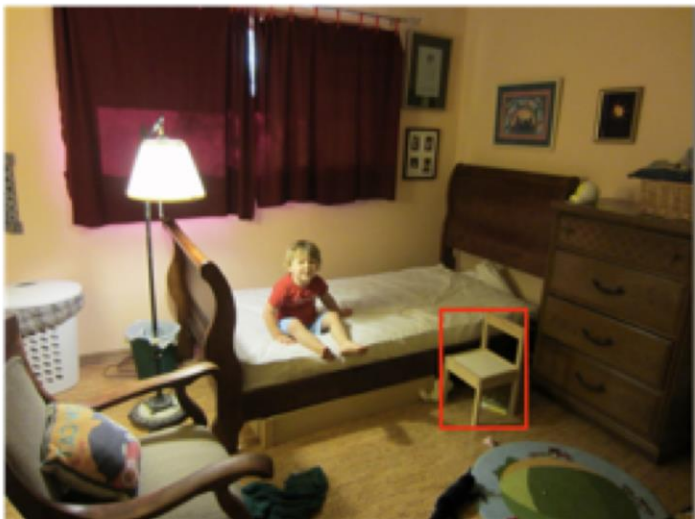
Mask R-CNN: Example Training Targets



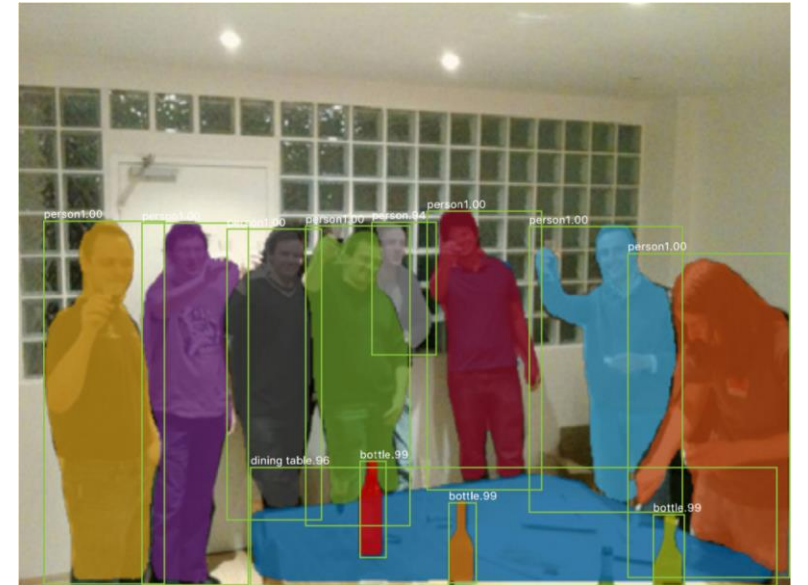
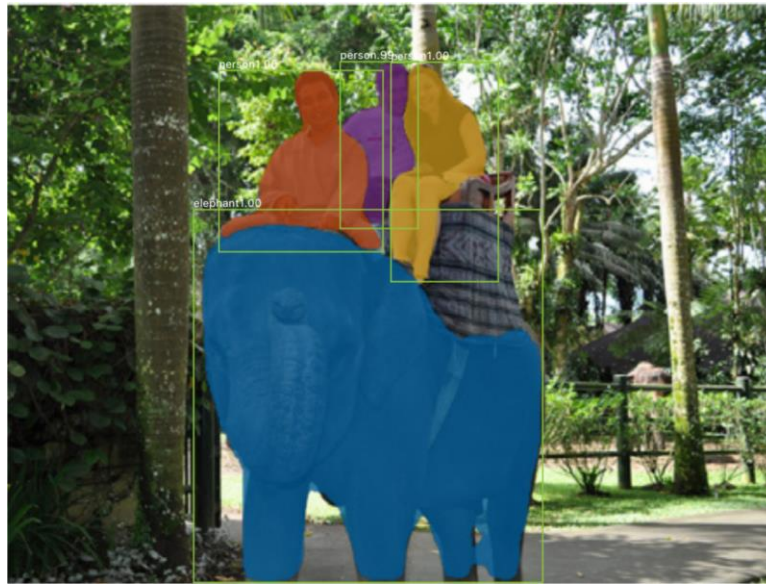
Mask R-CNN: Example Training Targets



Mask R-CNN: Example Training Targets

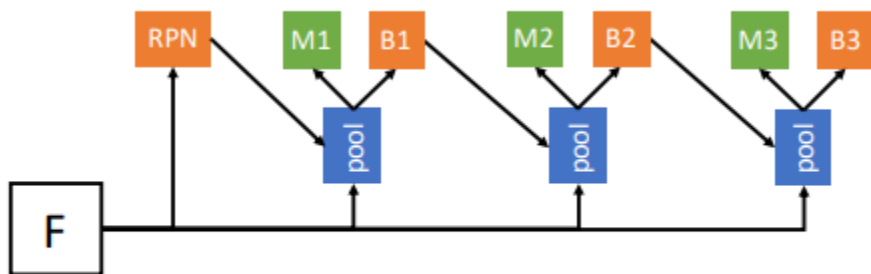


Mask R-CNN: Very Good Results!

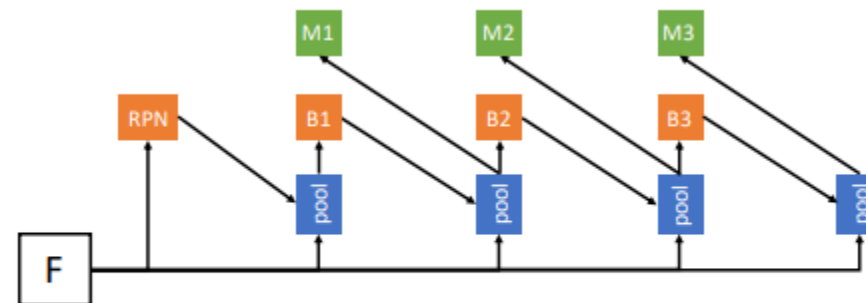


Accurate Classification and Localization

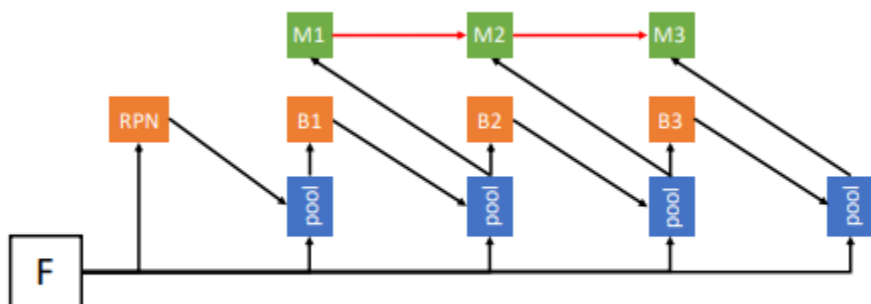
Hybrid Task Cascade



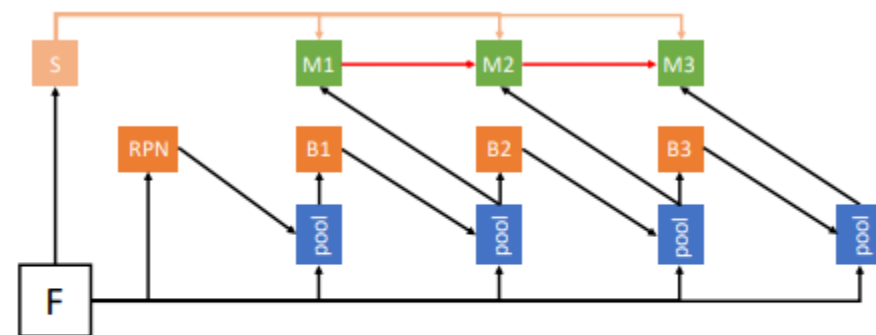
(a) Cascade Mask R-CNN



(b) Interleaved execution



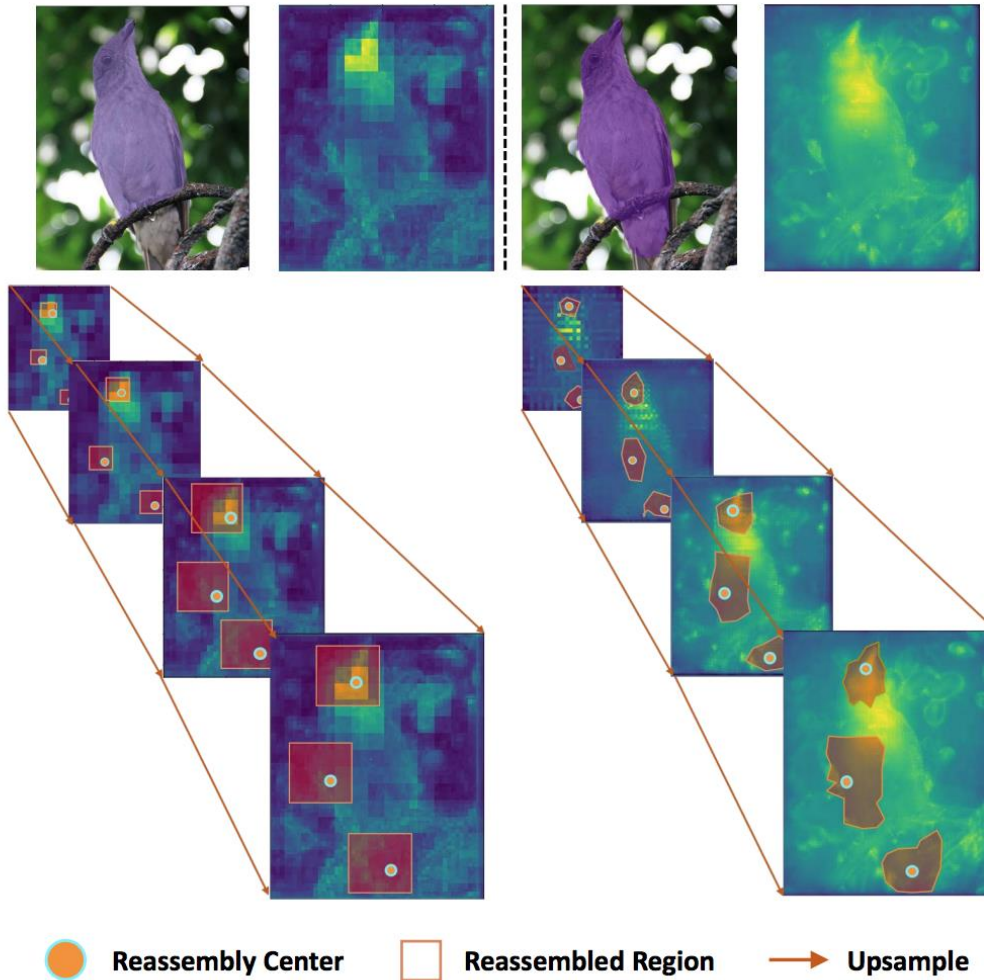
(c) Mask information flow



(d) Hybrid Task Cascade (semantic feature fusion with box branches is not shown on the figure for neat presentation.)

- Information fusion between bbox and mask prediction

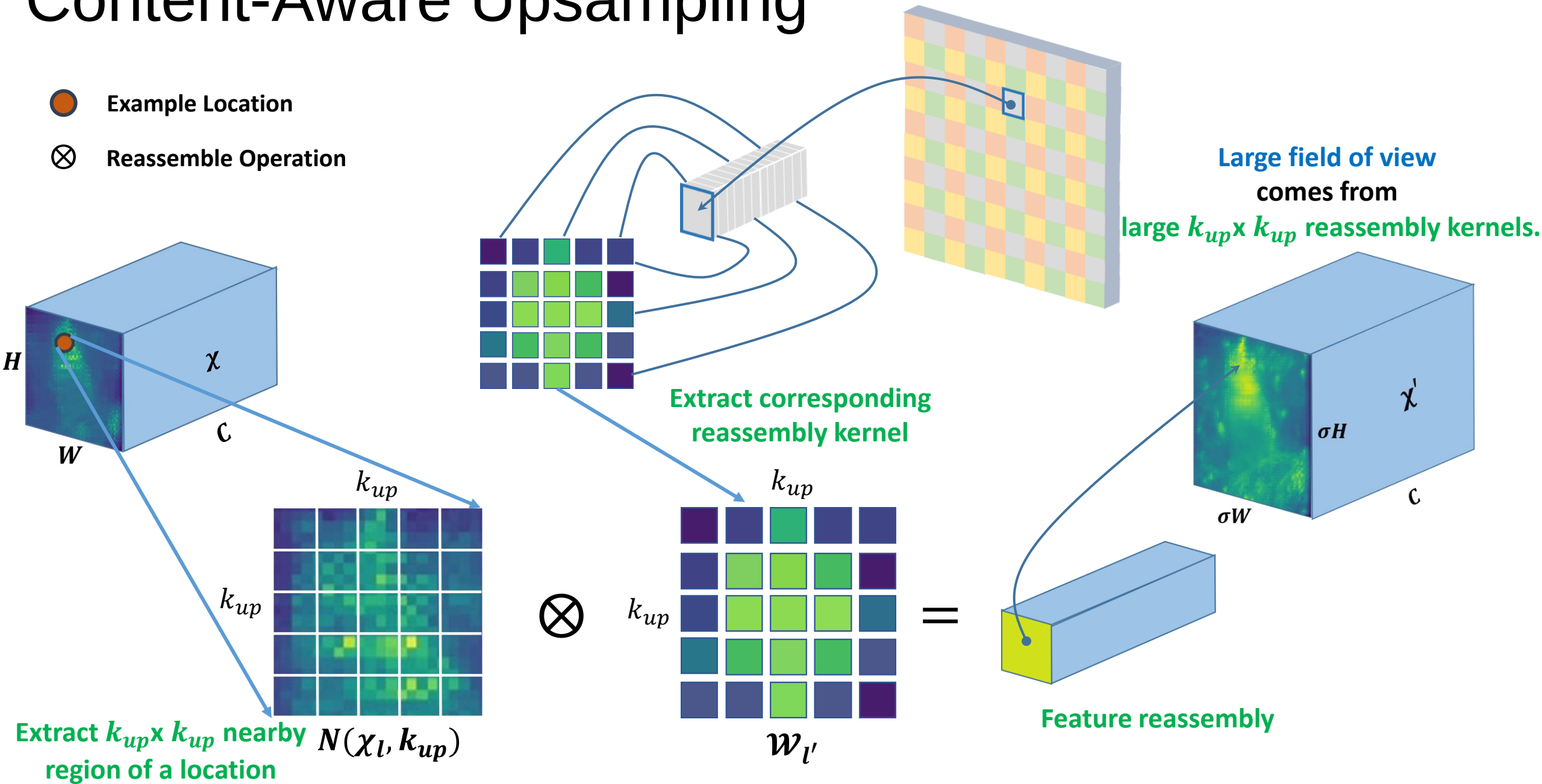
Content-Aware Upsampling



- Feature upsampling using adaptive kernels that incorporate contexts

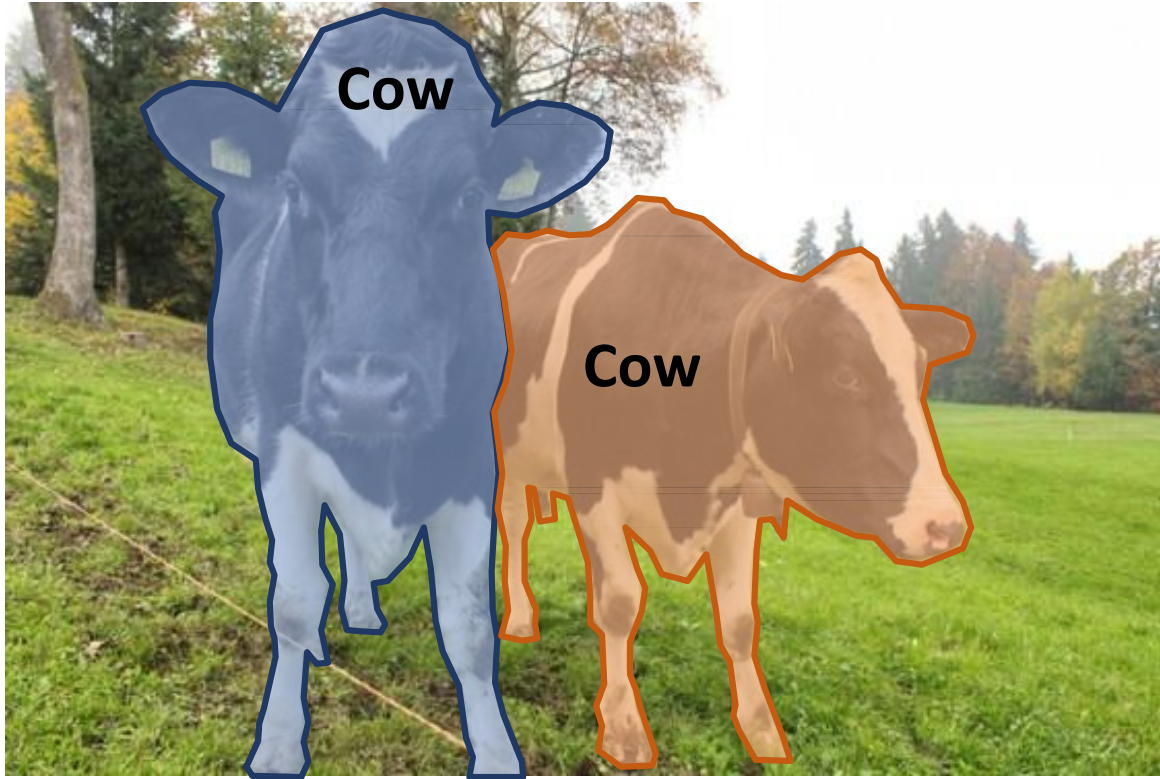
Content-Aware Upsampling

- Example Location
- ⊗ Reassemble Operation

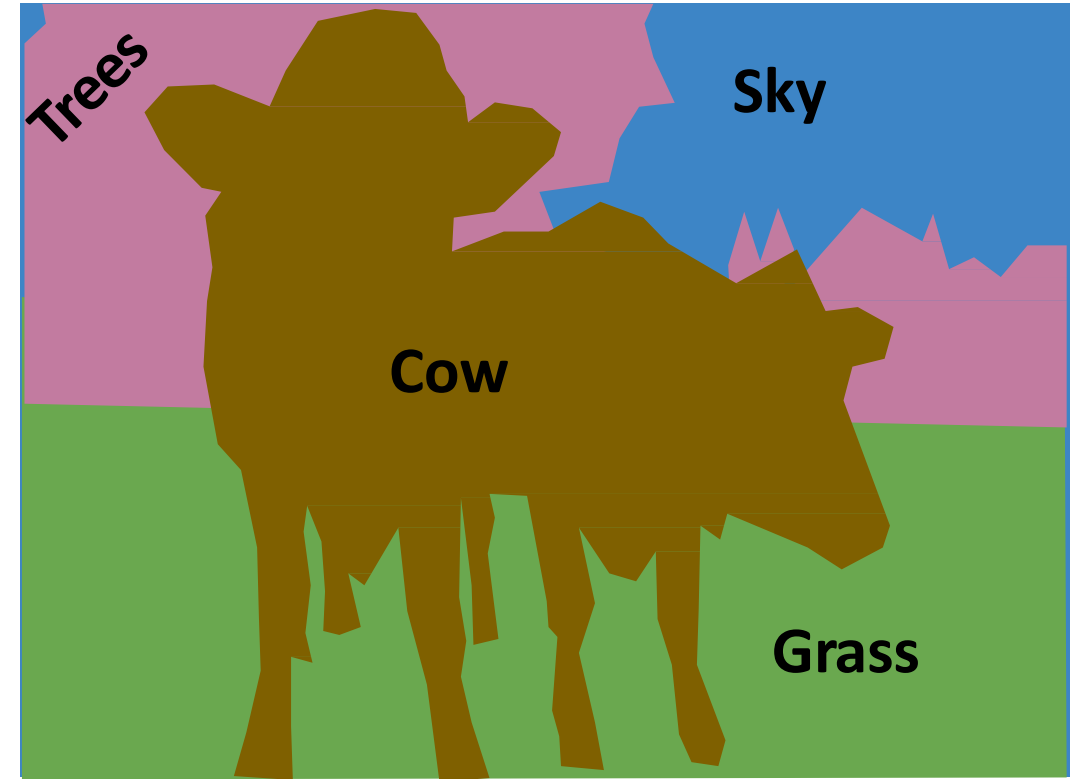


Beyond Instance Segmentation

Instance Segmentation: Separate object instances, but only things



Semantic Segmentation: Identify both things and stuff, but doesn't separate instances

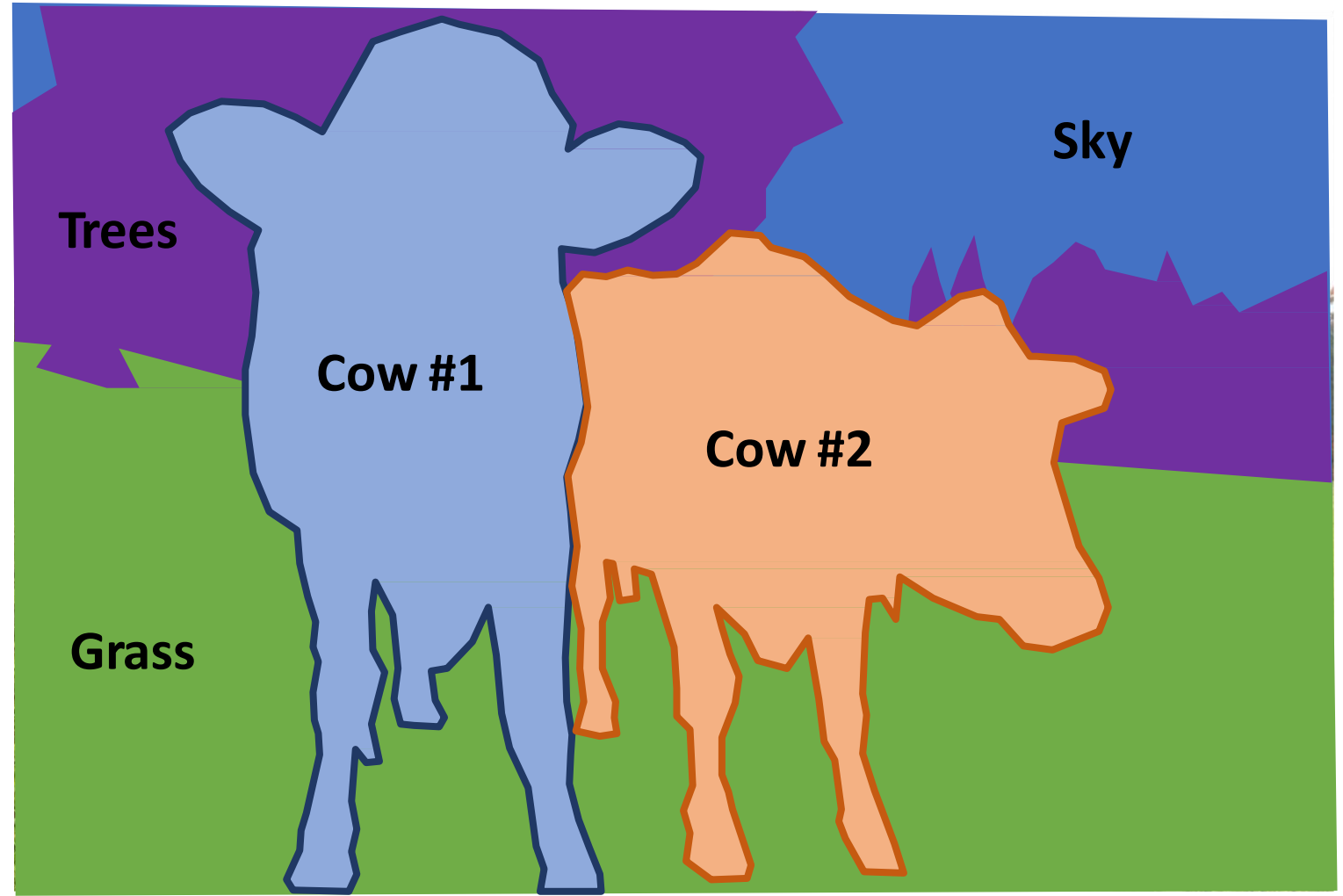


Panoptic Segmentation

Panoptic Segmentation

Label all pixels in the image (both things and stuff)

For “thing” categories also separate into instances



Kirillov et al, “Panoptic Segmentation”, CVPR 2019

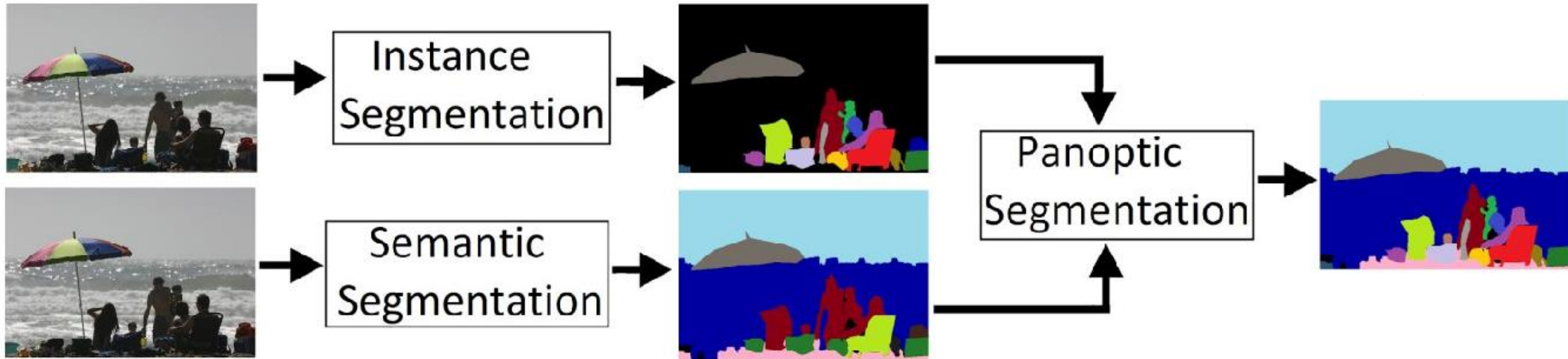
Kirillov et al, “Panoptic Feature Pyramid Networks”, CVPR 2019

Panoptic Segmentation



Kirillov et al, "Panoptic Feature Pyramid Networks", CVPR 2019

Panoptic Segmentation



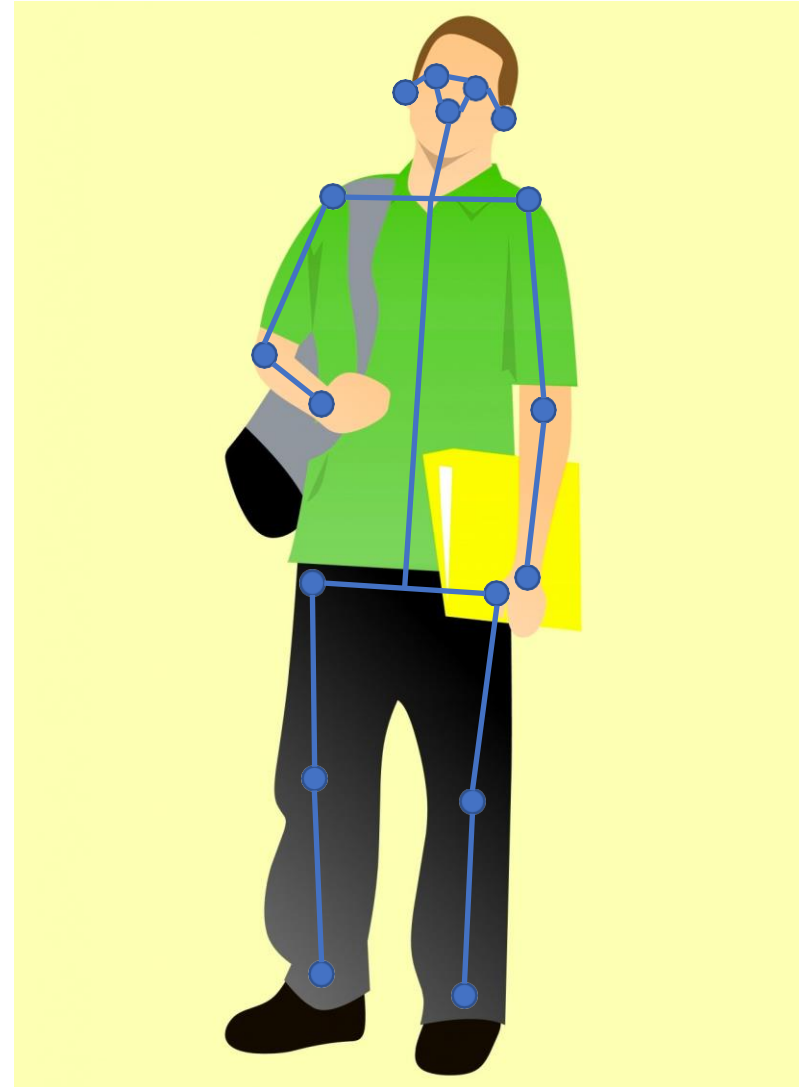
Joint Mask + X Prediction

Beyond Instance Segmentation: Human Keypoints

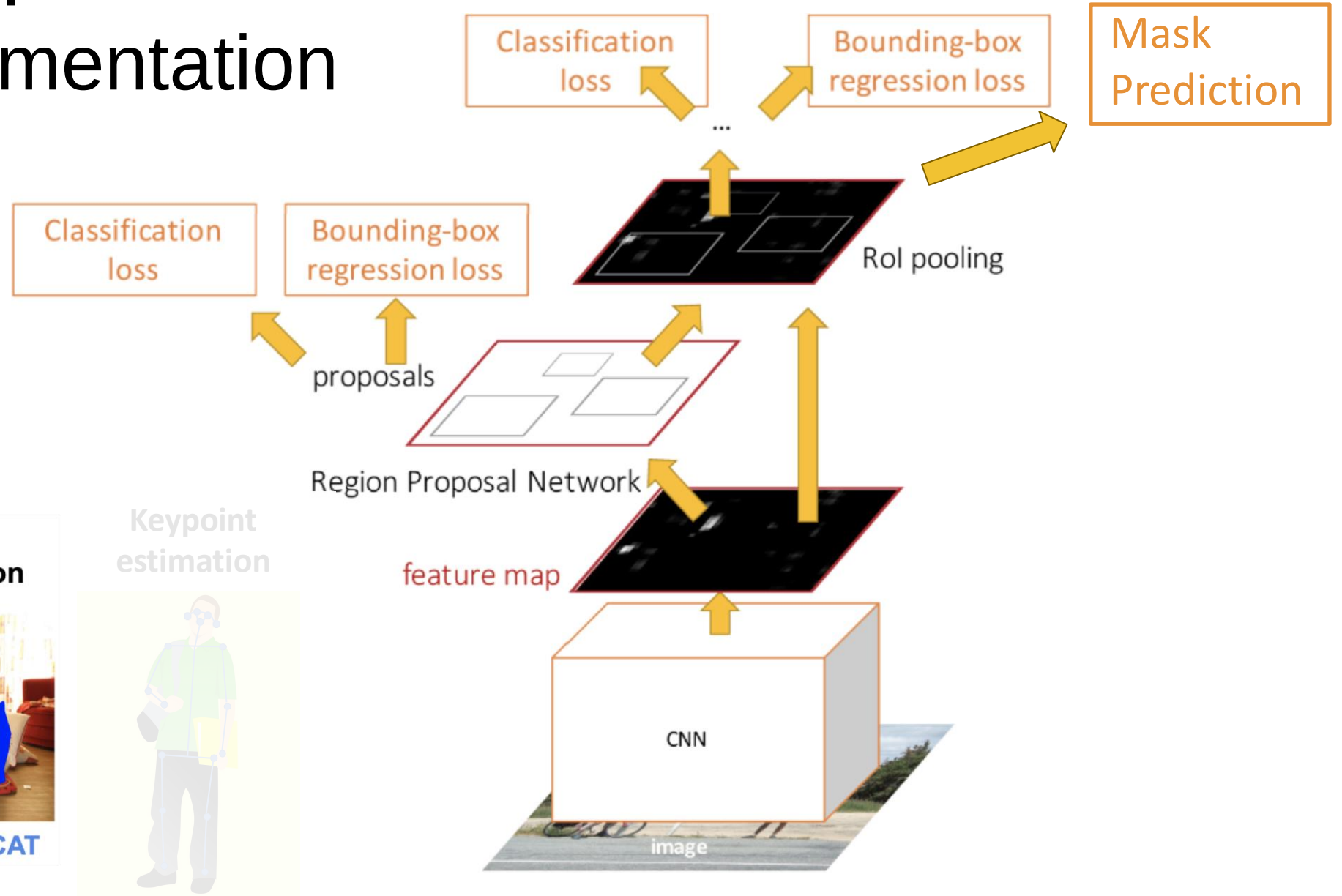
Represent the pose of a human by locating a set of **keypoints**

e.g. 17 keypoints:

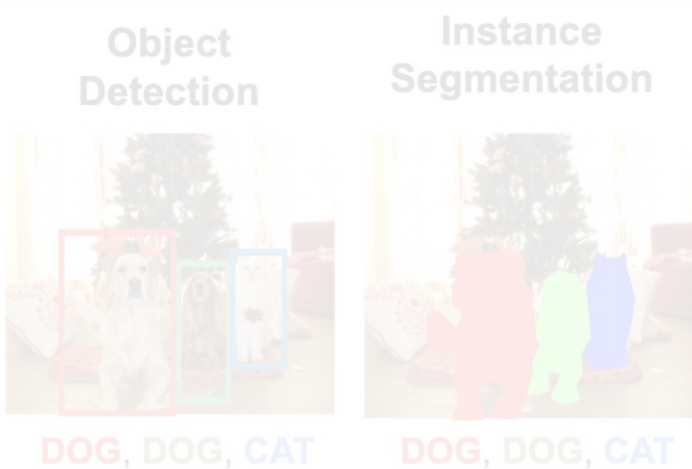
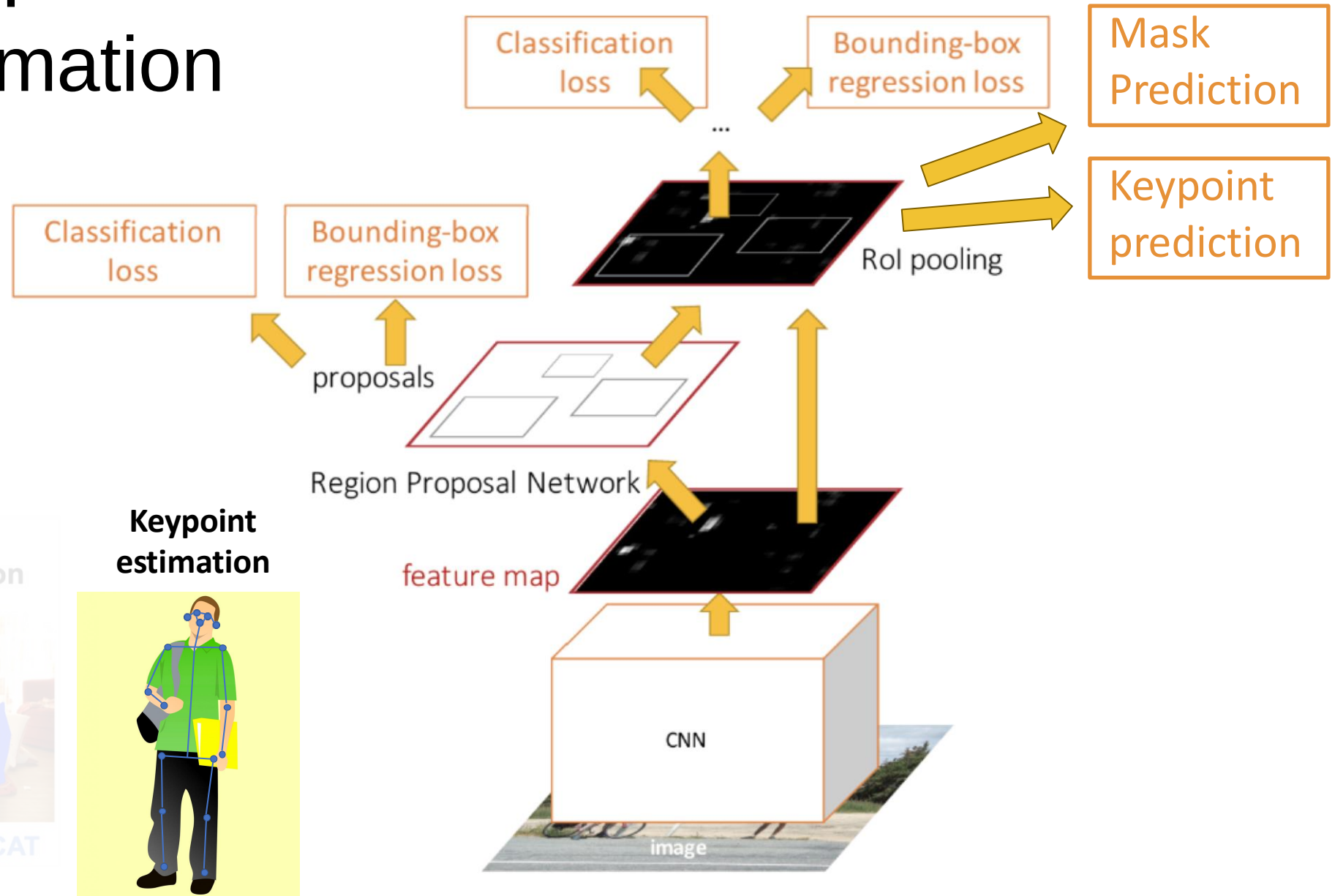
- Nose
- Left / Right eye
- Left / Right ear
- Left / Right shoulder
- Left / Right elbow
- Left / Right wrist
- Left / Right hip
- Left / Right knee
- Left / Right ankle



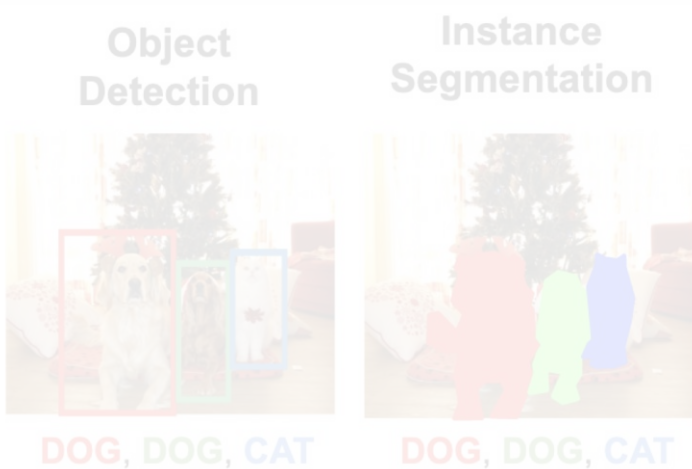
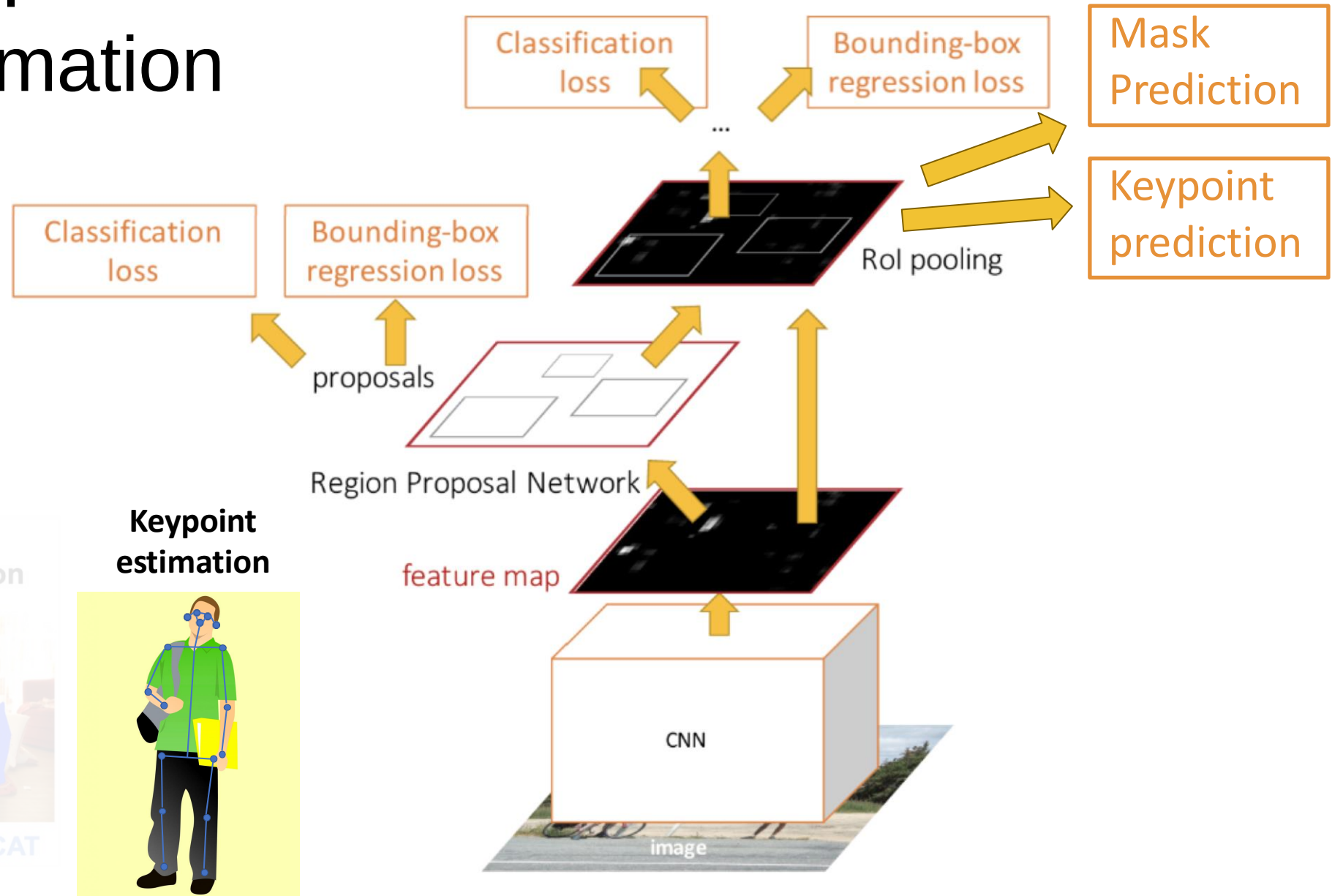
Mask R-CNN: Instance Segmentation



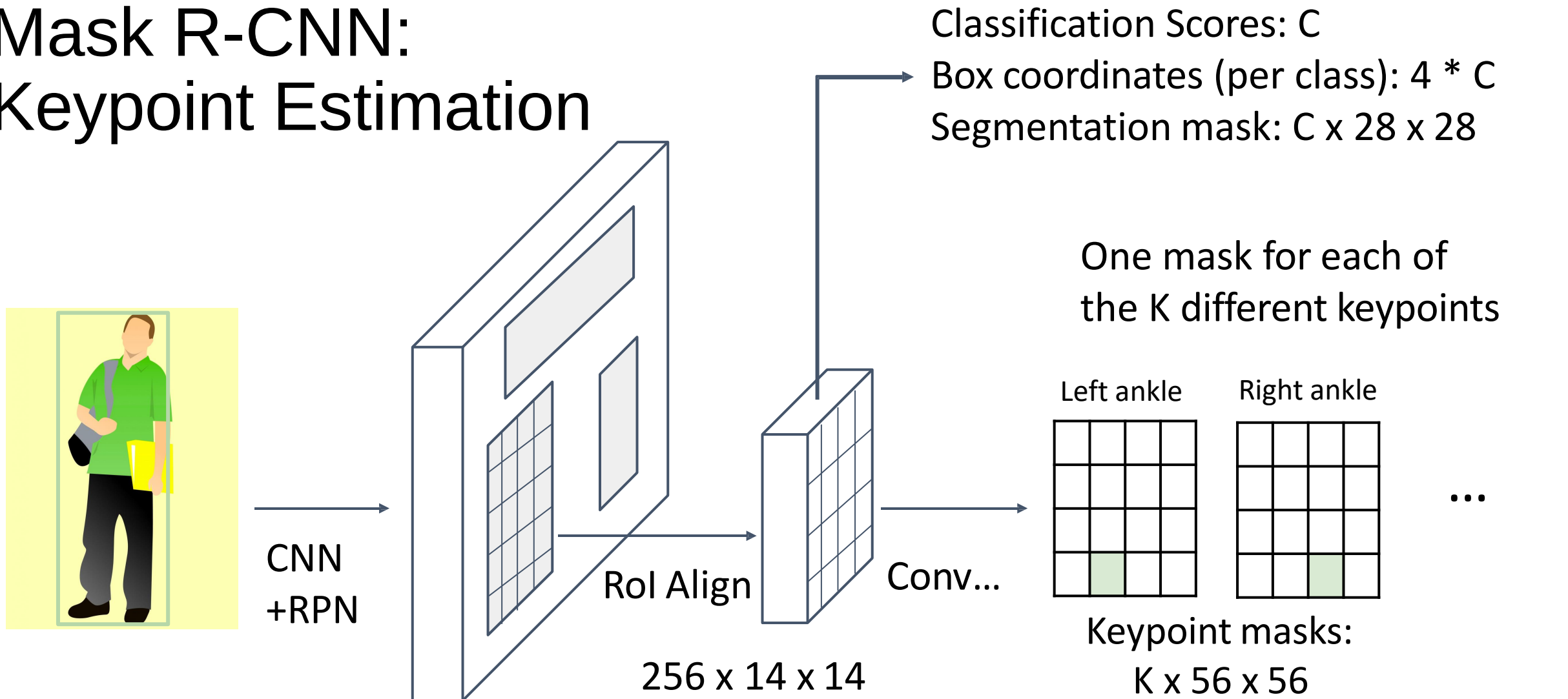
Mask R-CNN: Keypoint Estimation



Mask R-CNN: Keypoint Estimation

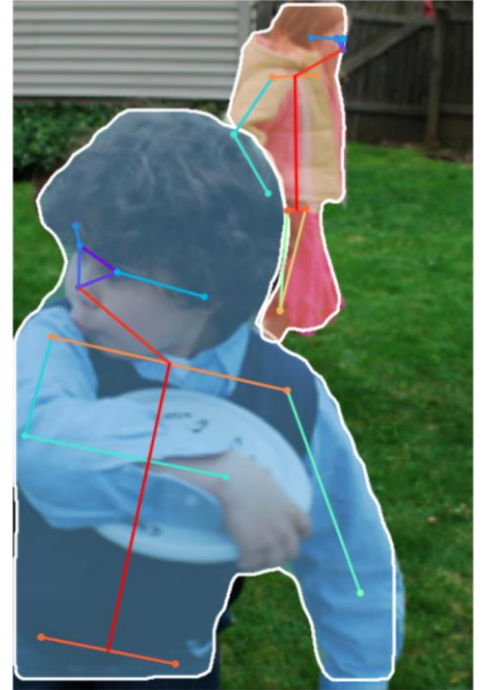
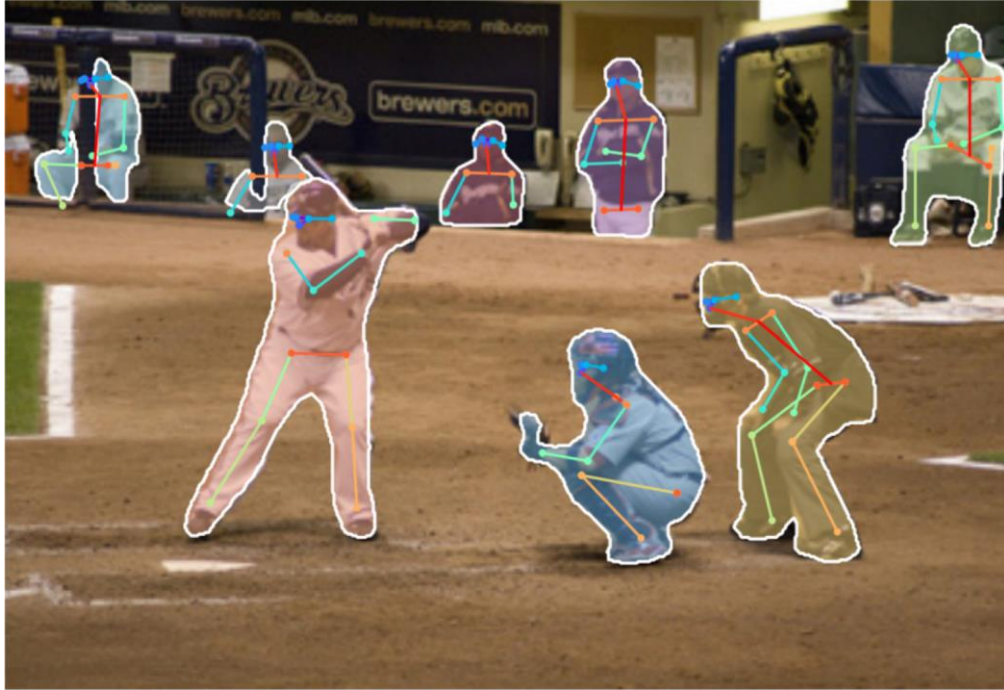


Mask R-CNN: Keypoint Estimation

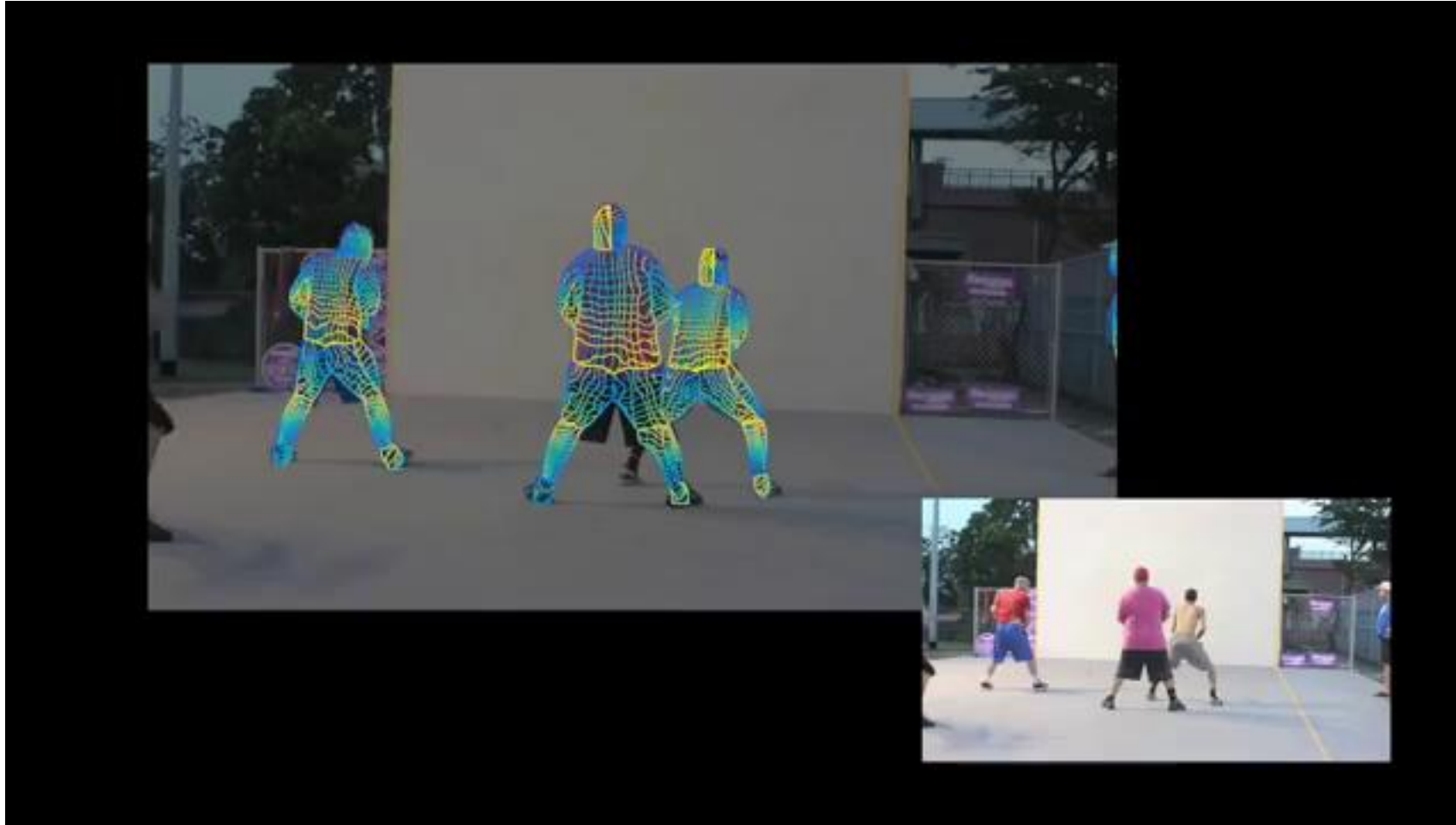


Ground-truth has one “pixel” turned on per keypoint. Train with softmax loss

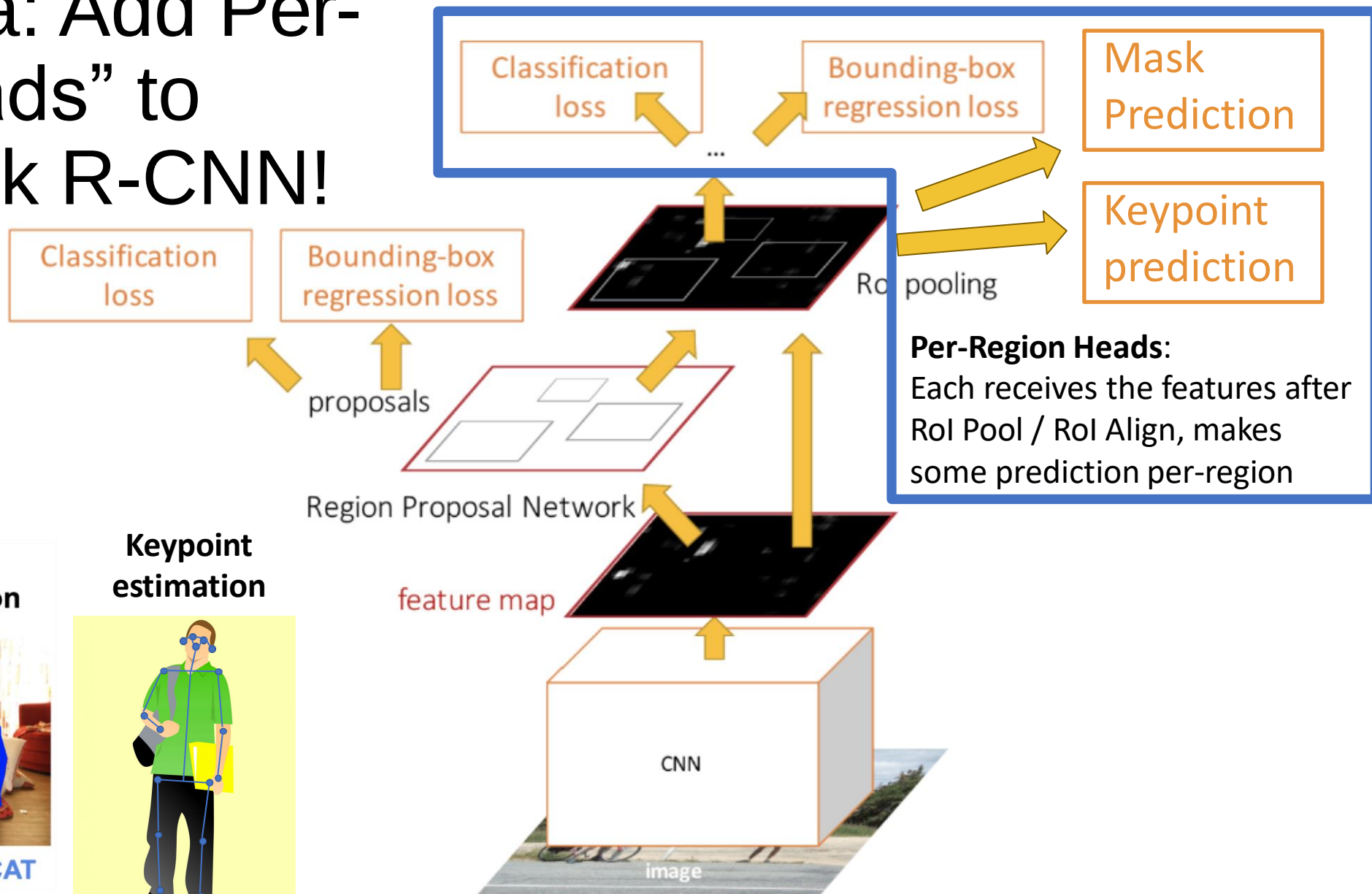
Joint Instance Segmentation and Pose Estimation



Joint Instance Segmentation and Pose Estimation



General Idea: Add Per-Region “Heads” to Faster / Mask R-CNN!



Object Detection



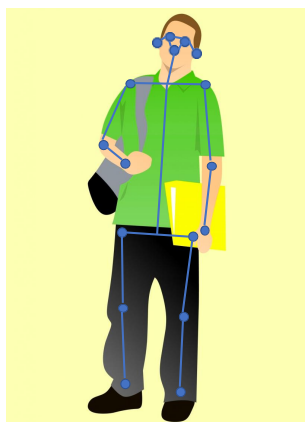
DOG, DOG, CAT

Instance Segmentation

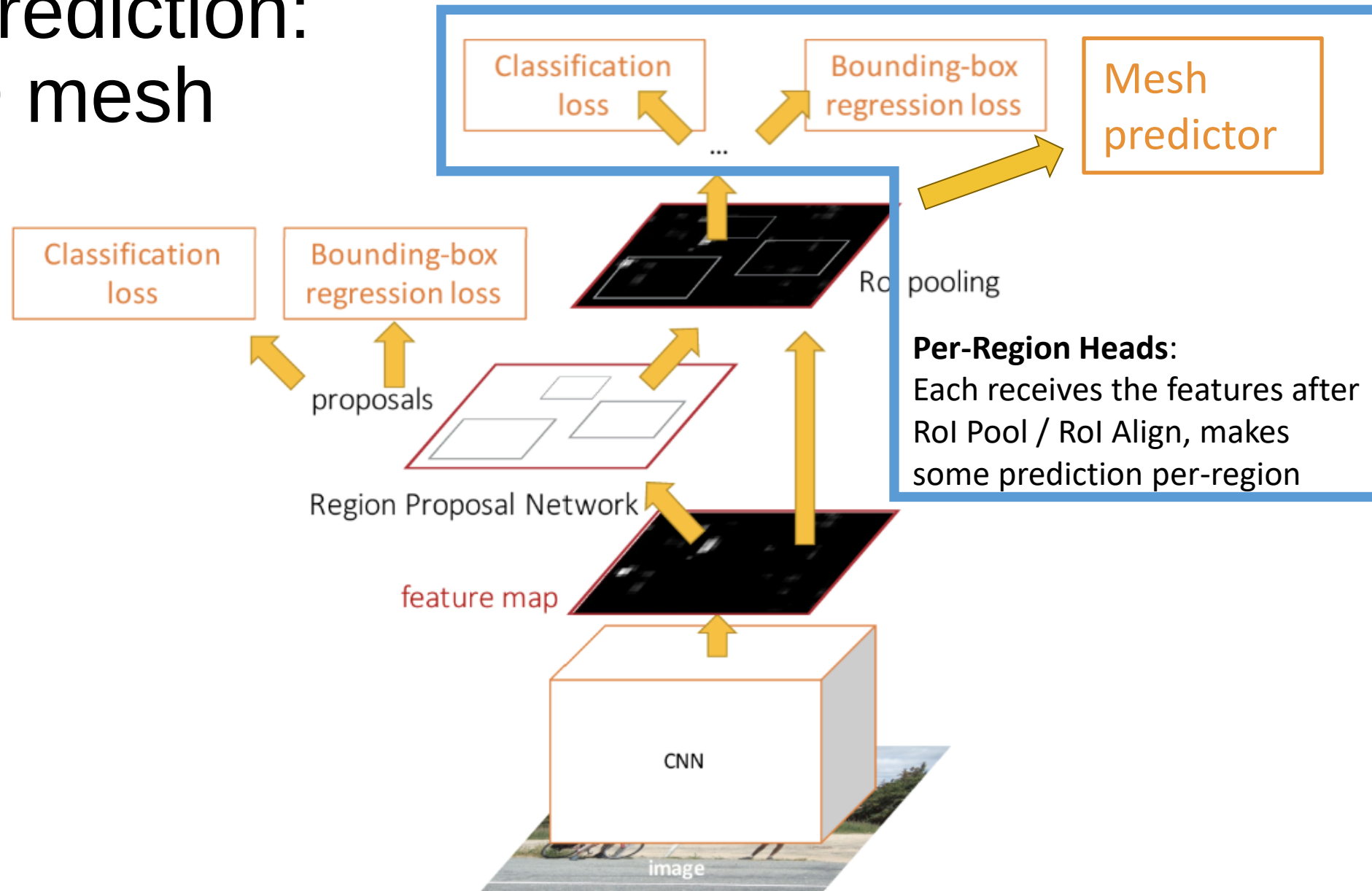


DOG, DOG, CAT

Keypoint estimation



3D Shape Prediction: Predict a 3D mesh per region!

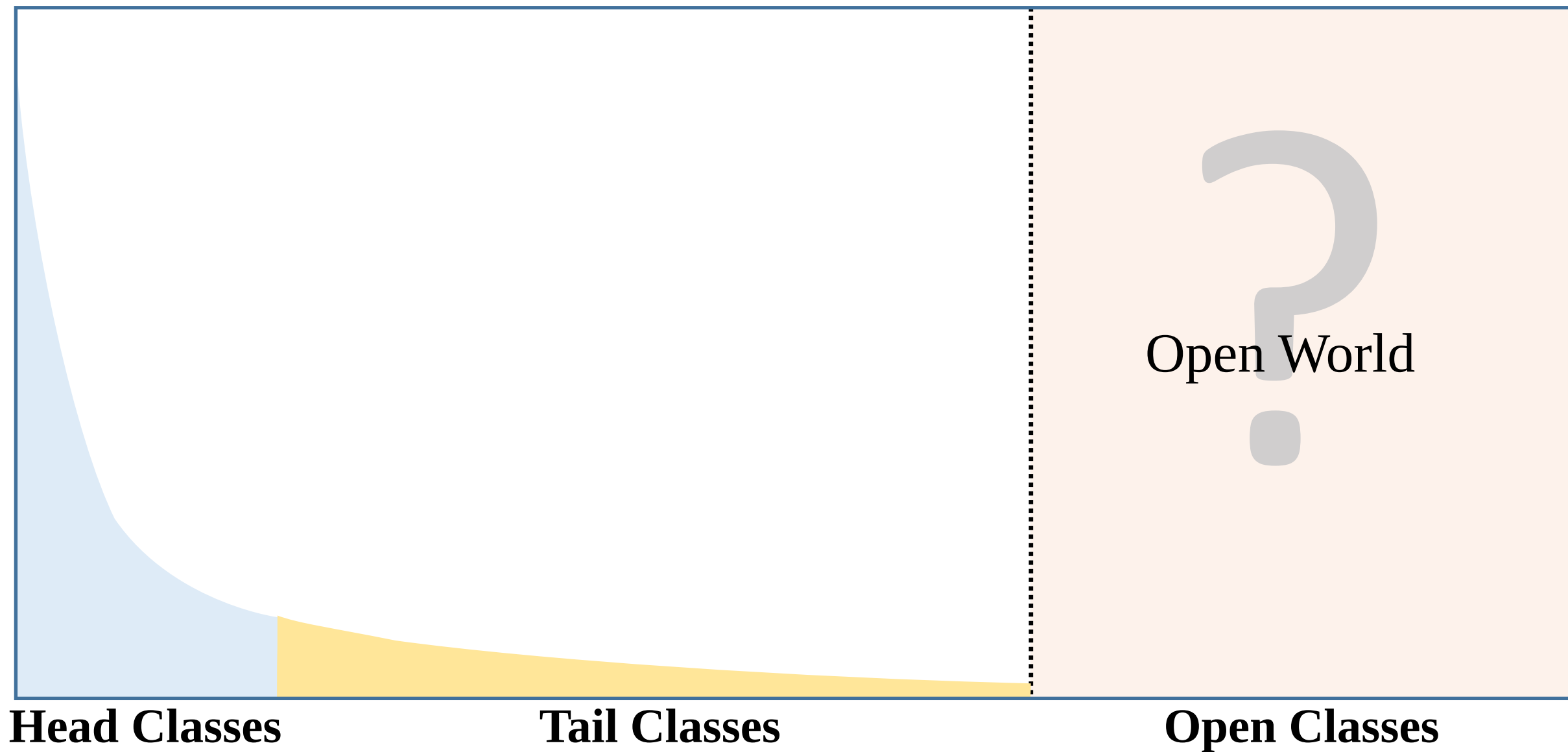


Open Problems

Problem I:

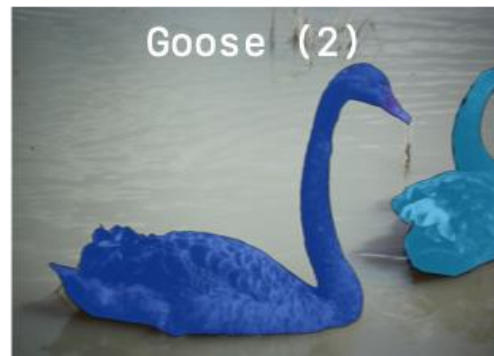
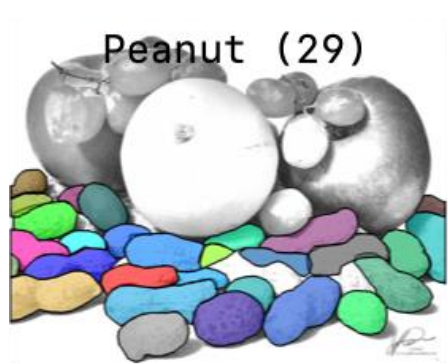
The Devils are in the Tail

Open Long-Tailed Recognition

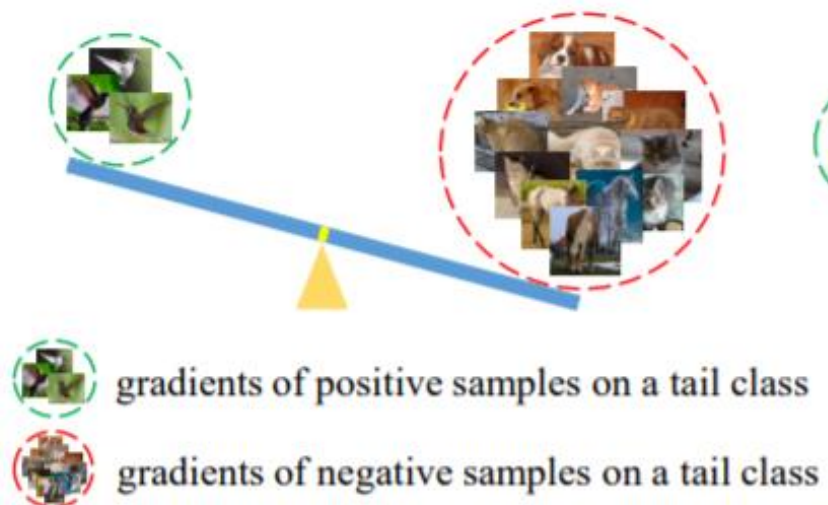




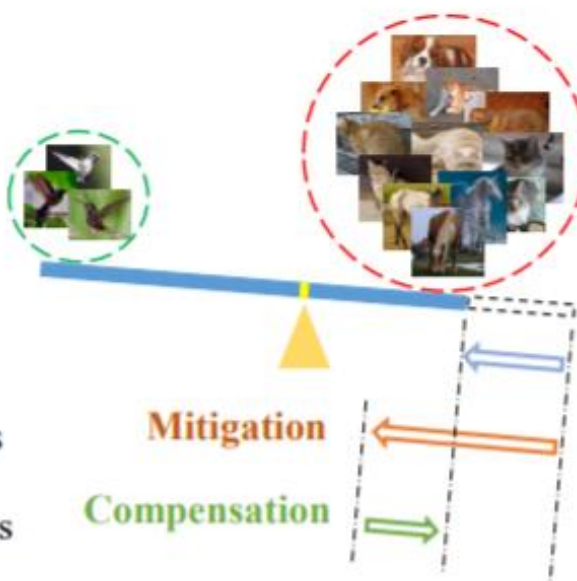
Gupta et al, "LVIS: A Dataset for Large Vocabulary Instance Segmentation", CVPR 2019



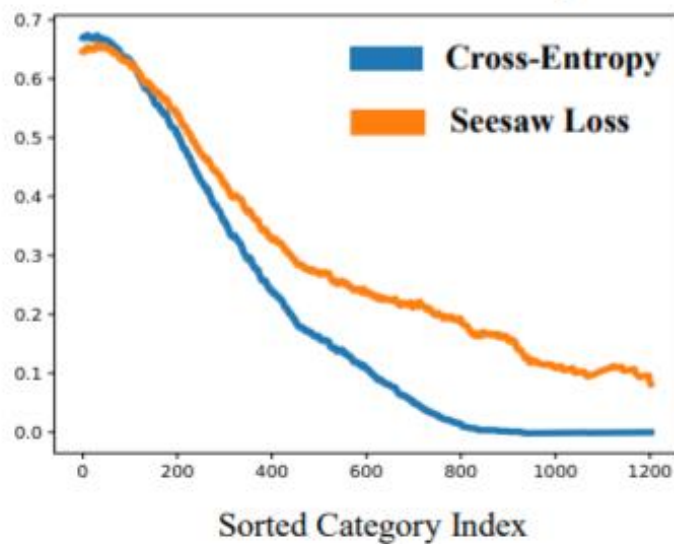
Cross-Entropy Loss



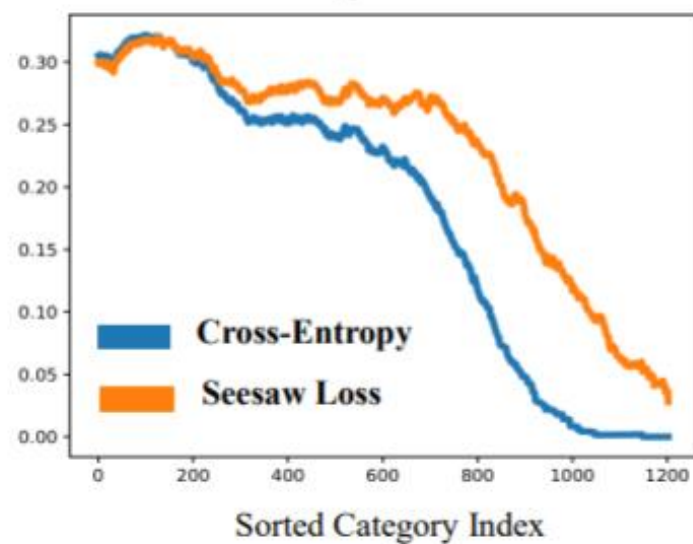
Seesaw Loss



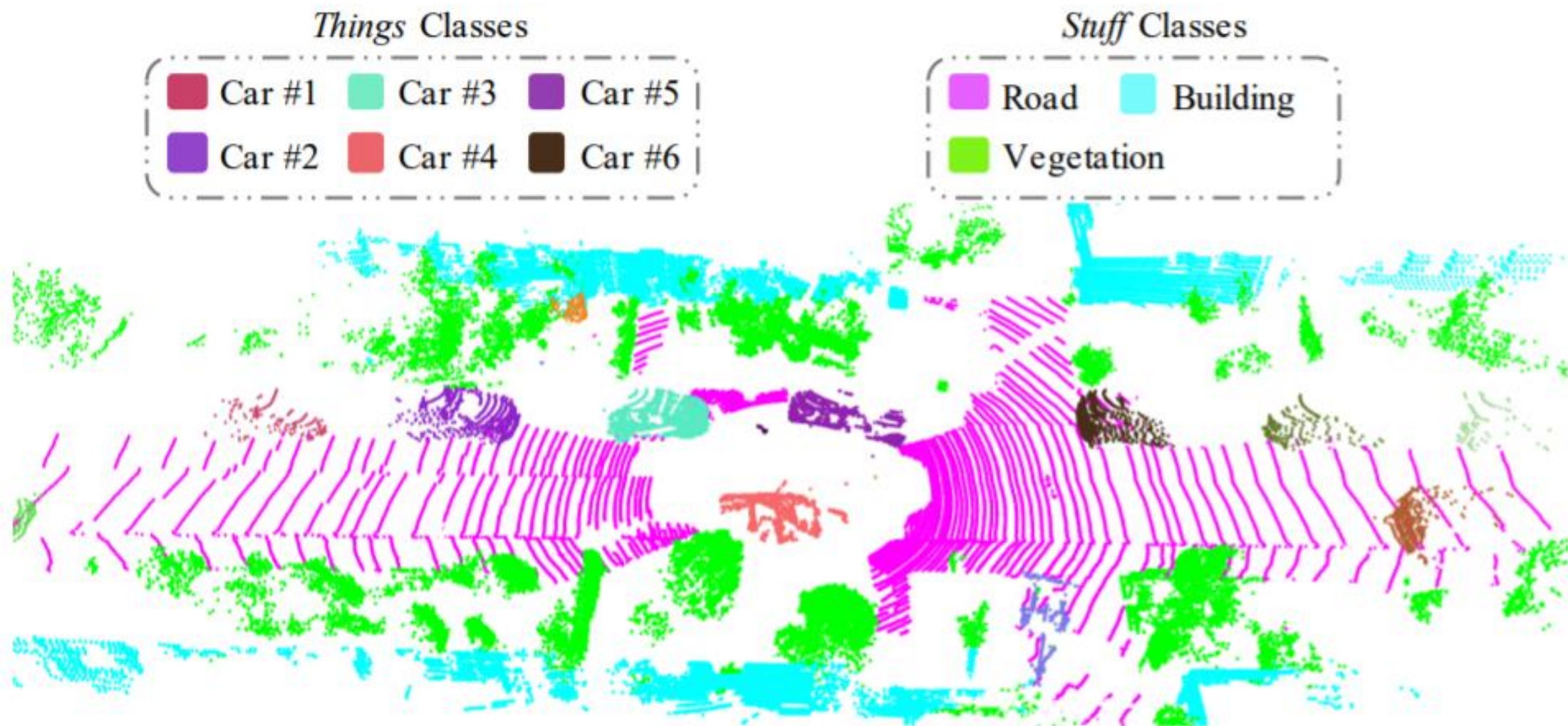
Classification Accuracy



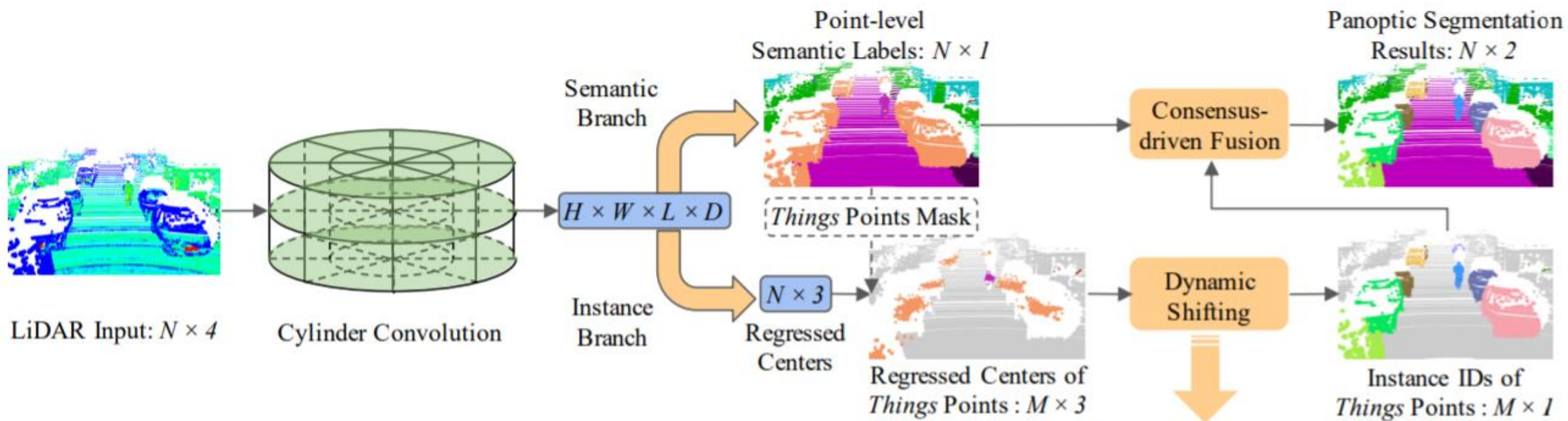
Instance Segmentation AP



Problem II: The Blessing of Dimensionality



(a) LiDAR-based Panoptic Segmentation



Summary: Many Structured Prediction Tasks!

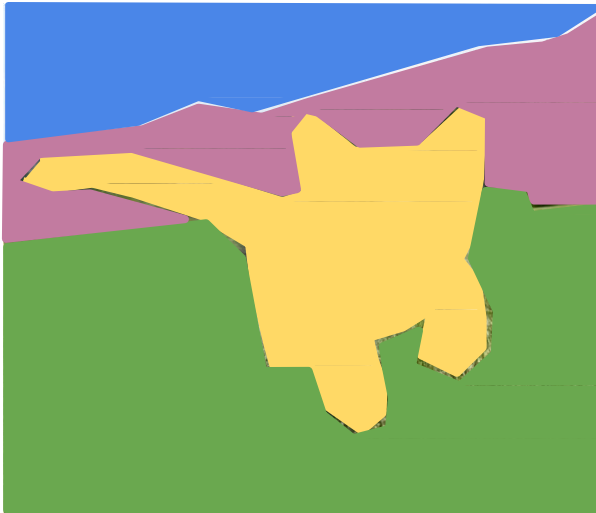
Classification



CAT

No spatial extent

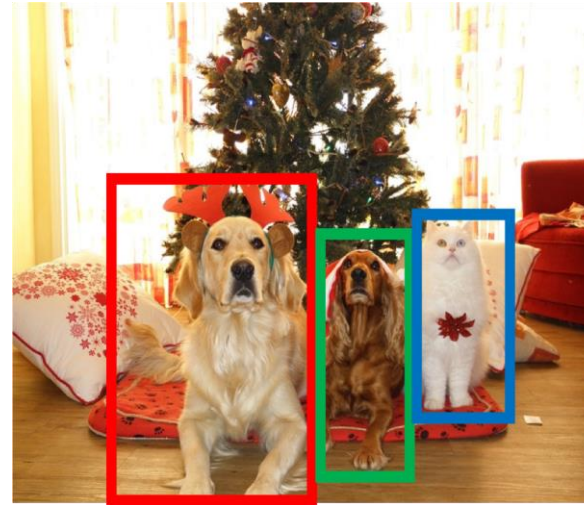
Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation



DOG, DOG, CAT

Deep Structured Prediction: Resources

ICCV Tutorial on Instance-Level Visual Recognition:

<https://instancetutorial.github.io/>

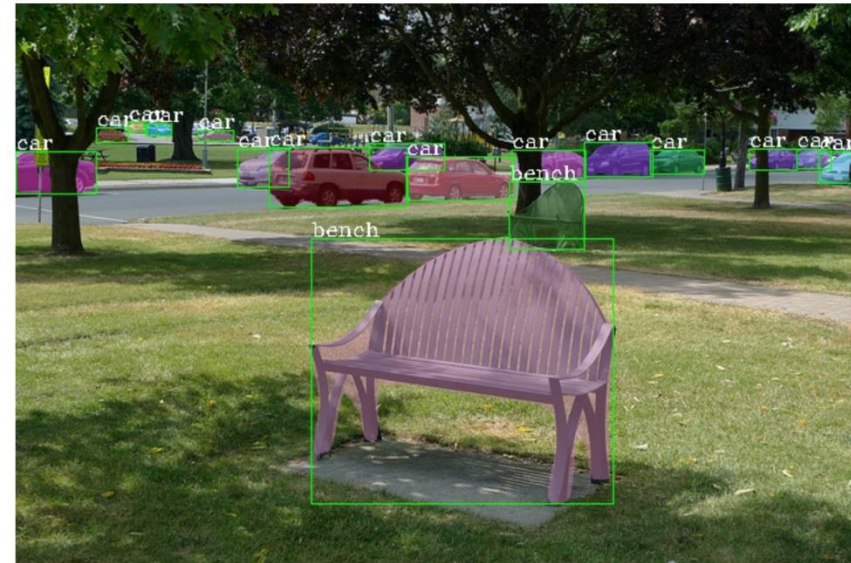
MMDetection:

<https://github.com/open-mmlab/mmdetection>

Introduction

MMDetection is an open source object detection toolbox based on PyTorch. It is a part of the OpenMMLab project developed by [Multimedia Laboratory, CUHK](#).

The master branch works with **PyTorch 1.3 to 1.6**. The old v1.x branch works with PyTorch 1.1 to 1.4, but v2.0 is strongly recommended for faster speed, higher performance, better design and more friendly usage.





Next Time:
Transformers